

Triton中国生态 Meetup（第一期）

主办单位：北京智源人工智能研究院

协办单位：上海人工智能实验室、中国互联网协会人工智能工作委员会、CSDN

2024年6月2日

多元芯片生态与 Triton算子库

- 多元芯片生态共建难题
- Triton可行性
- 基于Triton算子库的统一适配
- FlagGems算子库介绍

多元算力芯片软硬件生态共建难题

开发者的AI算法实现
(NLP、CV、语音、跨模态等)

AI深度学习框架
(PyTorch、Tensorflow、JAX、
PaddlePaddle、Mindspore等)

各种AI算子库

编程语言及编译器

设备端机器码

框架版本更新优先考虑NV生态，竞争性生态通过接口翻译方式保持与CUDA生态的兼容性，并不断完善功能

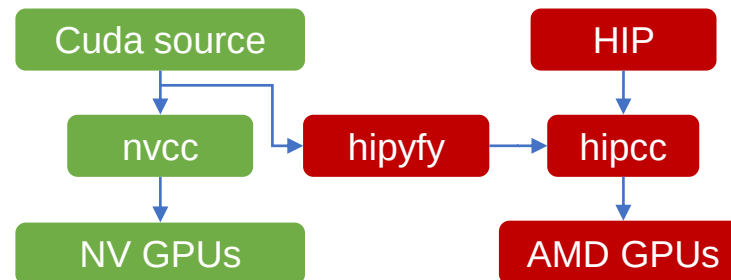
AI框架集成的CUDA高性能算子库
(Pytorch aten, Paddle Phi, etc)

CUDA-X libraries
(cuBLAS, cuDNN, TensorRT, etc)

CUDA 及开发工具链
(nvcc, cuobjdump, nsight, etc)

可在NV GPU上执行的机器代码

NV 生态



AI框架集成的HIP高性能算子库
(Pytorch aten, Paddle Phi etc)

ROCm libraries
(hipBLAS, rocFFT, etc)

ROCm 及开发工具链
(hipcc, amdclang, etc)

可在AMD GPU上执行的机器代码

AMD 生态

多元算力芯片软硬件生态共建难题

开发者的AI算法实现
(NLP、CV、语音、跨模态等)

AI深度学习框架
(PyTorch、Tensorflow、JAX、
PaddlePaddle、Mindspore等)

各种AI算子库

编程语言及编译器

设备端机器码

AI框架集成的CUDA高性能算子库
(Pytorch aten, Paddle Phi, etc)

CUDA-X libraries
(cuBLAS, cuDNN, TensorRT, etc)

CUDA 及开发工具链
(nvcc, cuobjdump, nsight, etc)

可在NV GPU上执行的机器代码

NV 生态

厂商不断维护框架的定制版本，合并主框架的最新改动，维护算子功能一致性，改进算子性能

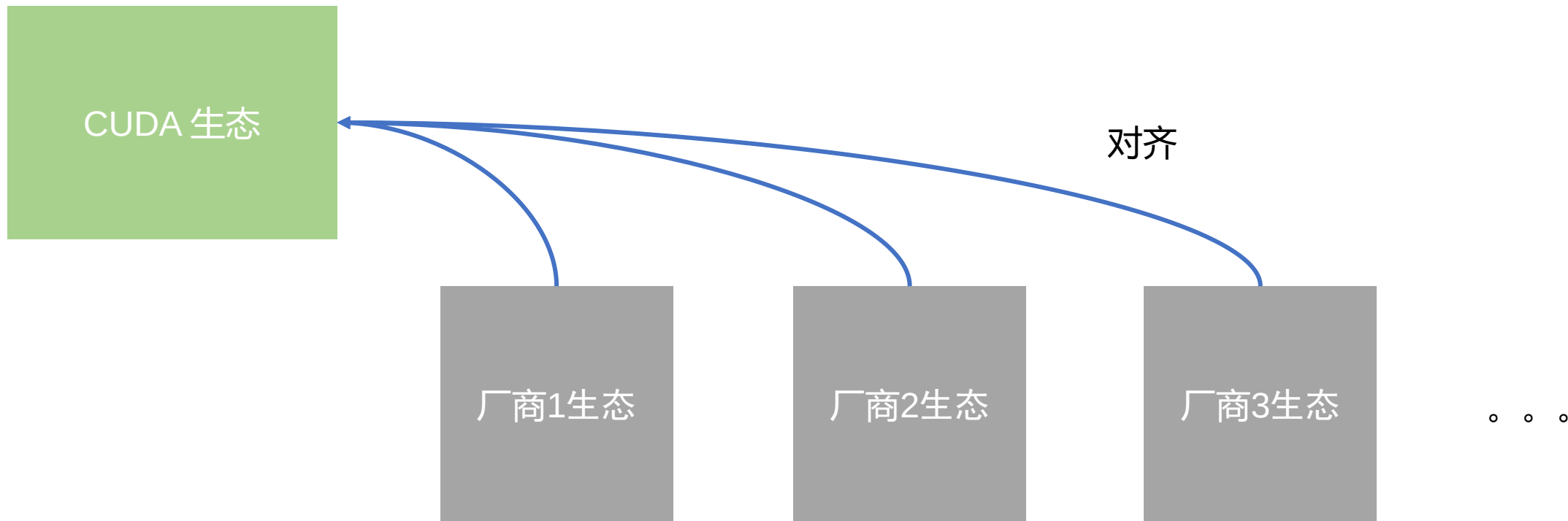
厂商适配的AI框架及框架算子库

厂商自研的加速算子库

厂商自研的芯片编程语言、编译器及开发工具链

可在厂商设备上执行的机器代码

其他厂商生态



- 编程接口限制：CUDA与NV架构存在绑定和共生关系，难以实现完美兼容
- 底层生态隔离：算子库以下各建体系，互补兼容，缺乏协同共建
- 适配负担较重：大量重复开发，注重功能铺量，且需适配多种框架
- 开发难度较高：算子库铺量难以兼顾实现质量，挖掘硬件性能考验编程技巧

- OpenCL
 - 2009年由多家科技公司联合推出，试图建立跨平台计算抽象，替代CUDA
 - OpenCL本身只是标准，被多方利益团体利用成为“跑马圈地”的工具，因此其设计并非针对GPU一种设备，导致目标不够聚焦；
 - 实现质量问题：针对任何一种设备的实际开发人员数量少，版本和功能滞后，测试不充分，bug频出，受人诟病
- ROCm
 - 2016年AMD推出的类CUDA软件套件，试图通过整体“平替”CUDA功能的方式构建AMD的软件生态，为了进一步提升对CUDA的兼容，近几年推出了HIP编程接口
 - 性能和易用性：一路追赶CUDA，但由于功能和性能差距，用户和开发者体验相对英伟达存在巨大差距
 - HIP自动CUDA迁移工具背负巨大兼容负担

多元生态共建需要打通编程接口和算子库



- 多元芯片生态共建难题
- Triton可行性
- 基于Triton算子库的统一适配
- FlagGems算子库介绍

为什么选择Triton?

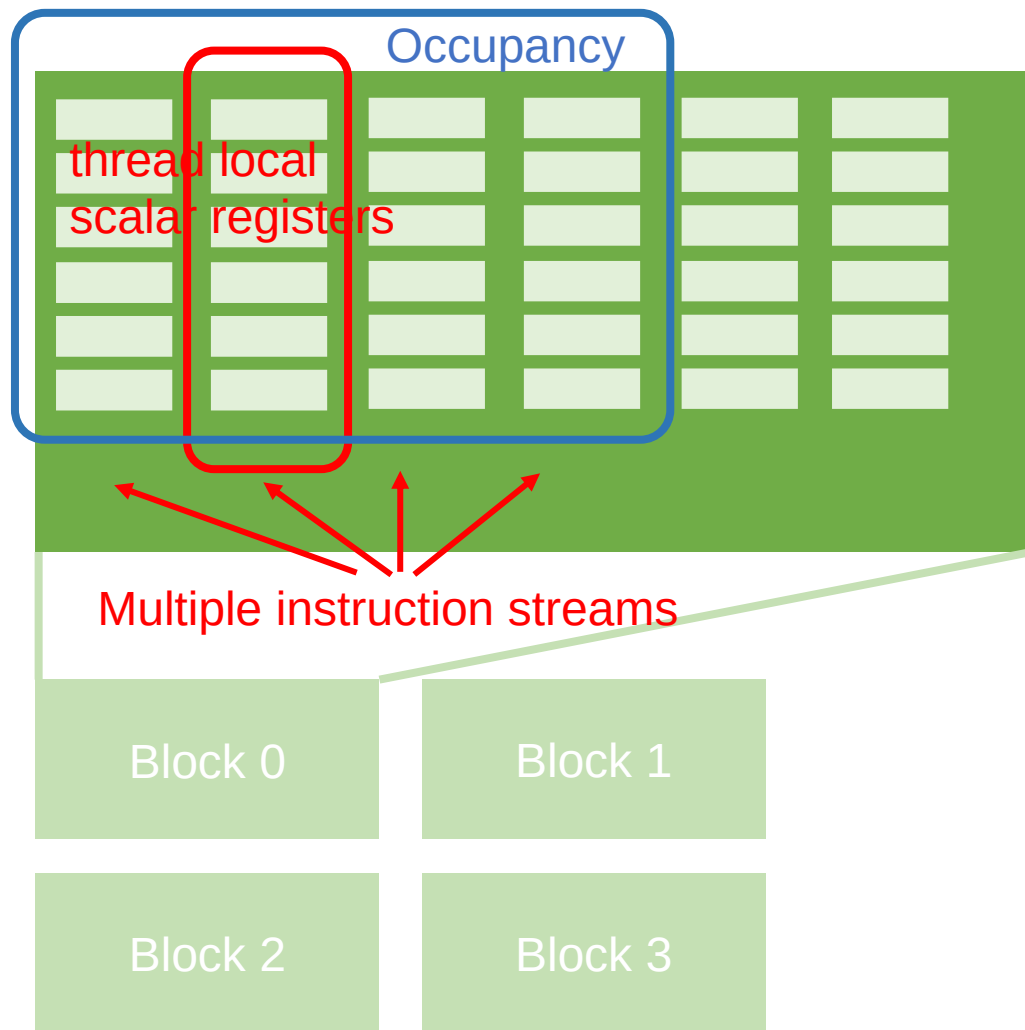
Block > SIMT

Triton开源优势

实测性能

厂商适配现状

为什么选择Triton? 先从SIMT说起



- SIMT既是CUDA编程模型也是GPU架构
 - SIMT优于SIMD，但未必适合DSA架构
 - SIMT消耗更多寄存器资源（高延时，高吞吐）
 - 要求程序保证足够的Occupancy
 - 控制流损害性能
- 差异性
 - 厂商芯片与SIMT架构的差异
 - 厂商芯片与英伟达架构的差异
- 其他
 - 完备性
 - CUDA版本功能演进
 - 迁移性
 - Legacy CUDA程序的迁移问题

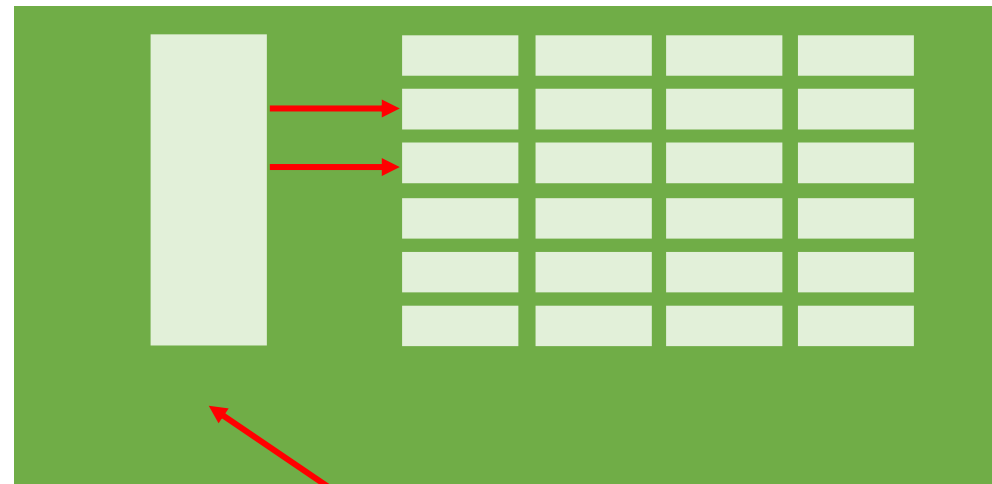
Block vs SIMT vs SIMD

SIMT



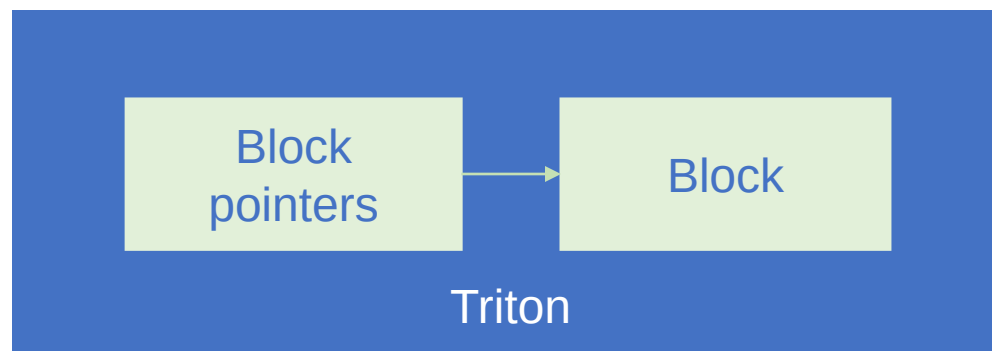
Multiple instruction streams

SIMD



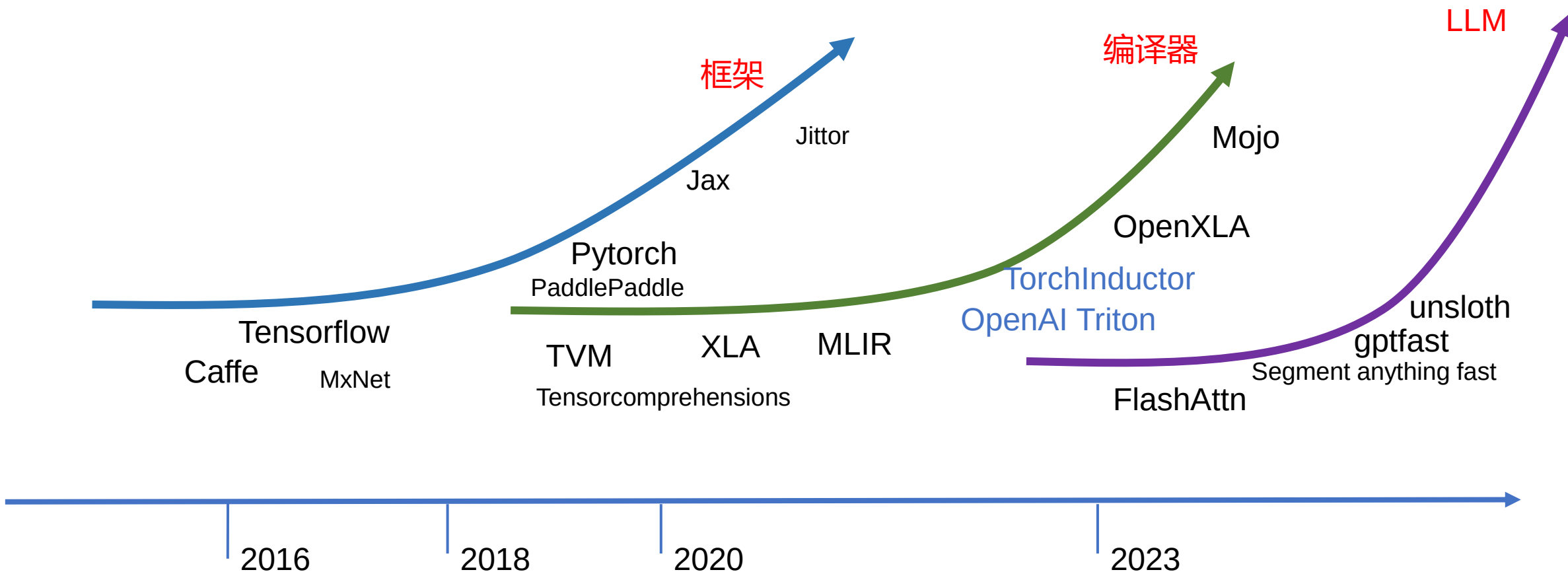
Single instruction stream

Block



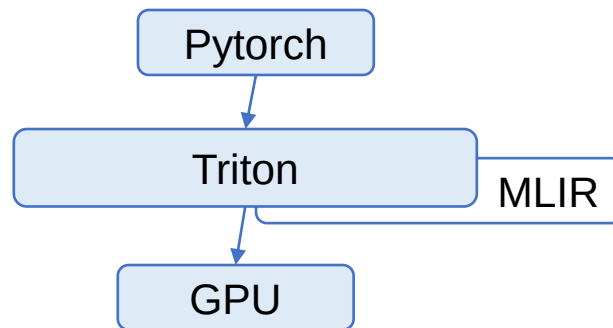
Triton

- Pytorch 框架集成
- 大模型领域的定制化算子



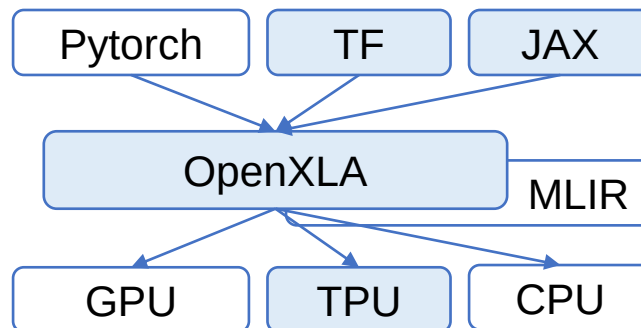
Triton vs 其他开源编程模型

Triton By OpenAI



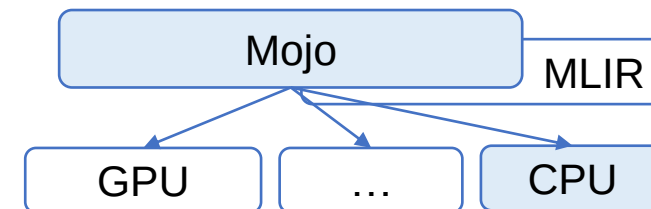
架构先进性: kernel编程领先一代
实现高效性: 领先
实现质量: 较好
易编程: 优于CUDA
适配度: Pytorch框架原生支持

OpenXLA By Google



架构先进性: 整体领先一代, kernel编程缺失
实现高效性: 领先, 但GPU性能低于Triton
实现质量: 较好
易编程: kernel编程模型借鉴Triton
适配度:
TF和JAX支持较好
Pytorch非原生支持, 需要导出模型

Mojo By Modular

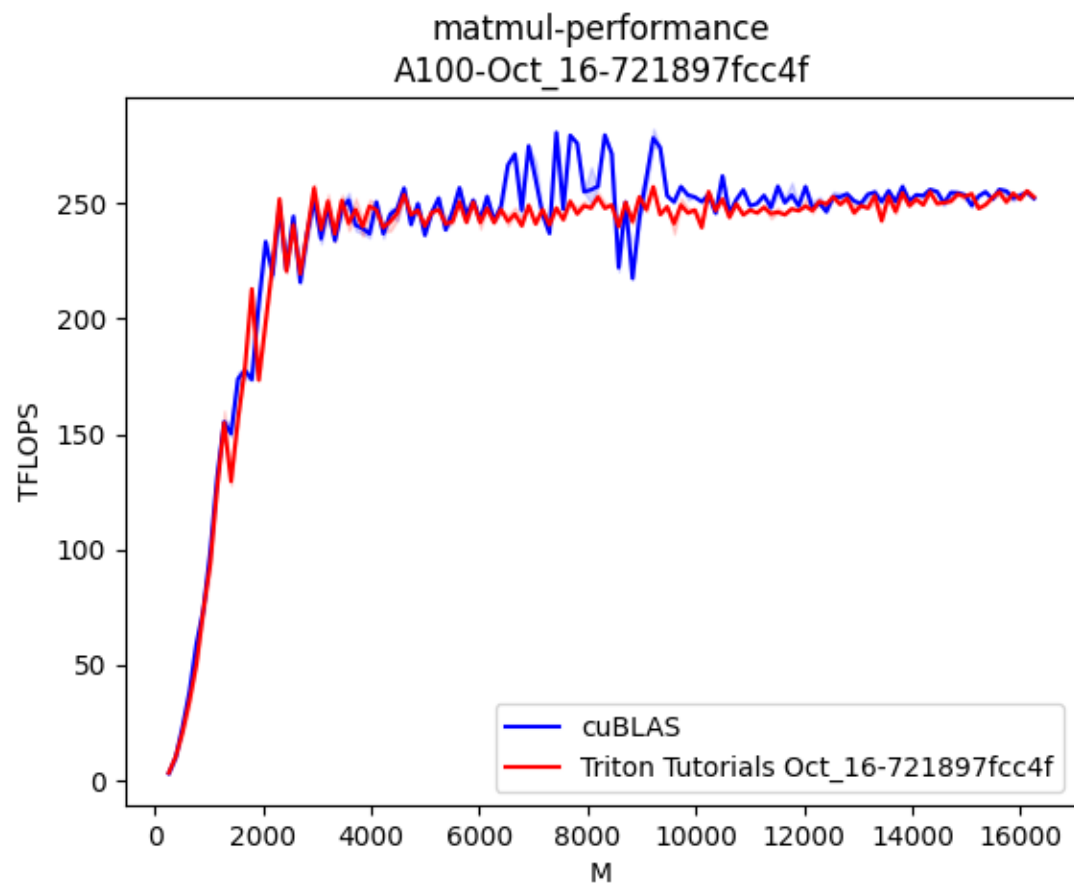


架构领先性: 领先两代
实现高效性: 有欠缺, 目前只支持CPU
实现质量: 目前不开源, 技术班底很硬
易编程: 全新编程模型, 用户反馈不足
适配度: 尚无框架支持

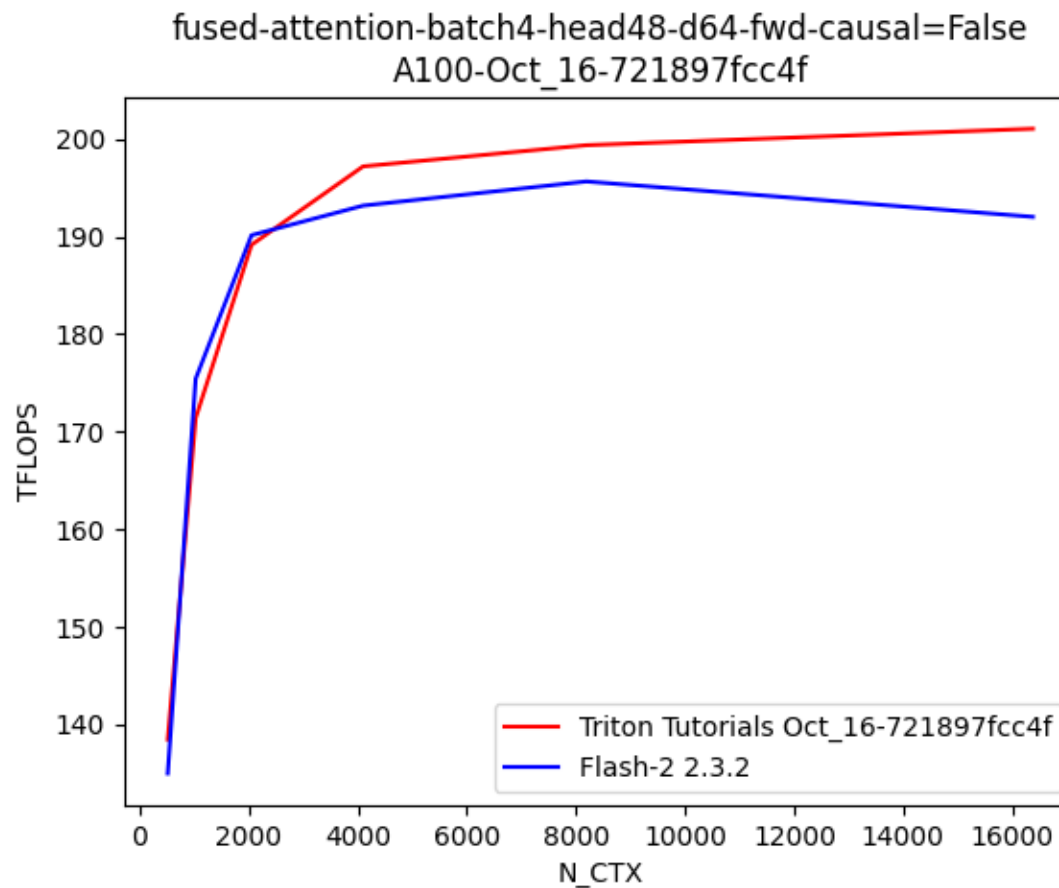
• 衡量标准

- 架构先进性: 技术代际和软件架构的先进性
- 实现高效性: 是否已高效实现各种重要算子和复杂算子, 性能逼近或超过手动优化
- 实现质量: 作为进一步推广和二次开发的软件基础设施质量如何
- 易编程性: 是否能降低编程难度, 有效提升开发生产率
- 适配度: 不同框架的适用性和限制情况, 上层生态的适用广度如何

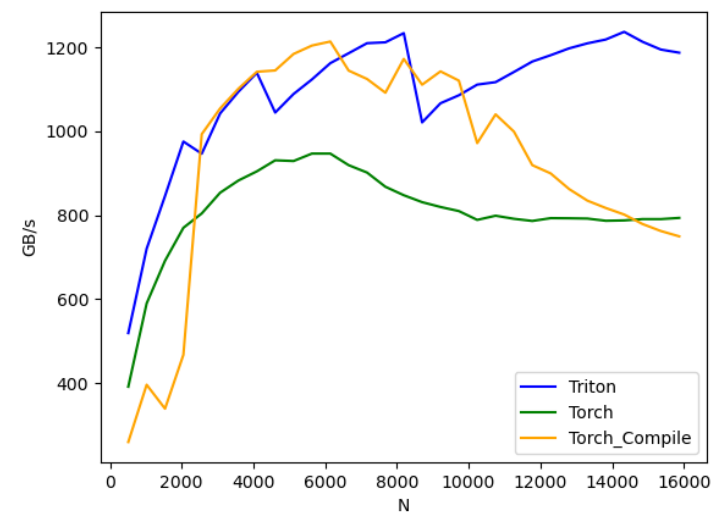
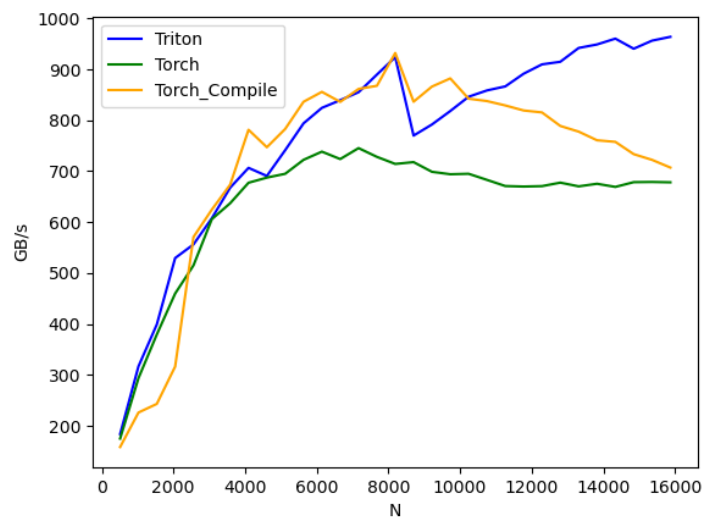
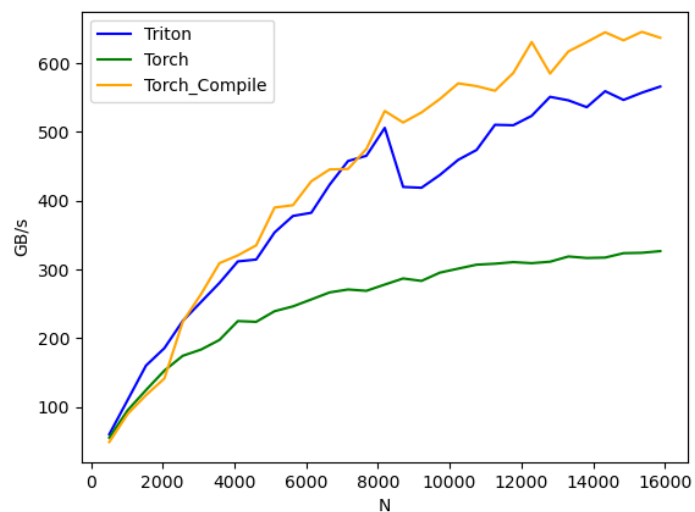
  蓝色表示主要路径



矩阵乘性能



FlashAttention前向性能

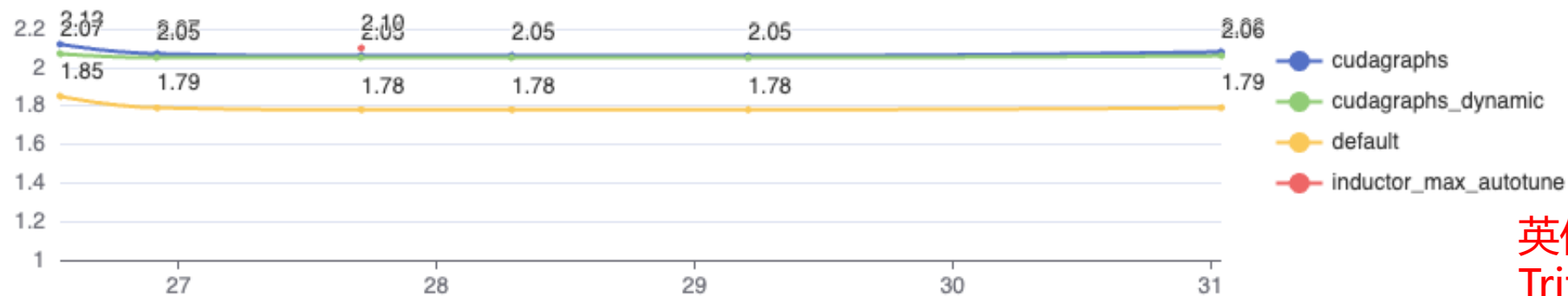


Layernorm性能

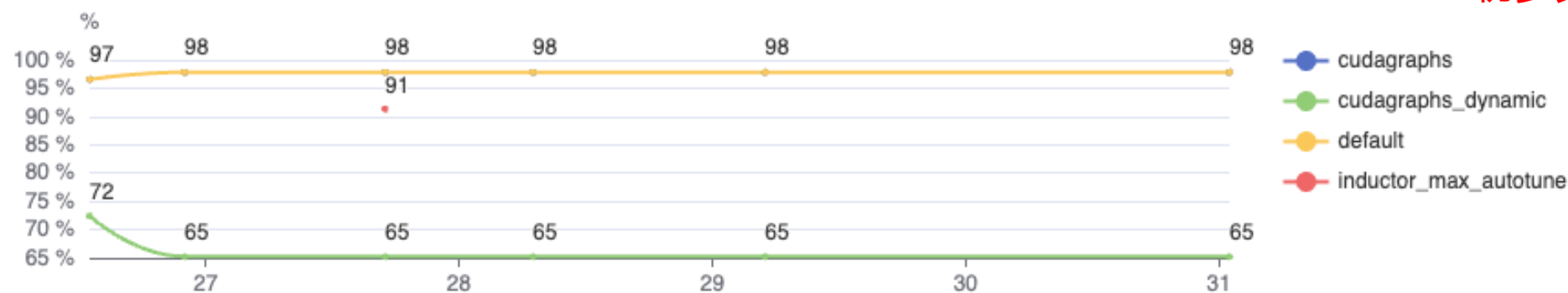


透过 torch compile 管窥 Triton 的潜在模型性能

Geomean / Huggingface



Passrate / Huggingface



英伟达A100上实测性能表明
Triton作为独立算子编程语言
和Inductor代码生成后端，已
初步具备与CUDA相当的性能

Torch compile 性能表现，通过率

Source: <https://hud.pytorch.org/benchmark/compilers>

厂商适配现状

已有数家芯片厂商开展了Triton适配

以torch compile为主要接入点

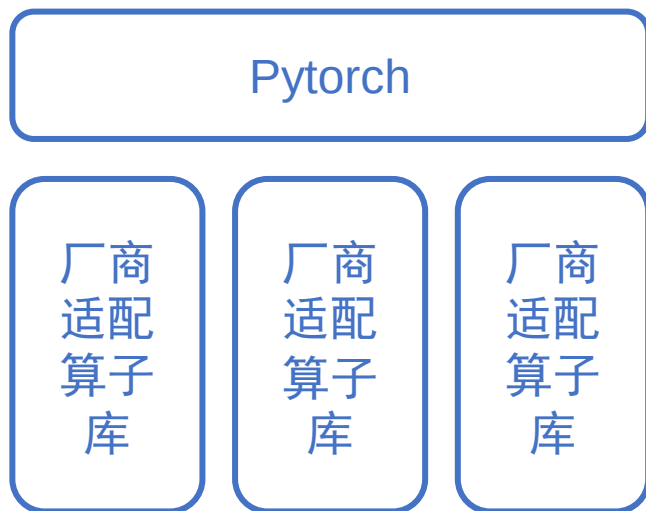
部分LLM模型算子替换率可达90%以上

算子性能仍存在差距，需进一步完善编译优化

- 多元芯片生态共建难题
- Triton可行性
- 基于Triton算子库的统一适配
- FlagGems算子库介绍

多芯片适配路线对比

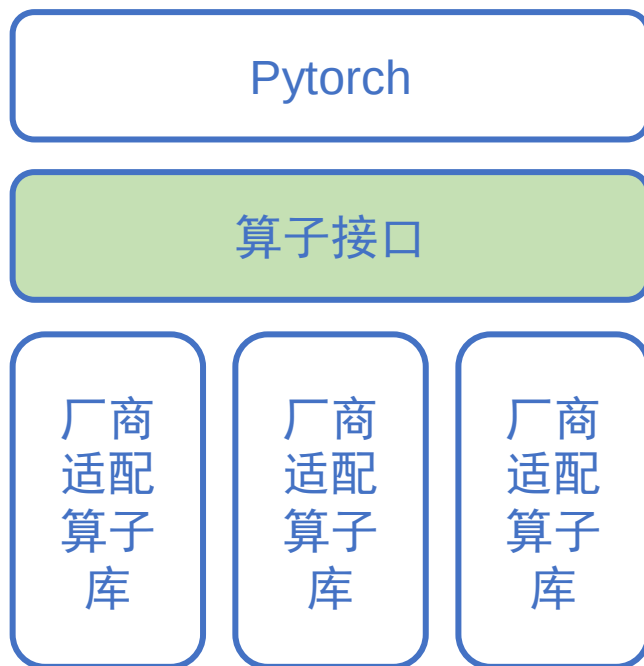
传统厂商适配模式



问题

- 厂商需要开发各自的算子库
- 框架版本不统一
- 算子特性不一致

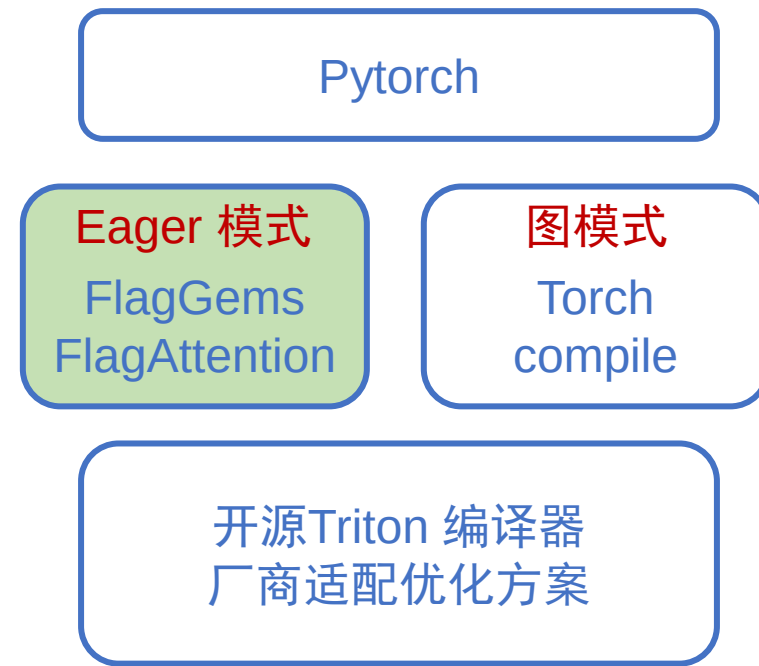
统一算子接口定义模式



问题

- 厂商需要开发各自的算子库
- 不支持torch compile, 无法进行图优化
- 仅统一接口, 算子特性不一致

统一的开源算子库+编译器模式



收益

- 厂商共享算子库, 无需独立开发
- Eager动态图和compile两种执行模式
- 算子库源码共享, 算子实现一致性高
- Triton抽象层次高, 便于开展编译优化

多源现状限制了 Triton算子的适用性

- Triton自带算子，数量少，接口与框架不一致
- Pytorch框架自动生成的elementwise和reduction以IR生成为主，依赖torch compile过程
- 第三方加速库，如flash-attn等，来源分散，接口不一

覆盖面低使其无法成为模型训练的主要算子来源

- Triton仍然多用于快速定制化实现Flash attention等特殊加速算子及fused activation dropout等融合算子
- 扩大覆盖面有助于加速替代现有CUDA算子

为了弥补现有Triton开源生态的不足，更好的支持多元芯片大模型训练平台，需要构建满足如下要求的新型Triton算子库：

- 统一仓库，开源共建，有助于功能对齐，减少重复投入；
- 统一接口，面向Pytorch框架对齐；
- 算子覆盖度高，支持典型大模型训练的全流程；
- 算子性能接近现有算子库水平；
- 支持多种异构算力芯片；

FlagGems算子库的设计初衷

- 能自动、透明接入Pytorch, 减少模型适配者的心智负担

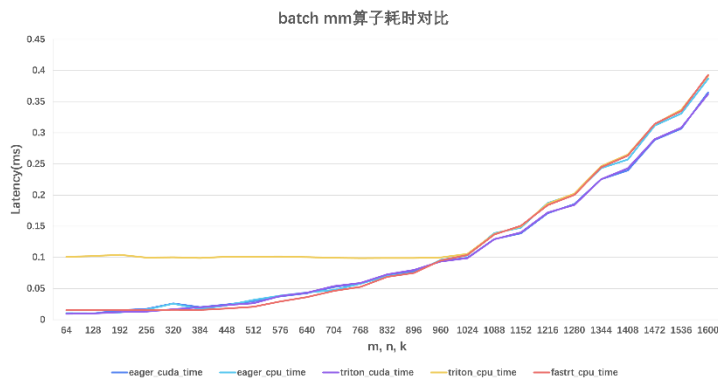
```
import flag_gems
flag_gems.enable()
```

```
import torch
import flag_gems
```

```
M, N, K = 1024, 1024, 1024
A = torch.randn((M, K), dtype=torch.float32)
B = torch.randn((K, N), dtype=torch.float32)
with flag_gems.use_gems():
    C = torch.mm(A, B)
```

透明替换aten算子, 用户模型无需修改代码

- 无需torch compile, 算子端到端性能与eager原有性能齐平



FlagGems改进了runtime和caching机制, 达到更高效的端到端表现

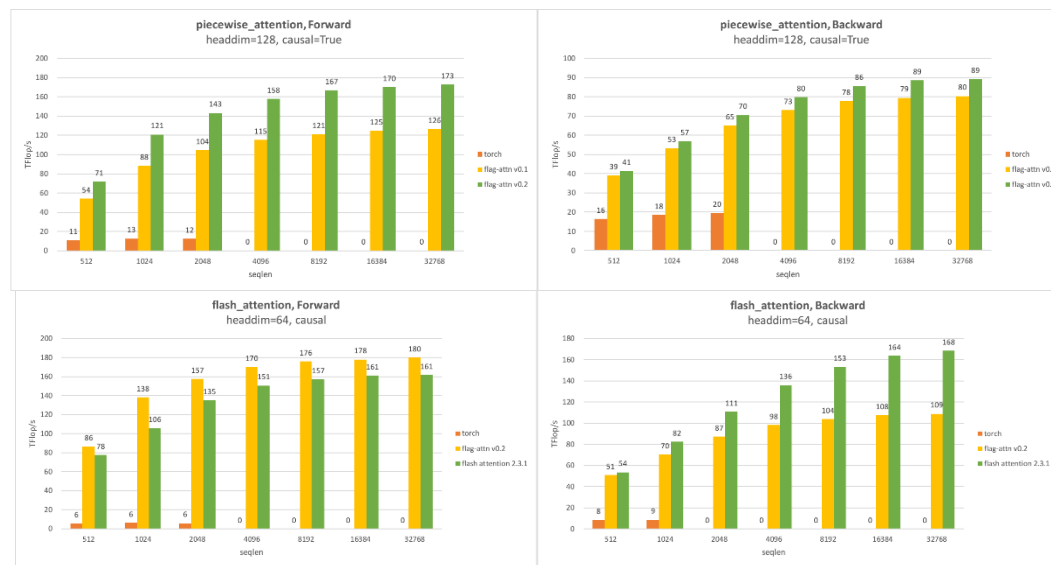
- 大量替换现有Pytorch算子, 优先满足大模型训练需要

llama2, <https://huggingface.co/meta-llama/Llama-2-7b-hf>
gpt2, <https://huggingface.co/openai/gpt2>
Mixtral, <https://huggingface.co/mistralai/Mixtral-8x7B-v0.1>
ChatGlm, <https://huggingface.co/THUDM/chatglm3-6b>
Phi2, <https://huggingface.co/microsoft/Phi-2>
Bert base-uncased, <https://huggingface.co/Bert-BASE-uncased>
Qwen-7b, <https://huggingface.co/Qwen/Qwen-7B>
LLava-7b, <https://huggingface.co/LLaVA-7B>

从典型模型中梳理了127个高频算子作为首要实现的目标



- 目标:
 - 面向大模型的定制化算子高效实现
 - 快速验证大模型领域新的算法实现机制
- 近似对标: xformers, unsloth
- 目前已实现特性:
 - 自定义 piecewise_attention
 - Flash attention
 - Total attention
 - Flash decoding, split kv
 - Kv cache, paged attention
 - MQA, GQA
 - Dropout



- 性能显著优于 pytorch 实现, 而且 v0.2 比 v0.1 性能进一步提升

- 多元芯片生态共建难题
- Triton可行性
- 基于Triton算子库的统一适配
- FlagGems算子库介绍

FlagGems

High-Performance Triton Library



<https://github.com/FlagOpen/FlagGems>

一个使用OpenAI Triton语言实现的算子库

高性能

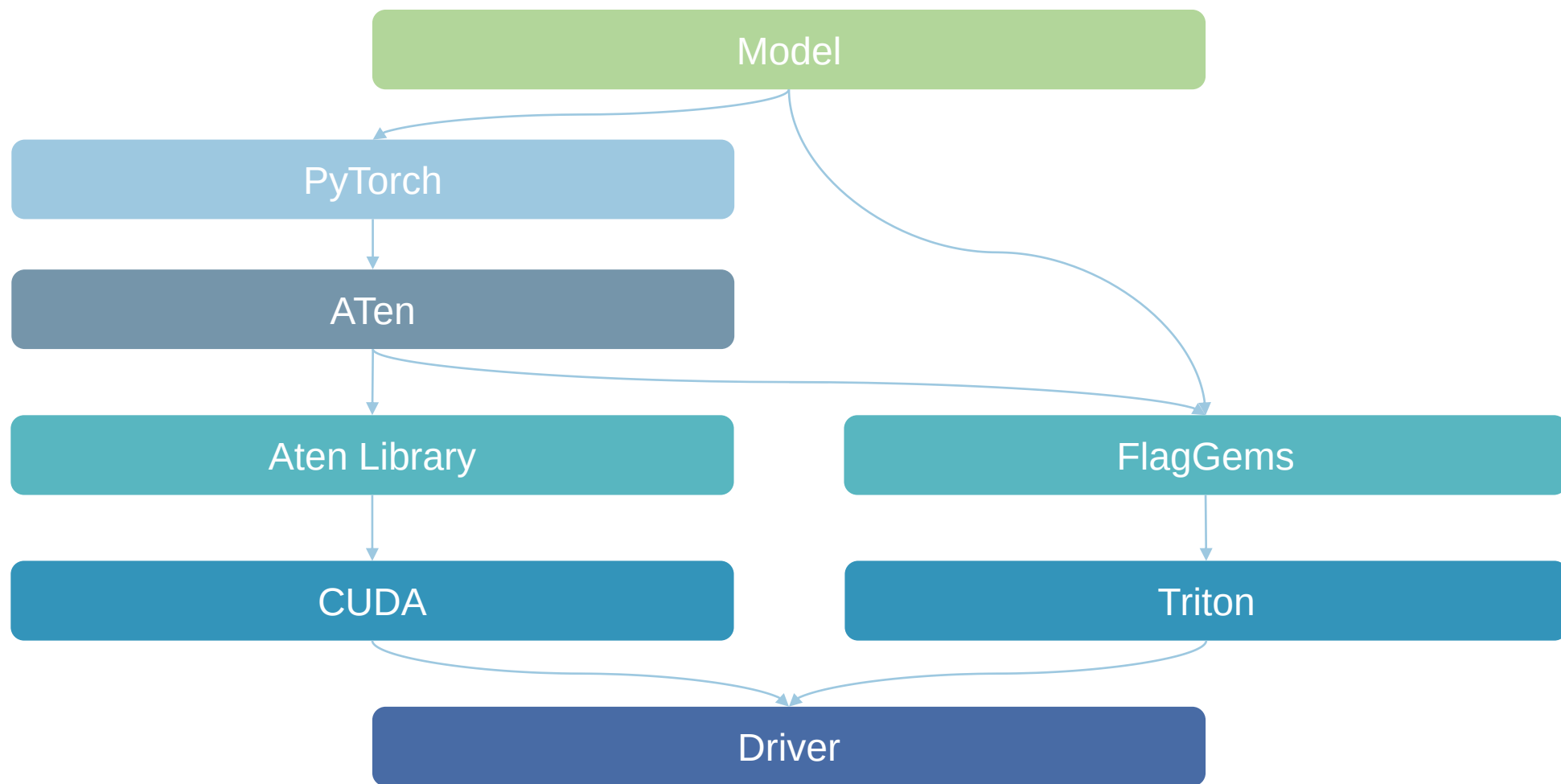
通用算子性能与torch eager持平
融合算子性能优于torch eager

广覆盖

V2.0共实现66个算子，替换76个Aten接口
预计V3.0完成133+算子，基本覆盖大模型需求

轻量级

多款硬件支持，涵盖NVIDIA与国产芯片
安装使用简便，无缝嵌入PyTorch
开发优化效率高，快速更新迭代



➤ 在进程中永久启用

```
import torch
import flag_gems
flag_gems.enable()

M, N, K = 1024, 1024, 1024
A = torch.randn((M, K), dtype=torch.float16, device="cuda")
B = torch.randn((K, N), dtype=torch.float16, device="cuda")
C = torch.mm(A, B)
```

➤ 在进程局部临时启用

```
import torch
import flag_gems

M, N, K = 1024, 1024, 1024
A = torch.randn((M, K), dtype=torch.float16, device="cuda")
B = torch.randn((K, N), dtype=torch.float16, device="cuda")
with flag_gems.use_gems():
    C = torch.mm(A, B)
```

➤ 直接调用

```
import torch
import flag_gems

M, N, K = 1024, 1024, 1024
A = torch.randn((M, K), dtype=torch.float16, device="cuda")
B = torch.randn((K, N), dtype=torch.float16, device="cuda")
C = flag_gems.mm(A, B)
```

LinearAlgebra

BLAS	Reduction	
addmm	cumsum	amax
bmm	mean	argmax
linear*	sum	max
matmul**	prod	min
mm	var_mean	
mv		
outer		

NeuralNetwork

BasicMath

Fusion

Backward

* linear底层调用addmm实现

** matmul底层调用bmm, mm, mv实现

LinearAlgebra

NeuralNetwork

Activation	Normalization	Other
gelu	layer_norm*	dropout***
relu	group_norm**	triu
silu	vector_norm	cross_entropy_loss
softmax	rms_norm	native_dropout
log_softmax	native_layer_norm	
sigmoid	native_group_norm	

BasicMath

Fusion

Backward

* layer_norm底层调用native_layer_norm实现

** group_norm底层调用native_group_norm实现

*** dropout底层调用native_dropout实现

LinearAlgebra

NeuralNetwork

BasicMath

Mathmatical		Logical		Reduction
add	abs	bitwise_and	eq	all
div	exp	bitwise_not	ge	any
sub	reciprocal	bitwise_or	gt	
mul	rsqrt	__or__	le	
pow	neg	isinf	lt	
rsub	cos	isnan	ne	
clamp	sin			
	tanh			

Fusion

Backward

FlagGems算子清单

LinearAlgebra

NeuralNetwork

BasicMath

Fusion

Mathematical

Pointwise

Attention

skip_layernorm

gelu_and_mul

rotary_position_embedding

skip_rmsnorm

silu_and_mul

Backward

LinearAlgebra

NeuralNetwork

BasicMath

Fusion

Backward*

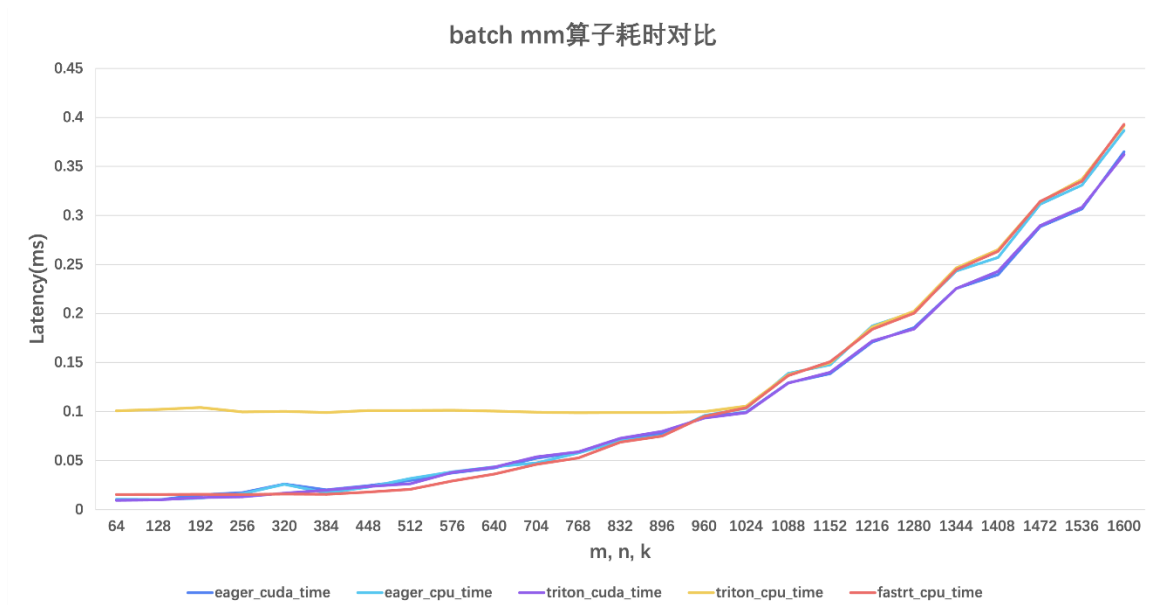
Pointwise	Reduction	Other
native_dropout_backward	native_layer_norm_backward	outer_backward
threshold_backward	_softmax_backward_data	cross_entropy_loss_backward
silu_backward	_log_softmax_backward_data	
tanh_backward	native_group_norm_backward	
sigmoid_backward		

* Backward算子未计入总数

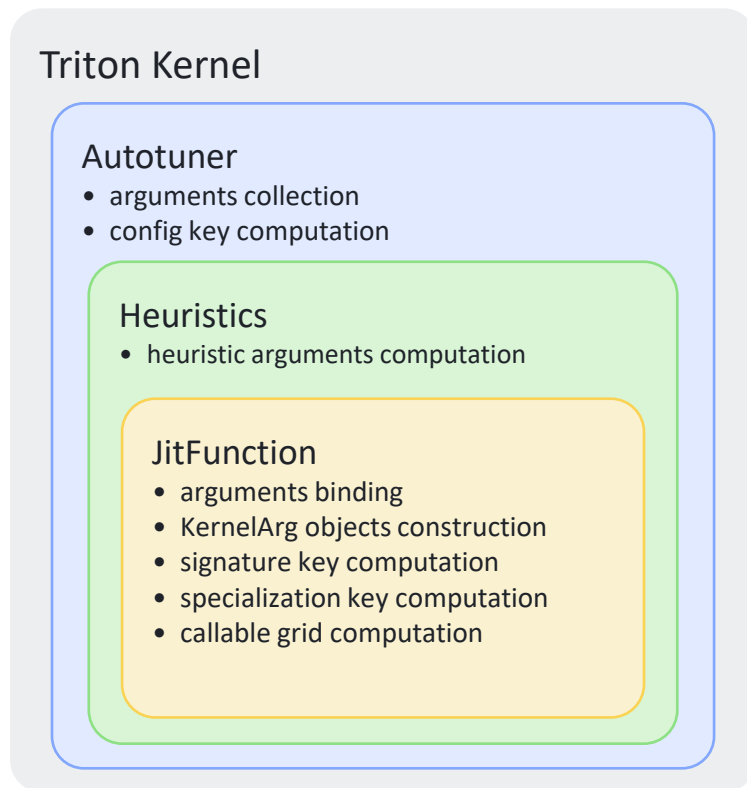
Triton v2 Runtime开销

- 以batch matmul算子为例
 - A100平台
 - 取batch=10, dtype=torch.float16
 - 充分warmup和repeat
- 优化的triton kernel可以达到与eager kernel持平的性能
- 而triton的CPU的端到端开销远高于torch eager
 - 大规模算子开销接近
 - 小规模算子上差距更加明显
- fastrt: kernel外置, 运行时只计算最简cache key

triton kernel的运行时空开销是性能差距的主要原因



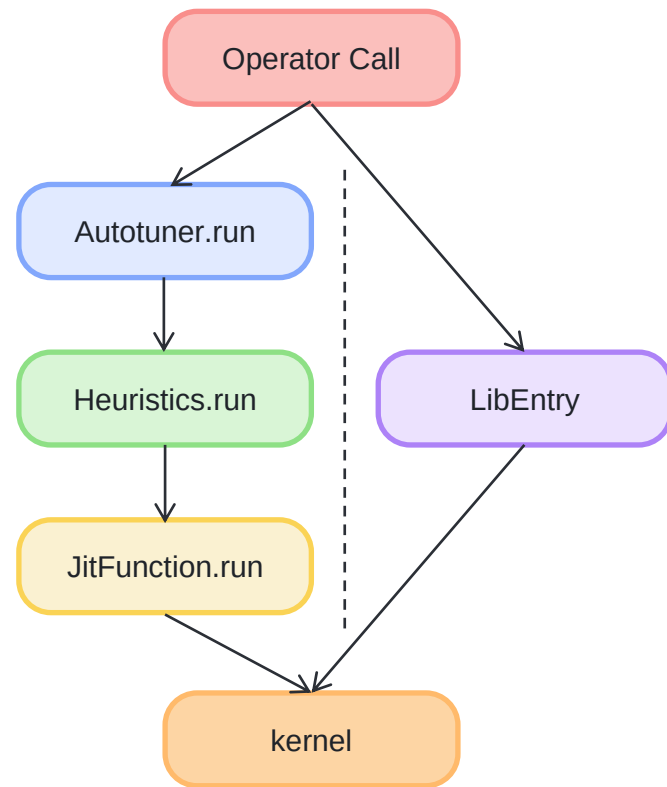
- 运行时开销开源
 - Autotuner
 - Heuristics
 - JitFunction
- 调参之后的每次执行都会调用所有层次装饰器的run函数
- JitFunction.run是主要瓶颈
 - 参数绑定调用python inspect
 - KernelArg类型包装
 - cache key计算



```
1 @triton.autotune(  
2     configs=configs(),  
3     key=["M", "N", "K"]  
4 )  
5 @triton.heuristics(  
6     {  
7         "DIVISIBLE_M": lambda args: args["M"] % args["TILE_M"] == 0,  
8         "DIVISIBLE_N": lambda args: args["N"] % args["TILE_N"] == 0,  
9         "DIVISIBLE_K": lambda args: args["K"] % args["TILE_K"] == 0,  
10    }  
11 )  
12 @triton.jit  
13 def bmm_kernel(*args):  
14     ...
```

- 策略
 - 构造LibEntry独立维护kernel cache
 - 绕过Autotuner、Heuristics和JitFunction的runtime
- 功能
 - 仅需在triton kernel前装饰即可
 - 支持Autotuner、Heuristics、JitFunction的直接包装
 - 首次执行时进入原调用路径，不影响调参功能的正常使用
- 优化
 - 无需经过运行时类型的嵌套调用
 - 节省了重复的参数处理，无需绑定和类型包装
 - 简化了cache key格式，减少不必要的键值计算

```
1 @libentry()
2 @triton.autotune(...)
3 @triton.heuristics(...)
4 @triton.jit
5 def bmm_kernel(*args):
6     ...
```



- 效果
 - cpu端到端显著低于未优化triton，节省近70%
 - runtime耗时无法完全消除，只能逼近torch eager

- 局限

- 必需键值比fast rt实验场景更复杂
- 相比原cache key，存在一定的信息损失
- 独立的kernel cache额外占用了存储空间
- 只在小规模算子上效果明显

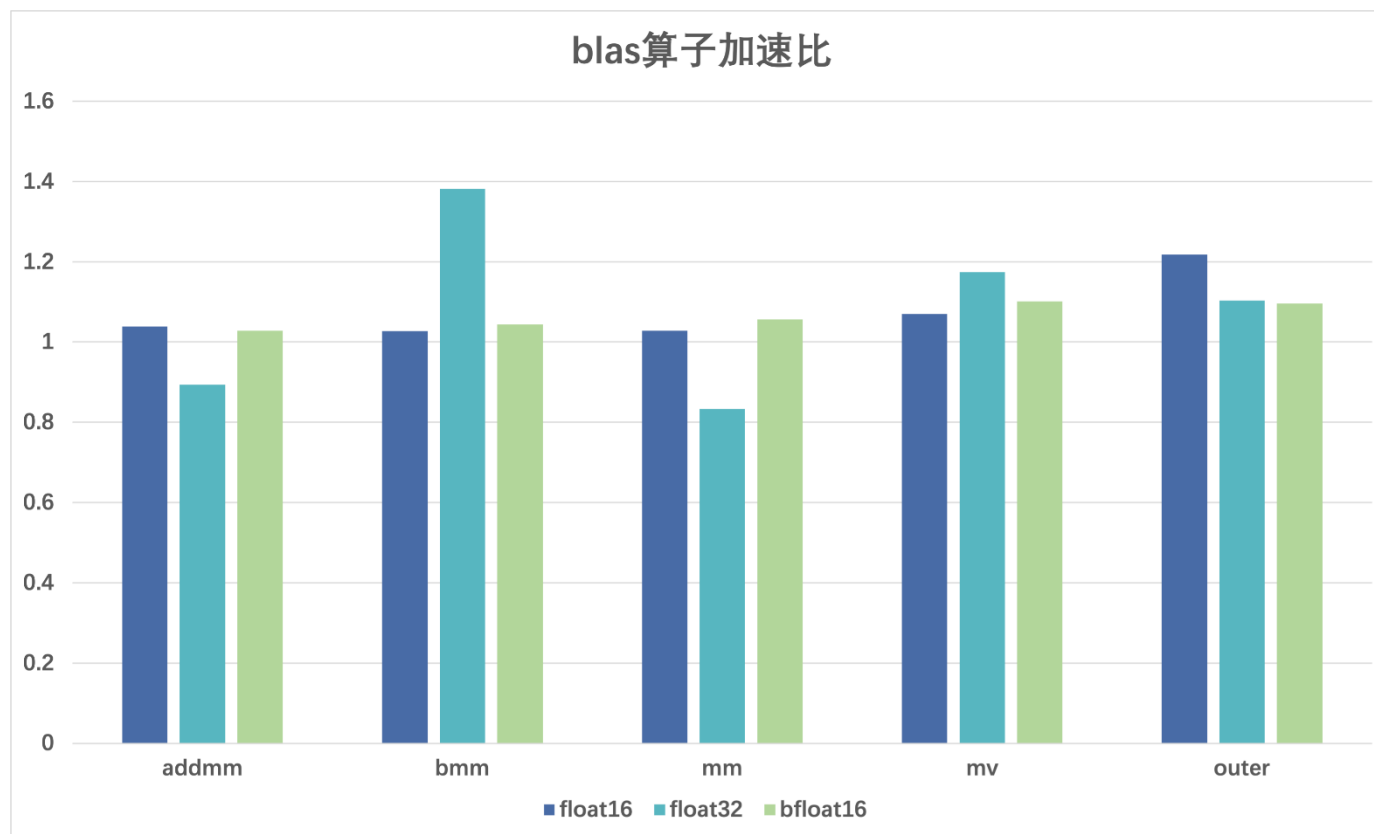
大规模算子的cuda发射周期短于kernel执行周期

瓶颈不在runtime



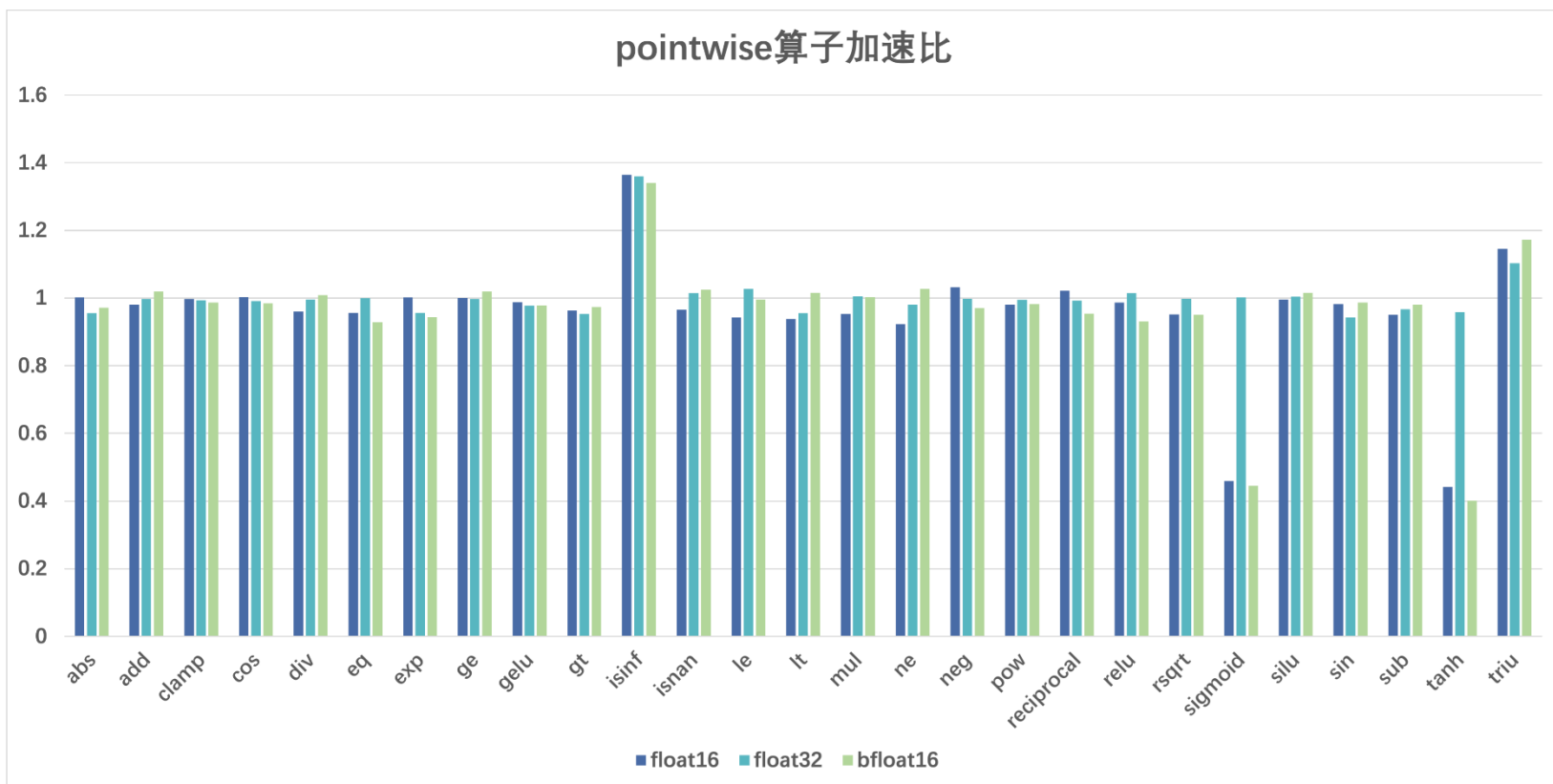
FlagGems与Aten Kernel性能对比

- 计算密集型：blas线性代数计算
 - 单精度计算性能接近cuda
 - 半精度计算性能追平cuda



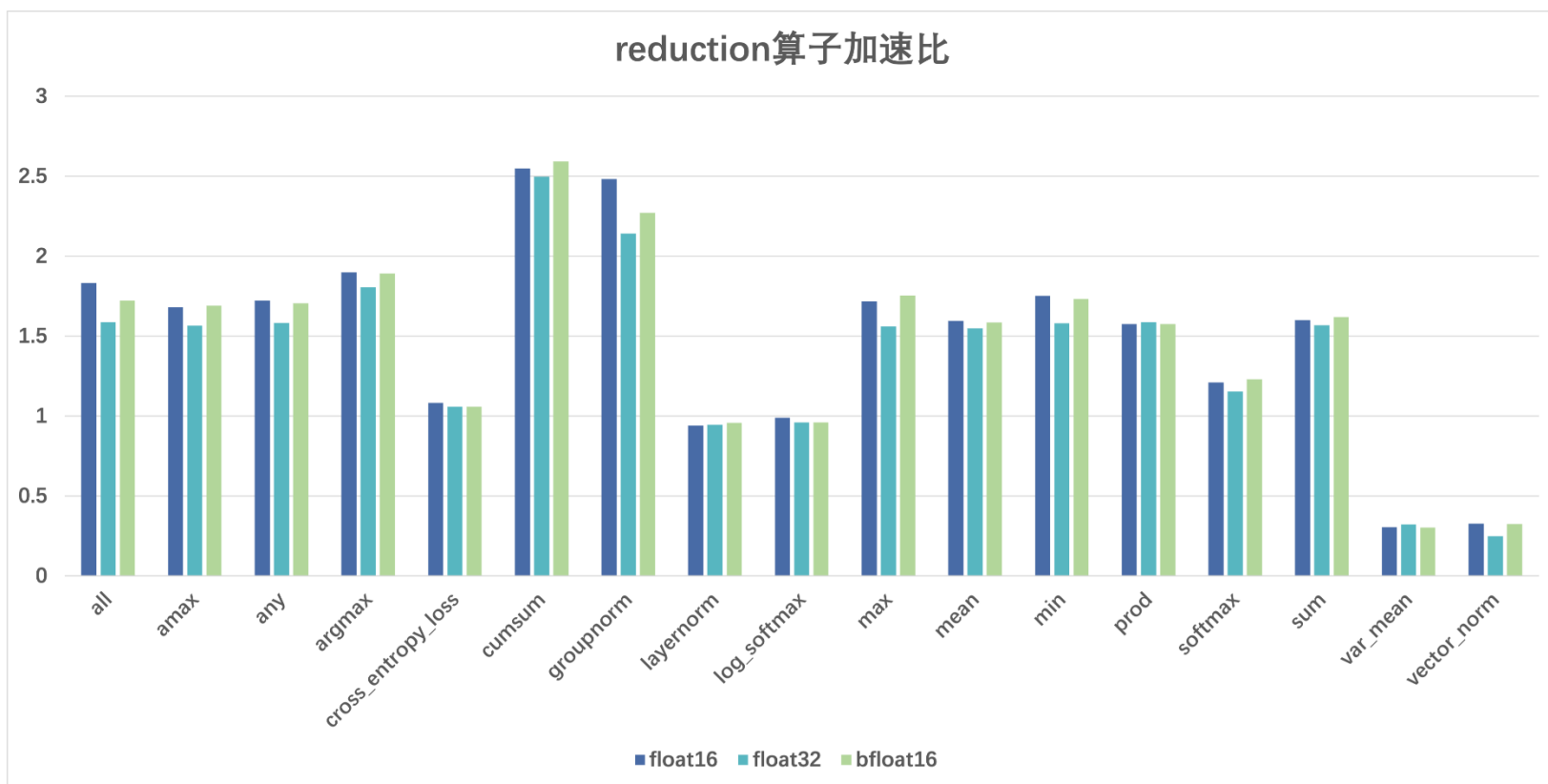
FlagGems与Aten Kernel性能对比

- 访存密集型：pointwise对位计算
 - 多数算子各种精度计算性能基本追平cuda
 - 少数算子受triton language功能接口的数据类型限制，upcast会增大半精度的时间成本



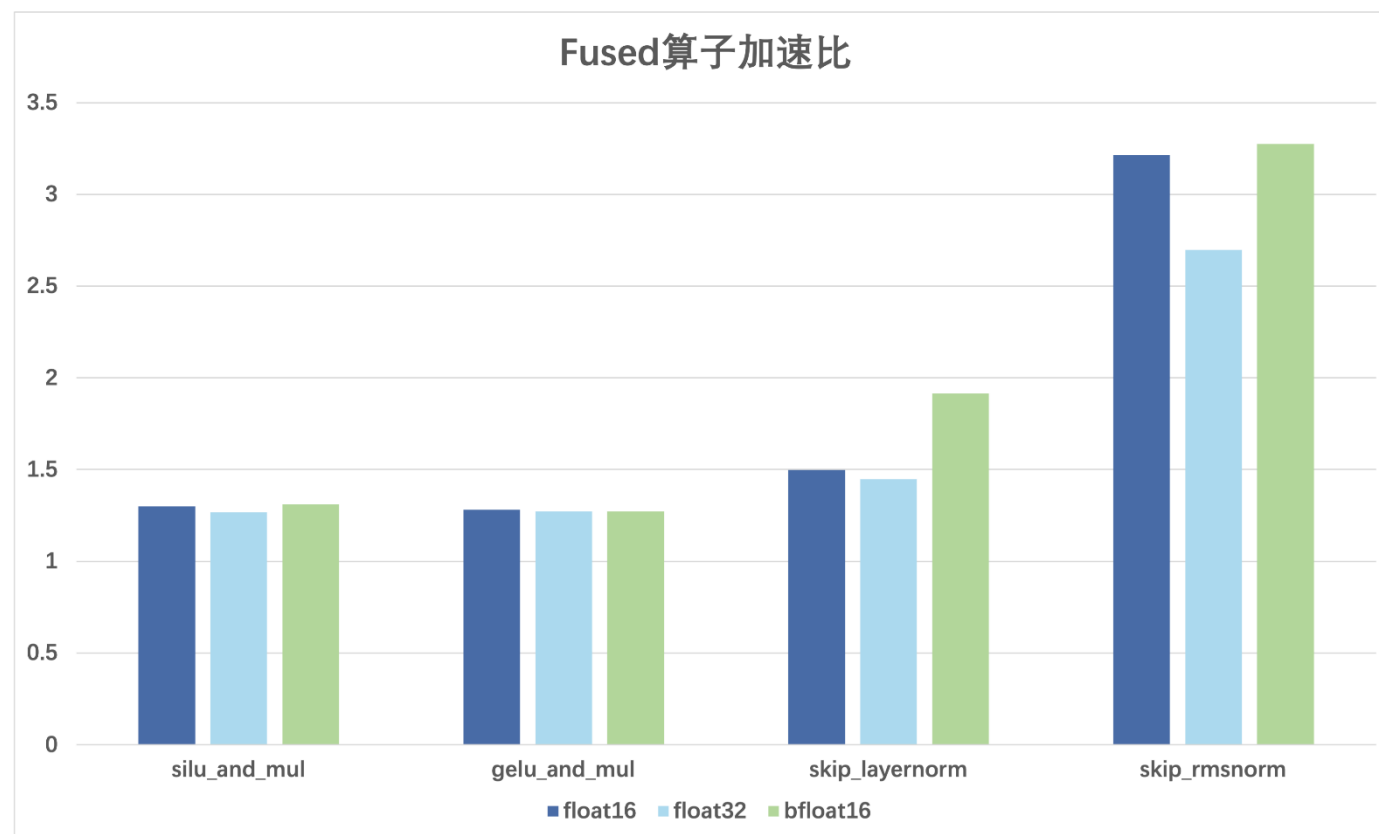
FlagGems与Aten Kernel性能对比

- 访存密集型：reduction归约计算
 - 多数算子各精度计算性能超过cuda
 - 少数算子计算性能劣于cuda，仍需进一步优化



FlagGems与Aten Kernel性能对比

- 融合算子
 - 全部算子、各种精度计算性能超过cuda





谢谢聆听!

欢迎访问开源社区:

<https://github.com/FlagOpen/FlagGems>

<https://github.com/FlagOpen/FlagAttention>

Triton中国生态Meetup反馈收集表



Triton中国社区微信群



扫码添加微信, 私信“加群”

FlagGems代 码 生 成 技 术 讲 解

Pointwise 算子自动代码生成

01 为什么使用 Code Generation

02 核心: PointwiseDynamicFunction

03 实现机制

04 性能分析

05 总结与展望

如何用 triton 写一个功能较全的 pointwise 算子。从最简单的 vector add 开始。

```
@triton.jit
```

```
def add_kernel(a_ptr, b_ptr, o_ptr, N, TILE_SIZE: tl.constexpr):
```

```
    pid = tl.program_id(0)
```

```
    offsets = pid * TILE_SIZE + tl.arange(0, TILE_SIZE)
```

```
    mask = offsets < N
```

```
    a = tl.load(a_ptr + offsets, mask=mask)
```

```
    b = tl.load(b_ptr + offsets, mask=mask)
```

```
    o = a + b
```

```
    tl.store(o_ptr + offsets, o, mask=mask)
```

```
def add(a, b):
```

```
    o = torch.empty_like(a)
```

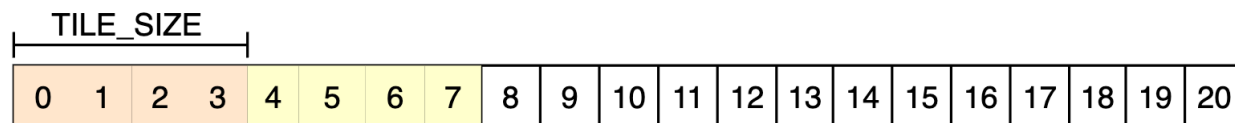
```
    N = a.numel()
```

```
    TILE_SIZE = 512
```

```
    grid = triton.cdiv(N, TILE_SIZE)
```

```
    add_kernel[grid](a, b, o, N, TILE_SIZE)
```

```
    return o
```



上述代码的问题：

- 当 tensor 形状不同，需要 broadcast 是无法处理；
- tensor 数据不连续，broadcast 或，或者 permute 过，或者数据是分散的情况不能处理。

原因来自其加载和保存部分的地址偏移量的计算，就是预设计算首地址后连续的一篇内存区域。

解法一: 使用 wrapper 处理 non-contiguous

```
def add_func_naive(x, y):
    # supports broadcasting now
    x, y = torch.broadcast_tensors(x, y)

    # support non-contiguous now
    x = x.contiguous()
    y = y.contiguous()
    o = torch.empty(x.shape, dtype=x.dtype, device=x.device)

    TILE_SIZE = 512
    num_warps = 4
    N = x.numel()
    grid = triton.cdiv(N, TILE_SIZE), 1, 1
    add_kernel[grid](x, y, o, N, TILE_SIZE, num_warps=num_warps)
    return o
```

使用 wrapper 调用 torch.broadcast_tensor 来处理 broadcasting, 使用 contiguous 来处理数据不连续的问题。

使用 wrapper 处理 broadcast 和非连续的输入 tensor, 性能不一定好, 这个过程本身就可能有一次数据 copy.

解法二：支持 stride

假设 a, b 来自两个大的 tensor 上的不连续切片。

```
a = torch.randn(200)[::2]  
b = torch.randn(300)[::3]
```

修改其内存偏移量计算方式即可，需要传入三个 tensor 各自的 stride.

```
a = tl.load(a_ptr + offsets * stride_a, mask=mask)  
b = tl.load(b_ptr + offsets * stride_b, mask=mask)  
  
tl.store(o_ptr + offsets * stride_o, o, mask=mask)
```


解法二：支持 stride

假设 a, b 来自两个 2d tensor 上的不连续切片。

```
a = torch.randn(200, 200)[::2, ::2]
b = torch.randn(300, 300)[::3, ::3]
```

```
mask = tid < (M * N)
i1 = tid % N
i0 = tid // N
```

```
a = tl.load(a_ptr + i0 * a_stride0 + i1 * a_stride1, mask=mask)
b = tl.load(b_ptr + i0 * b_stride0 + i1 * b_stride1, mask=mask)
o = a + b
```

```
tl.store(o_ptr + i0 * o_stride0 + i1 * o_stride1, o, mask=mask)
```

数组不连续，必须考虑每个维度的 stride, 因此需要计算每个维度的 index。 (i0, i1)

有了各个维度的索引，以及 tensor 每个维度的 stride 就可以求出偏移量。

Q: 怎么把每个 tensor 的 stride 传给 JITFunction 呢？

A: Triton JITFunction 不接受 list tuple 等类型的参数，只能展开作为多个 int 来传。而 tensor 的 rank 在编写 jit function 的时候是不确定的。无法写出一份适用所有情况的代码。

计算内存偏移量的代码模式化，所有的 pointwise 运算原理上具有相似性，可以采用**自动代码生成**的方式。

(program id -> task id) -> multi index -> memory offset

triton 编程里各个 CTA 的差异是从 CTA id (program id) 开始的, 类似 cuda 的 blockIdx 和 threadIdx.

Pointwise Operation 总是可以视为若干个独立标量运算, 这些 task 可以展平视为 1d 的 vector. 每个 task 有自己的 task id. 每个 CTA 计算 TILE_SIZE 个 task. (**1d task space**)

$$\text{tid} = \text{pid} * \text{TILE_SIZE} + \text{tl.arange}(0, \text{TILE_SIZE})$$

例:

task space: (5, 5)

tile size: 4

task id

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

program id

0	0	0	0	1
1	1	1	2	2
2	2	3	3	3
3	4	4	4	4
5	5	5	5	6

program id -> (task id -> multi index) -> memory offset

通过取余和整除，将 task id 对应于每个 tensor 上的数据的多维索引计算出来。

Q: 为什么要使用 multi-index?

A: 每个维度的 stride 都是独立的，不预设 tensor 内存连续。所以 task id 无法直接对应到内存偏移量。

$i1 = tid \% N$

$i0 = tid // N$

task id

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

index 0

0	0	0	0	0
1	1	1	1	1
2	2	2	2	2
3	3	3	3	3
4	4	4	4	4

index 1

0	1	2	3	4
0	1	2	3	4
0	1	2	3	4
0	1	2	3	4
0	1	2	3	4

program id -> task id -> (multi index -> memory offset)

每个 tensor 在每个维度都有一个 stride，表示该维度索引增加1带来的内存偏移。多维索引和 strides 的内积就是内存偏移量。

```
a = tl.load(a_ptr + i0 * a_stride0 + i1 * a_stride1, mask=mask)
b = tl.load(b_ptr + i0 * b_stride0 + i1 * b_stride1, mask=mask)
tl.store(o_ptr + i0 * o_stride0 + i1 * o_stride1, o, mask=mask)
```

$$\begin{array}{|c|c|c|c|c|} \hline \text{offset} & & & & \\ \hline 0 & 1 & 2 & 3 & 4 \\ \hline 5 & 6 & 7 & 8 & 9 \\ \hline 10 & 11 & 12 & 13 & 14 \\ \hline 15 & 16 & 17 & 18 & 19 \\ \hline 20 & 21 & 22 & 23 & 24 \\ \hline \end{array} = \begin{array}{|c|c|c|c|c|} \hline \text{index 0} & & & & \\ \hline 0 & 0 & 0 & 0 & 0 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline 2 & 2 & 2 & 2 & 2 \\ \hline 3 & 3 & 3 & 3 & 3 \\ \hline 4 & 4 & 4 & 4 & 4 \\ \hline \end{array} * \text{stride0} + \begin{array}{|c|c|c|c|c|} \hline \text{index 1} & & & & \\ \hline 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 1 & 2 & 3 & 4 \\ \hline \end{array} * \text{stride1}$$

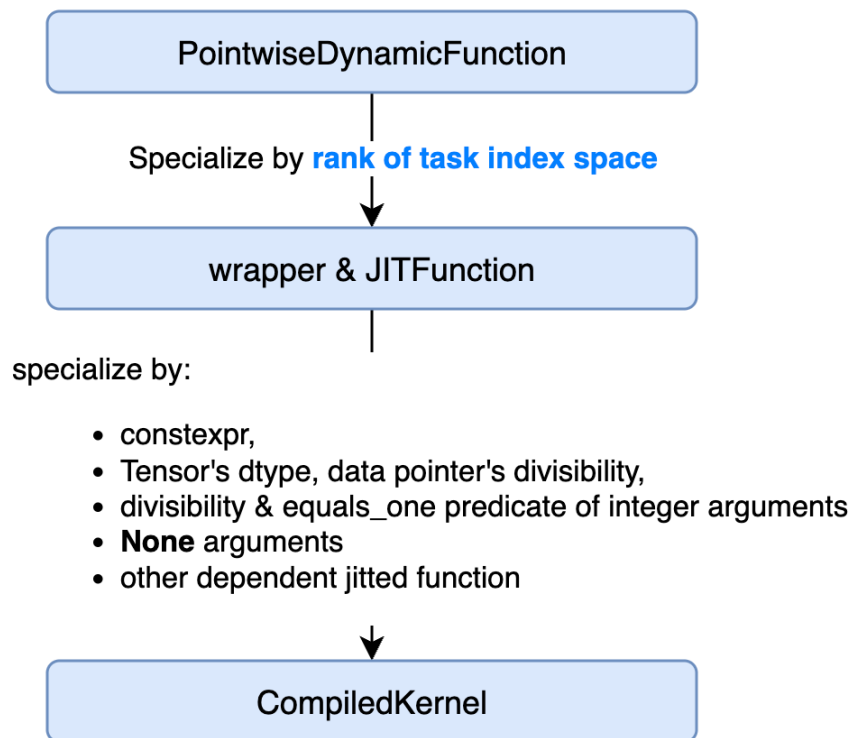
Pointwise 算子自动代码生成

- 01 为什么使用 Code Generation
- 02 核心: PointwiseDynamicFunction**
- 03 实现机制
- 04 性能分析
- 05 总结与展望

核心: PointwiseDynamicFunction

给定需要执行的标量运算，自动生成能适用于 tensor 的 pointwise operation.

- 根据输入的 argument 来确定**任务索引空间的 rank**（各个 tensor broadcast 后的形状）；
- 根据 rank 按需生成特化版本的函数。
- 转发参数，调用适用的特化版函数。



一个 **PointwiseDynamicFunction** 可能对应若干个 wrapper 和 JITFunction. 此处有一层特化，由 PointwiseDynamicFunction 维护。

pointwise_dynamic 实现为一个装饰器，装饰在一个 JITFunction 上。得到一个 PointwiseDynamicFunction。

一个 **JITFunction** 可能对应若干个 CompiledKernel, 此处是另外一层特化，这是 triton 运行时维护的。

PointwiseDynamic 的思路类似。

最基本的用法，有多个 tensor 输入，有一个输出，输出的类型和第一个 operand 一致。

```
@pointwise_dynamic
@triton.jit
def ordinary2(x, y):
    return tl.sin(x) + tl.cos(y)
```

```
@pointwise_dynamic()
@triton.jit
def ordinary(x, y):
    return tl.sin(x) + tl.cos(y)
```

可以指定某些参数不是 tensor，而是**作为普通标量传到标量函数**。还可以指定这些参数的数据类型。

```
@pointwise_dynamic(
    is_tensor=[True, False, True],
    dtypes=[None, float, None])
@triton.jit
def axpy(x, alpha, y):
    return x * alpha + y
```

```
@pointwise_dynamic(is_tensor=[True, False, True])
@triton.jit
def axpy(x, alpha, y):
    return x * alpha + y
```

支持指定每个输出 tensor 的数据类型，比如 compare 类输出 torch.bool

```
@pointwise_dynamic(output_dtypes=[torch.bool])  
@triton.jit  
def ge(x, y):  
    return x > y
```

支持指定输出个数（而不是从函数体推断）

```
@pointwise_dynamic(num_inputs=2, num_outputs=2)  
@triton.jit  
def f(a, b):  
    return a + b, a - b
```

支持更复杂的函数体，不只是一行的函数。比如 FlagGems 中的 gelu 函数

```
@pointwise_dynamic  
@triton.jit  
def gelu_none(x):  
    scale = 0.7071067811  
    output = 0.5 * x * (1 + tl.math.erf(x * scale))  
    return output
```


Pointwise 算子自动代码生成

- 01 为什么使用 Code Generation
- 02 核心: PointwiseDynamicFunction
- 03 实现机制**
- 04 性能分析
- 05 总结与展望

索引: 关于如何生成索引部分的代码;

函数嵌入: 如何把 scalar function 嵌入生成的代码中;

动态生成函数: 如何动态编译生成的代码;

代码缓存: 如何缓存生成的函数。

PointwiseDynamic 生成的代码有两个部分, wrapper 和 jit function.

wrapper function 分两个:

- wrapper: 主要负责生成 output 并调用 warpper_out;
- warpper_out: 参数准备, 计算索引空间, 调用 jit function;

```
def _wrapper(in0: torch.Tensor, val0: float, in1: torch.Tensor):  
    """Generated wrapper function with Pointwise: (Tensor, float, Tensor) -> (Tensor)"""  
    shape = broadcast_shapes([in0.shape, in1.shape])  
    out0 = torch.empty(shape, dtype=in0.dtype, device=in0.device)  
    out0 = _wrapper_out(in0, val0, in1, out0)  
    return out0
```

```
def _wrapper_out(in0: torch.Tensor, val0: float, in1: torch.Tensor, out0: torch.Tensor):
    """Generated wrapper function with Pointwise: (Tensor, float, Tensor, Tensor) -> (Tensor)"""
    shape = broadcast_shapes([in0.shape, in1.shape])
    num_tasks = volume(shape)

    # strides of each tensor argument w.r.t the task space
    in0_strides = broadcasted_stride(in0.shape, in0.stride(), shape)
    in1_strides = broadcasted_stride(in1.shape, in1.stride(), shape)
    out0_strides = out0.stride()

    # kernel launch
    grid = triton.cdiv(num_tasks, 512), 1, 1
    _jit_function[grid](
        in0, val0, in1, out0,
        in0_strides[0], in0_strides[1], # stride for in0
        in1_strides[0], in1_strides[1], # stride for in1
        out0_strides[0], out0_strides[1], # stride for out0
        shape[0], shape[1], # task indexing space
        num_tasks, # num tasks
        tile_size=512,
        num_warps=4,
    )
    return out0
```

并非所有代码都自动生成，我们写了少量计算形状的代码，在生成的代码中使用。

```
@triton.jit(do_not_specialize=['val0'])
def _jit_function(
    in0_ptr: tl.tensor, # of tl.pointer_type
    val0: float,
    in1_ptr: tl.tensor, # of tl.pointer_type
    out0_ptr: tl.tensor, # of tl.pointer_type
    in0_stride0: int, in0_stride1: int, # strides for in0
    in1_stride0: int, in1_stride1: int, # strides for in1
    out0_stride0: int, out0_stride1: int, # strides for out0
    s0: int, s1: int, # task_space
    num_tasks: int,
    tile_size: tl.constexpr,
):
    # task id & masking
    pid = tl.program_id(0)
    tid = pid * tile_size + tl.arange(0, tile_size)
    mask = tid < num_tasks

    # multi index reconstruction
    i1 = tid % s1
    tid //= s1
    i0 = tid % s0

    # loads
    in0 = tl.load(in0_ptr + i0 * in0_stride0 + i1 * in0_stride1, mask=mask)
    in1 = tl.load(in1_ptr + i0 * in1_stride0 + i1 * in1_stride1, mask=mask)

    # compute
    out0 = in0 * val0 + in1

    # stores
    tl.store(out0_ptr + i0 * out0_stride0 + i1 * out0_stride1, out0, mask=mask)
```

Python 中动态执行代码的方式:

- eval/exec 动态执行生成的代码字符串。或者先 compile 再 exec. 这样可以执行一些依赖当前 scope 的代码(比如依赖 exec 时 local 或 global scope 中的变量)。
- 写入到文件, 然后再 compile exec.
- 写入到文件, 用 importlib 来导入, 这样要求模块是不依赖当前的 scope.

Triton JITFunction 本身使用了 **inspect** 库来获取函数源码, 而如果没有写入到文件, 单纯 exec 字符串得到的函数是没有源码信息的, 会导致 triton 编译流程出错。Pointwise dynamic 的方案是写入到文件, 然后在用 importlib 来导入。代码独立, 也方便 debug 时直接修改生成的代码进行测试。

```
with open(cache_dir() / file_name, "wt", encoding="utf-8") as f:  
    f.write(code.getvalue())
```

```
spec = importlib.util.spec_from_file_location(  
    f"_gen_module_{self._scalar_fn_cache_key}", f.name)  
m = importlib.util.module_from_spec(spec)  
spec.loader.exec_module(m)  
overload = getattr(m, "_wrapper")
```

被装饰的函数要求是一个 JITFunction. 因此其内部可以调用 tl 内的函数, 而不是写 torch 的函数。(这主要是为了避免去做代码翻译带来的不准确)

Inline 发生在 Python AST 层面。使用 **ast.Visitor** 来改写 AST, 然后再使用 unparsed 重新生成 python 代码, 嵌入 jit function 代码中。(这主要是因为 triton JITFunction 不支持传入 triton JITFunction 作为参数)

- scalar_function 中的输入形参将会被传入的实参的名字替换;
- scalar_function 中的定义的 local 变量名字会避免与当前命名空间中已经使用的名字冲突;
- scalar_function 返回表达式将会改为赋值表达式, 赋值给输出变量;

```
@pointwise_dynamic(is_tensor=[True, False, True])
@triton.jit
def axpy(x, alpha, y):
    return x * alpha + y
```

```
# loads
in0 = tl.load(in0_ptr + ...)
in1 = tl.load(in1_ptr + ...)

# compute
out0 = in0 * val0 + in1

# stores
tl.store(out0_ptr + ..., out0, mask=mask)
```

PointwiseDynamicFunction 的缓存: 以 rank 为 key 的字典, 运行时维护。

```
def __call__(self, *args):
    # It does not accept kwargs
    key = f"{self.arg_key(*args)}"
    if key in self.overloads:
        overload = self.overloads[key]
    else:
        m = ... # dynamically generated module
        overload = getattr(m, "_wrapper")
        self.overloads[key] = overload
    return overload(*args)
```

生成文件: 对于 pointwise_dynamic 装饰得到的函数, 影响生成的文件(wrapper & jit function) 的因素有 scalar_fn 的 hash 值和任务索引空间的 rank. 目前这两个会称为生成代码的文件名的一部分。

生成的代码位于 ~/.flaggems 中。

```
pointwise_dynamic_94aa80c89e0e8d39c672944d86c04d5c1cebf5c314_rank_2.py
pointwise_dynamic_94aa80c89e0e8d39c672944d86c04d5c1cebf5c314_rank_3.py
pointwise_dynamic_94aa80c89e0e8d39c672944d86c04d5c1cebf5c314_rank_4.py
pointwise_dynamic_be7ae4ada7c58c74798a8eb3457980c60e03778e7_rank_2.py
pointwise_dynamic_be7ae4ada7c58c74798a8eb3457980c60e03778e7_rank_3.py
pointwise_dynamic_be7ae4ada7c58c74798a8eb3457980c60e03778e7_rank_4.py
```


Pointwise 算子自动代码生成

- 01 为什么使用 Code Generation
- 02 核心: PointwiseDynamicFunction
- 03 实现机制
- 04 性能分析**
- 05 总结与展望

x y 都是行主序排布。x + y

contiguous, no broadcasting

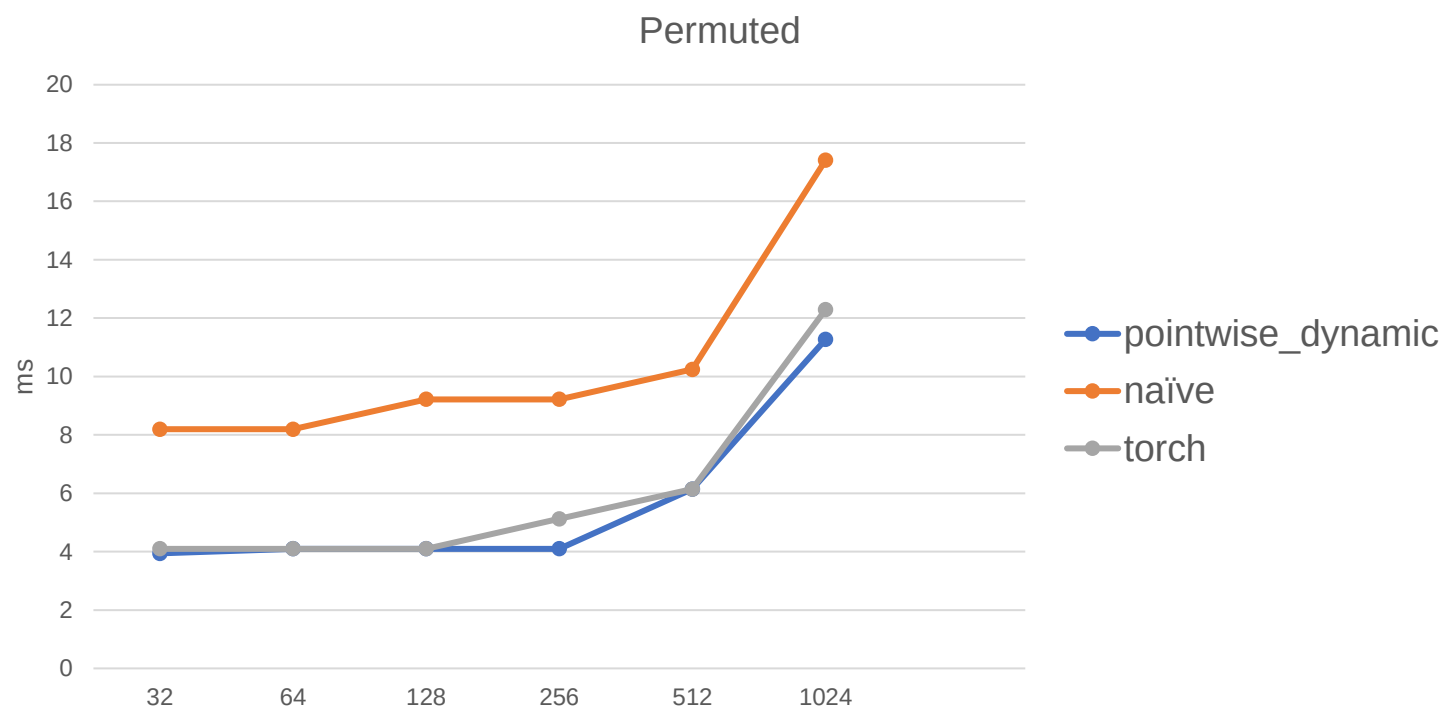
pointwise_dynamic: 4.096	naive: 4.096	torch: 4.032	Shape: (32,), (32,)
pointwise_dynamic: 4.032	naive: 3.328	torch: 3.328	Shape: (64,), (64,)
pointwise_dynamic: 3.840	naive: 3.840	torch: 3.744	Shape: (128,), (128,)
pointwise_dynamic: 3.904	naive: 3.936	torch: 3.936	Shape: (256,), (256,)
pointwise_dynamic: 4.096	naive: 4.096	torch: 4.096	Shape: (32, 32), (32, 32)
pointwise_dynamic: 4.096	naive: 3.136	torch: 3.776	Shape: (64, 64), (64, 64)
pointwise_dynamic: 4.096	naive: 4.096	torch: 4.096	Shape: (128, 128), (128, 128)
pointwise_dynamic: 5.120	naive: 5.120	torch: 5.120	Shape: (256, 256), (256, 256)

contiguous, with broadcasting

pointwise_dynamic: 4.000	naive: 3.328	torch: 3.360	Shape: (32,), (32, 1)
pointwise_dynamic: 3.936	naive: 3.264	torch: 3.872	Shape: (64,), (64, 1)
pointwise_dynamic: 3.968	naive: 3.360	torch: 3.776	Shape: (128,), (128, 1)
pointwise_dynamic: 3.936	naive: 4.000	torch: 3.840	Shape: (256,), (256, 1)
pointwise_dynamic: 4.096	naive: 4.096	torch: 3.968	Shape: (32, 32), (1, 32)
pointwise_dynamic: 4.096	naive: 4.096	torch: 4.096	Shape: (64, 64), (1, 64)
pointwise_dynamic: 4.096	naive: 4.096	torch: 4.096	Shape: (128, 128), (1, 128)
pointwise_dynamic: 9245.696	naive: 9240.576	torch: 9243.616	Shape: (25600, 25600), (1, 25600)
pointwise_dynamic: 4.096	naive: 4.096	torch: 4.096	Shape: (25600, 1), (1, 25600)
pointwise_dynamic: 18.432	naive: 18.432	torch: 18.432	Shape: (1024, 1, 1024), (2560, 1)

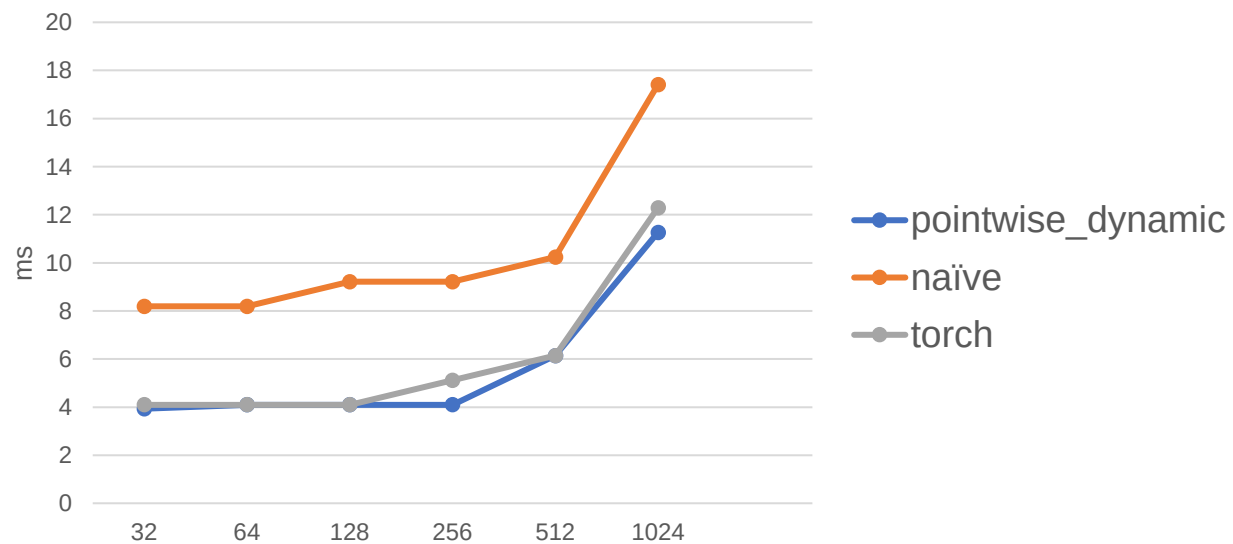
这种情况 pointwise_dynamic 生成的性能略差。因为并没有针对 contiguous 做特别处理，而是按照一般情形去计算 memory offset,所以性能差点。而且也说明 torch.broadcast_tensors 和 contiguous 确实优化不错。

x 和 y 都是方阵, x 列主序, y 行主序。 $x + y$

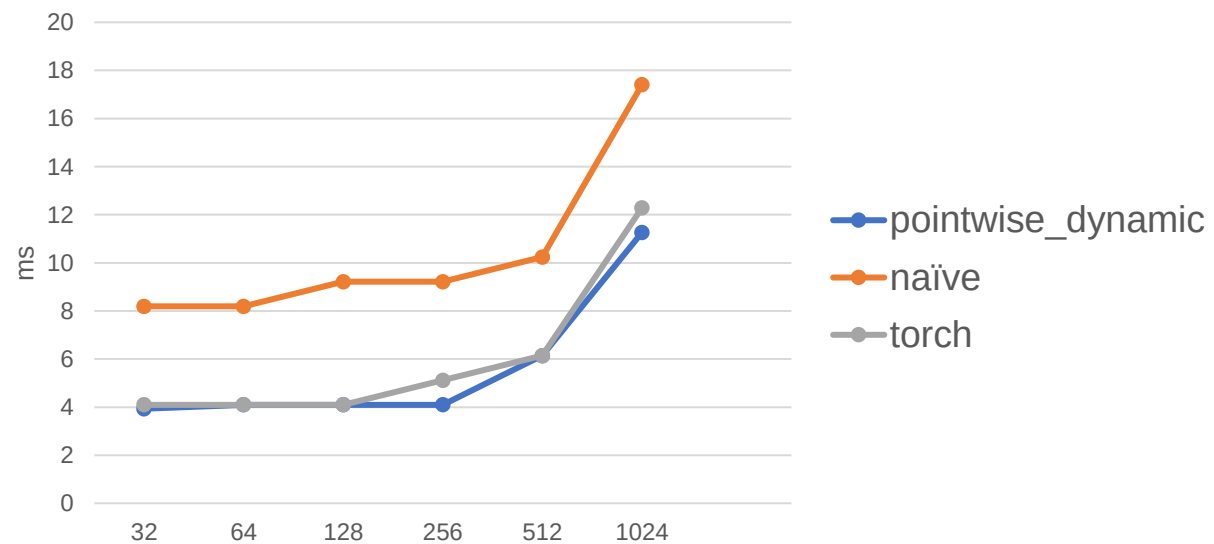


x, y 都是从行主序方阵上的切片。

隔一列取一列



隔一行取一行



Pointwise 算子自动代码生成

- 01 为什么使用 Code Generation
- 02 核心: PointwiseDynamicFunction
- 03 实现机制
- 04 性能分析
- 05 总结与展望**

在设计代码生成的时候借鉴了

- torch 的 TensorIterator, 以及 numpy nditer 来设计。其中二者是在写算子时作为 library 来使用的, 提供了一个高级迭代器的功能, 也可以把 map 一个标量运算函数。
- torch inductor. 它主要是用作 jit compiler, 生成的算子是针对 shape 和 stride 特化的。

后续展望

- 生成的代码后续优化 TODO:
 - 减少冗余计算, 面对 contiguous 做特别处理,
 - 对输出 tensor 的布局, 使用更好的算法, 实现更好的内存访问模式。
 - 根据输入 tensor 的 shape 和 stride 进行分析, 对任务索引空间进行混排来实现更好的内存访问模式。目前 pointwise dynamic 里任务索引空间都是行主序的。
- 函数嵌入: triton 目前不支持传入把 JITFunction 作为参数传给 JITFunction. 所以函数嵌入是通过 python AST 层来实现的。如果能让生成的代码直接调用被装饰的函数, 可以做得更稳健。
- 可能支持 reduction 类算子, 欢迎参与贡献。



谢谢聆听!

欢迎访问开源社区:

<https://github.com/FlagOpen/FlagGems>

<https://github.com/FlagOpen/FlagAttention>

Triton中国生态Meetup反馈收集表



Triton中国社区微信群



扫码添加微信，私信“加群”

Cambricon Triton



王天一

2024.06.02

Cambricon
寒 武 纪

Pytorch

Triton 公共算子库 FlagGems

Cambricon Triton编译器

目录

Contents

1. Cambricon Triton Overview
2. Motivation
3. Architecture
4. Triton-Linalg
5. Performance
6. Q & A

Cambricon Triton Overview

About Cambricon Triton



/Triton语言/

Triton是一个用于编写高效自定义深度学习算子的语言和编译器。相比CUDA，Triton在实现深度学习算子开发中具有更高的生产力和更大的灵活性。

/Cambricon Triton/

寒武纪Triton是基于Triton语言，适配寒武纪机器学习单元（Machine Learning Unit，简称MLU）的编译器。该编译器旨在尽量不修改Triton原语的前提下，最大限度地复用Triton的开发生态。

/Cambricon Triton具有以下特点/

- ① 基于原生Triton语言实现算子开发。
- ② 结合寒武纪硬件架构特点，优化Triton语言开发的算子核函数（Kernel Function，简称Kernel），使其在寒武纪硬件上的运行性能接近手写算子性能。

Cambricon Triton Overview



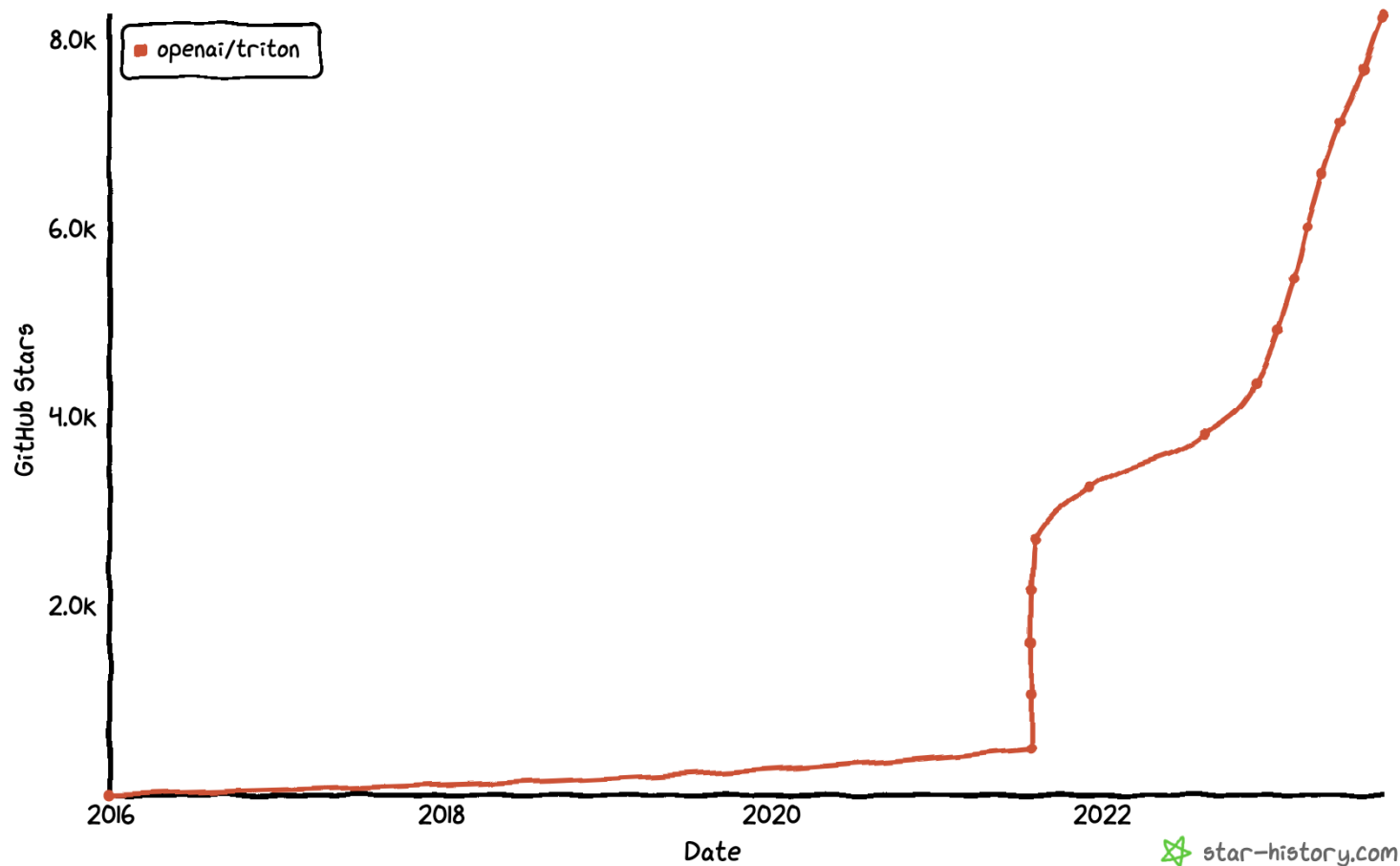
- ① 已发布Cambricon Triton v1.0.0版本
- ② 基于原生triton v2.2, 对齐PyTorch2.3版本依赖
- ③ 编译方式: 支持JIT和AOT编译
- ④ 已支持triton官方所有原语算子 (80+)
- ⑤ 已支持FlagGems算子库35个算子
- ⑥ 已支持官方所有后端无关特性: Auto Tune、Interpreter等
- ⑦ 差异特性
 - ✓ num_warps: 对标寒武纪任务类型, BLOCK/U1/Ux
 - ✓ num_stages: 只支持0/1的开关表示是否开启流水优化
 - ✓ 增大triton默认最大处理元素个数, 131072 → 1048576
 - ✓ 解除BLOCK_SIZE要求2的次幂限制

2

Motivation

Why Triton?

2022年开始，Triton的热度呈爆发式增长，Triton生态逐渐形成规模。



Why Triton?

/快速高效/

使用Triton，硬件厂商只需关注如何从Triton层编译到硬件代码，即可高效支持上层的各类AI程序，如PyTorch、xla等。

/易于上手/

Triton学习成本低，开发效率高。

/生态兼容性高/

使用Triton可以有效减少对于CUDA 单一生态的依赖，解决兼容性、软硬件适配度等问题。

/适配度高/

Triton支持PyTorch原生框架，输入可以直接接PyTorch Tensor。

Triton Example

```
import torch
import triton
import triton.language as tl

@triton.autotune(configs=[triton.config('BLOCK_SIZE': 128,
triton.config('BLOCK_SIZE': 256))])

@triton.jit
def add_kernel(x_ptr, # *Pointer* to first input vector.
y_ptr, # *Pointer* to second input vector.
output_ptr, # *Pointer* to output vector.
n_elements, # Size of the vector.
BLOCK_SIZE: tl.constexpr, # Number of elements each program should process.
# NOTE: `constexpr` so it can be used as a shape value.
):
    # There are multiple 'programs' processing different data. We identify which program
    # we are here:
    pid = tl.program_id(axis=0) # We use a 1D launch grid so axis is 0.
    # This program will process inputs that are offset from the initial data.
    # For instance, if you had a vector of length 256 and block_size of 64, the programs
    # would each access the elements [0:64, 64:128, 128:192, 192:256].
    # Note that offsets is a list of pointers:
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    # Create a mask to guard memory operations against out-of-bounds accesses.
    mask = offsets < n_elements
    # Load x and y from DRAM, masking out any extra elements in case the input is not a
    # multiple of the block size.
    x = tl.load(x_ptr + offsets, mask=mask, other=0)
    y = tl.load(y_ptr + offsets, mask=mask, other=0)
    output = x + y
    # Write x + y back to DRAM.
    tl.store(output_ptr + offsets, output, mask=mask)

N = 1024
X = torch.randn(N, device='mlu')
Y = torch.randn(N, device='mlu')
Z = torch.randn(N, device='mlu')
grid = lambda args : (triton.cdiv(N, args['BLOCK_SIZE']), )
add_kernel[grid](X, Y, Z, N, BLOCK_SIZE)
```

获取对应的分块id, 和下面的grid对应

控制数据边界

数据加载指令, 相当于gather/scatter操作

BangC Example

```
#define SIZE_PER_TASK 64
```

```
__mlu_entry__ void Add(half* dst, half* src1, half* src2) {
```

```
    __nram__ half output[SIZE_PER_TASK];
```

```
    __nram__ half input1[SIZE_PER_TASK];
```

```
    __nram__ half input2[SIZE_PER_TASK];
```

```
    __memcpy(input1, src1 + SIZE_PER_TASK * taskId, SIZE_PER_TASK * sizeof(half), GDRAM2NRAM);
```

```
    __memcpy(input2, src2 + SIZE_PER_TASK * taskId, SIZE_PER_TASK * sizeof(half), GDRAM2NRAM);
```

```
    __bang_add(output, input1, input2, SIZE_PER_TASK);
```

```
    __memcpy(dst + SIZE_PER_TASK * taskId, output, SIZE_PER_TASK * sizeof(half), NRAM2GDRAM);
```

```
}
```

```
int main() {
```

```
    ...
```

```
    Kernel<<<dim, ktype, pQueue>>>(mlu_result, mlu_source1, mlu_source2);
```

```
}
```

BangC vs. Triton

/BangC/

① 提供更多的优化选择

- 精确控制core内执行的每条指令(可以内嵌汇编)
- 精细管理nram/wram/shared memory内存

② 学习曲线陡峭 & 开发成本高

③ 下限低 & 上限高



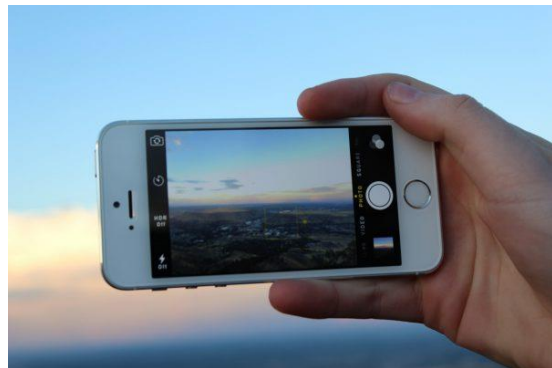
/Triton/

① 提供基础的能力:

- 描述每个Program的计算逻辑
- Triton托管了Program内的所有优化
 - 指令选择 & 指令融合
 - 片上内存管理
 - 流水

② 原语相对较少, 上手快

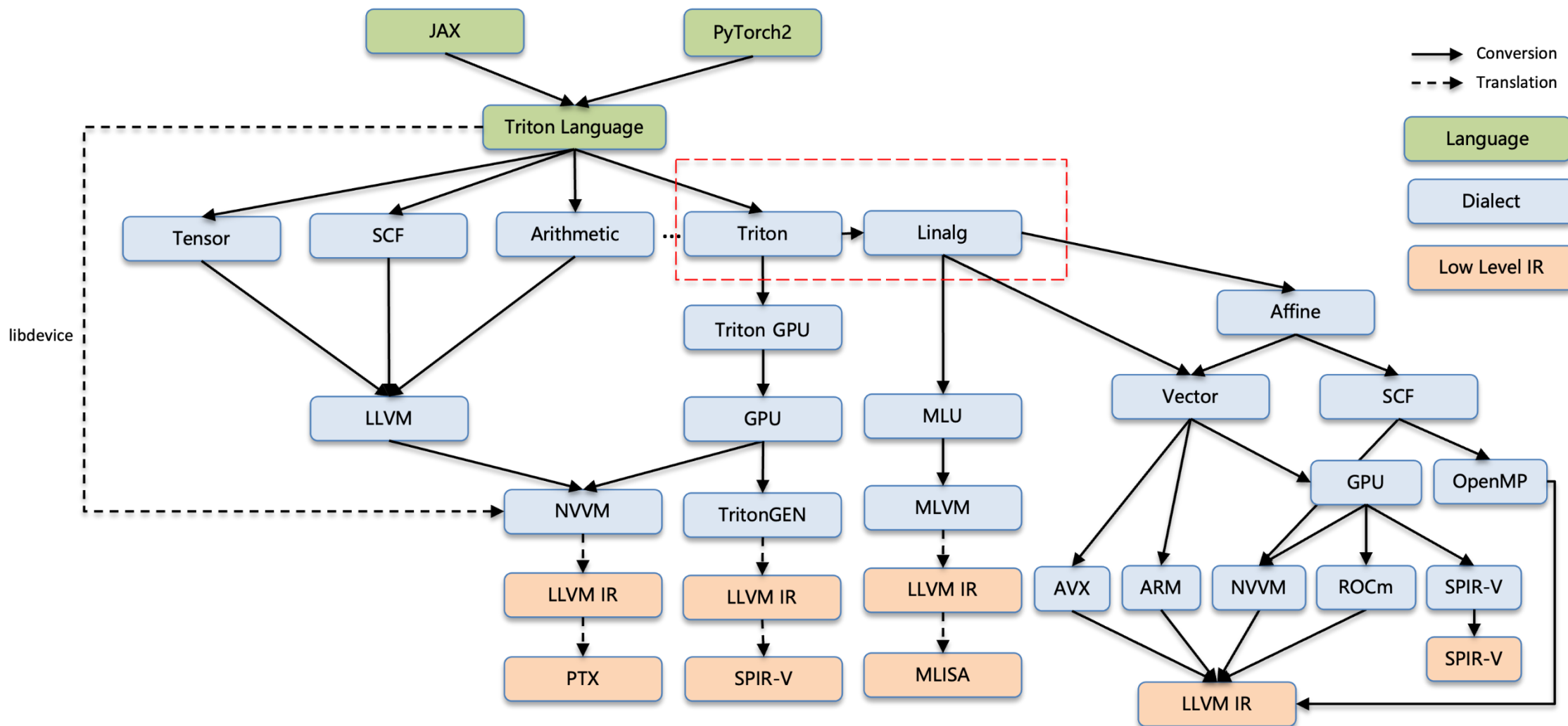
③ 下限高 & 上限低



3

Architecture

Architecture



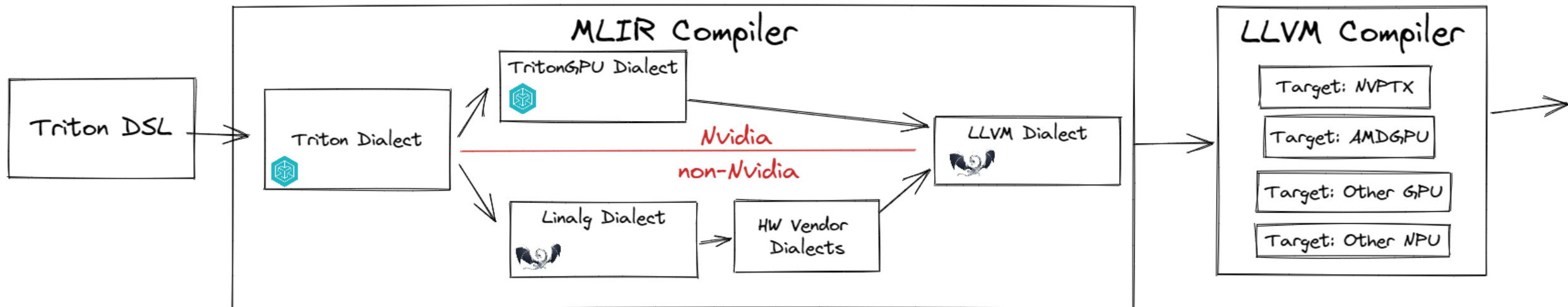
4

Triton-Linalg

Why Linalg?

Linalg是当前MLIR社区主推的一个面向CodeGen的方言，提供了一系列高度抽象的功能，以此来简化和优化线性代数运算。

- ① 官方已有的很多优化都是针对GPGPU架构，DSA架构需要重造。
- ② 丰富的基础设施：如Tile & tile reduction/Fuse/Promotion/etc...
- ③ 方便扩充op以及定制化优化pass。
- ④ 基于Tensor&MemRef的算子定义和Triton IR容易对应。
- ⑤ 社区微软也在推Linalg方案来接各种DSA。



在将triton转到linalg过程中，主要遵循下面几个原则：

① 尽可能使用structure算子，不使用linalg.generic。

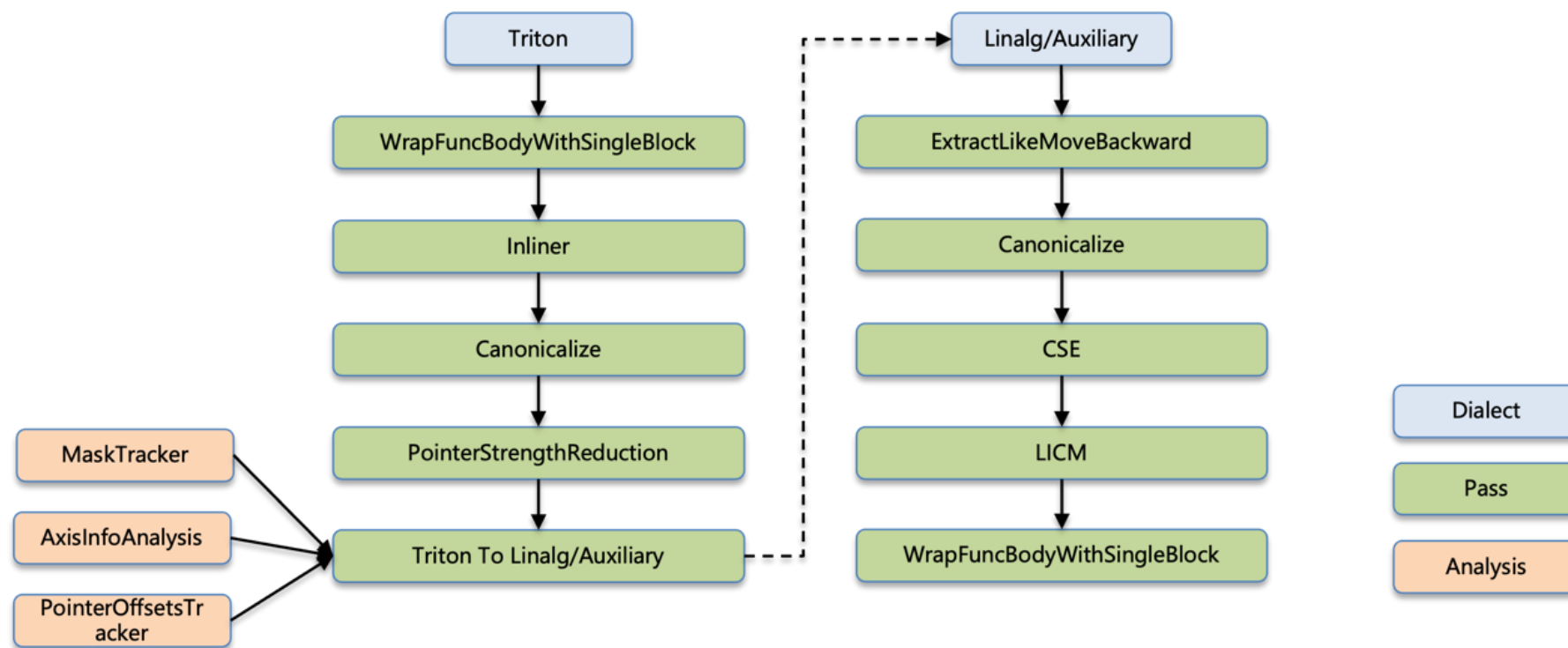
- `tt.reduce` → `linalg.reduce`
- `tt.dot` → `linalg.matmul`
- `tt.make_range` → `linalg_ext.make_range`

② 尽可能早的识别算子语义。

- `tt.reduce` → `linalg_ext.argmax`
- 根据连续性分析算法，将load转成copy

Triton-Linalg由4个部分构成：

- ① Dialect: 定义了Linalg Extension算子以及Auxiliary辅助算子；
- ② Transforms: 扩充了社区方言的标准化以及一些辅助的优化pass；
- ③ Analysis: 包含指针连续性分析算法、Mask和指针偏移分析算法等；
- ④ Conversion: 包含Triton/Arith/Math到Linalg的转换。



指针分析 (Pointer Analysis)

转换成gather

离散访存, DMA效率差

```
1. tt.func public @triton_add(%x: !tt.ptr<f32>, ..., %size: i32) {  
2.   %c128_i32 = arith.constant 128 : i32  
3.   %0 = tt.get_program_id x : i32  
4.   %1 = arith.muli %0, %c128_i32 : i32  
5.   %2 = tt.make_range {end = 128 : i32, start = 0 : i32} : tensor<128xi32>  
6.   %3 = tt.splat %1 : (i32) -> tensor<128xi32>  
7.   %offsets = arith.addi %3, %2 : tensor<128xi32>  
8.   %mask = arith.cmpi slt, %4, %size : tensor<128xi32>  
10.  %6 = tt.splat %x : (!tt.ptr<f32>) -> tensor<128x!tt.ptr<f32>>  
11.  %7 = tt.addptr %6, %offsets : tensor<128x!tt.ptr<f32>>, tensor<128xi32>  
12.  %X = tt.load %7, %mask {...} : tensor<128xf32>  
13.  ...  
14.  tt.return  
15. }
```

```
1. func public @triton_add(%x: !tt.ptr<f32>, ..., %size: i32) {  
2.   %memref = aux.view %x to offsets: [0] size: [INF]  
3.   %tensor = bufferization.to_tensor %memref  
4.   %offsets = ...  
5.   %X = linalg_ext.gather %tensor %offsets  
6.   ...  
7.   return  
8. }
```

指针分析 (Pointer Analysis)

转换成copy

```
1. tt.func public @triton_add(%x: !tt.ptr<f32>, ..., %size: i32) {  
2.   %c128_i32 = arith.constant 128 : i32  
3.   %0 = tt.get_program_id x : i32  
4.   %1 = arith.muli %0, %c128_i32 : i32  
5.   %2 = tt.make_range {end = 128 : i32, start = 0 : i32} : tensor<128xi32>  
6.   %3 = tt.splat %1 : (i32) -> tensor<128xi32>  
7.   %offsets = arith.addi %3, %2 : tensor<128xi32>  
8.   %mask = arith.cmpi slt, %4, %size : tensor<128xi32>  
10.  %6 = tt.splat %x : (!tt.ptr<f32>) -> tensor<128x!tt.ptr<f32>>  
11.  %7 = tt.addptr %6, %offsets : tensor<128x!tt.ptr<f32>>, tensor<128xi32>  
12.  %X = tt.load %7, %mask {...} : tensor<128xf32>  
13.  ...  
14.  tt.return  
15. }
```

AxisInfoAnalysis

```
1. func public @triton_add(%x: !tt.ptr<f32>, ..., %size: i32) {  
2.   %memref = aux.view %x to offsets: [0] size: [1128], stride[1]  
3.   %tensor = bufferization.to_tensor %memref  
4.   %data = tensor.empty  
5.   linalg.copy ins(%tensor) outs(%data)  
6.   ...  
7.   return  
8. }
```

指针分析 (Pointer Analysis)

AxisInfoAnalysis

① 识别每个轴的连续信息:

- strideVal: 在I维度, 连续两个元素的间隔值
- stride: 在I维度, 连续相等的strideVal的元素个数

② 示例:

- 某个维度连续: $\text{stride}[I] = \text{shape}[I]$, $\text{strideVal}[I] = 1$
- 某个维度broadcast: $\text{stride}[I] = \text{shape}[I]$, $\text{strideVal}[I] = 0$

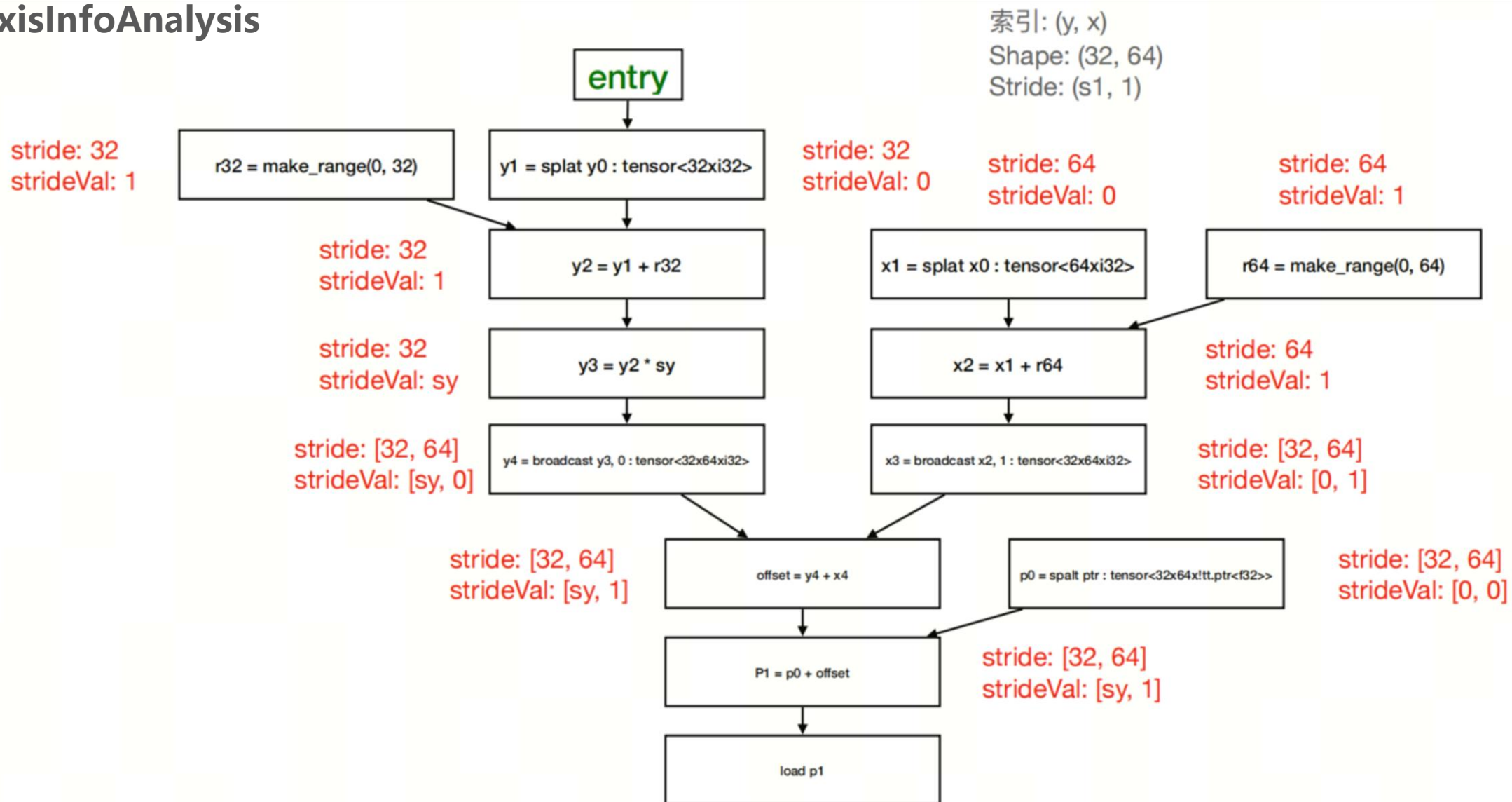
$\text{stride}[0] = \text{shape}[0] = 4$,
 $\text{strideVal}[0] = 1$

0	1	2	3
0	1	2	3
0	1	2	3
0	1	2	3

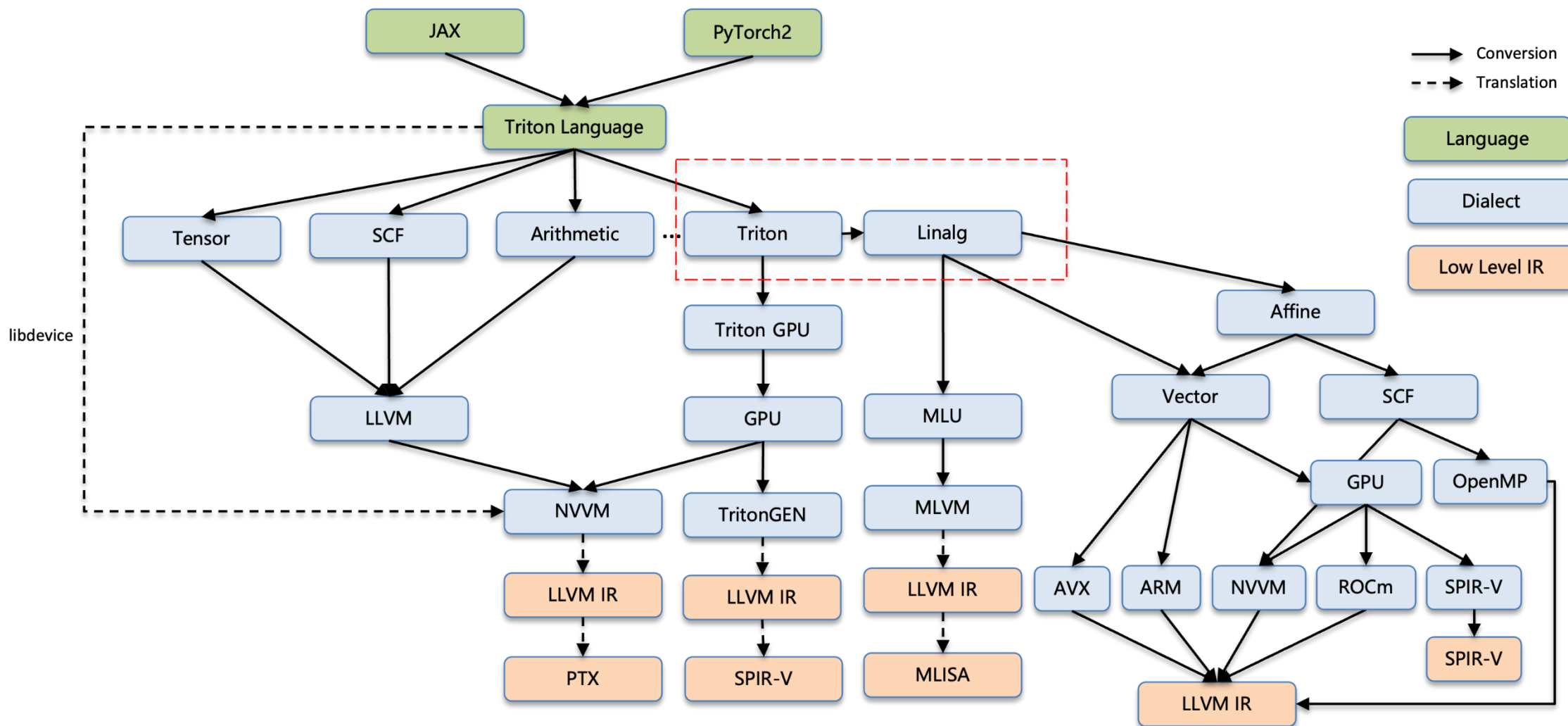
$\text{stride}[0] = \text{shape}[0] = 4$,
 $\text{strideVal}[0] = 0$

指针分析 (Pointer Analysis)

AxisInfoAnalysis



Architecture



Triton-Linalg开源项目



- 2024年5月28日，寒武纪开源了跨平台AI编译器前端Triton-Linalg，以此帮助开发者降低硬件适配成本，提高集成效率。
- 通过Triton-Linalg编译器前端，开发者或者硬件厂商可以以极低的开发成本，快速集成支持Triton语言特性的后端指令集，并对接上层AI应用。
- GitHub链接：<https://github.com/Cambricon/triton-linalg>



开放、协同、共享、共赢！
欢迎大家共建！

Thanks
& Any Questions?

Cambricon
寒 武 纪

Triton语言入门教程

Triton 语言入门教程

01 Triton 语言简介

02 Triton 算子开发示例

03 Triton 编译器

04 Triton 运行时

05 总结

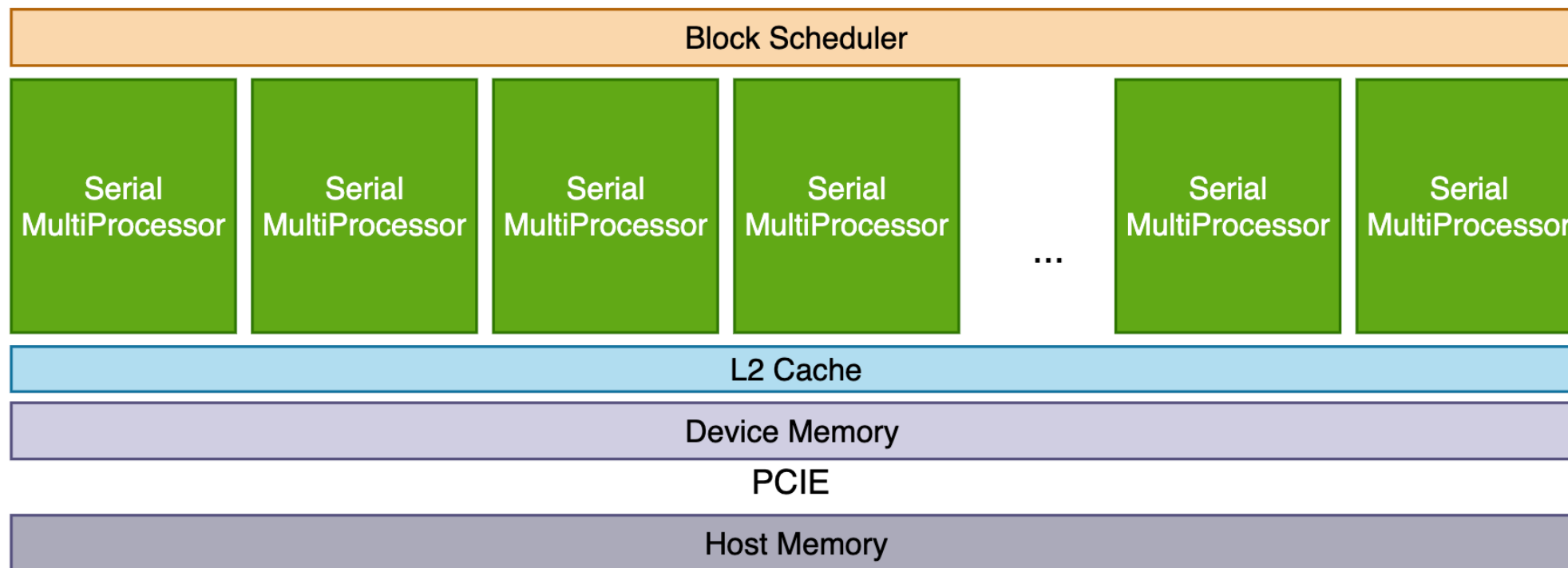
Triton 在 AI 技术栈中的位置

Triton 是一门 kernel 编程语言，主要用于编写 gpu kernels，也可以作为 AI 编译器的 Kernel 代码生成目标语言。和 CUDA C, openCL 等属于并列的关系。

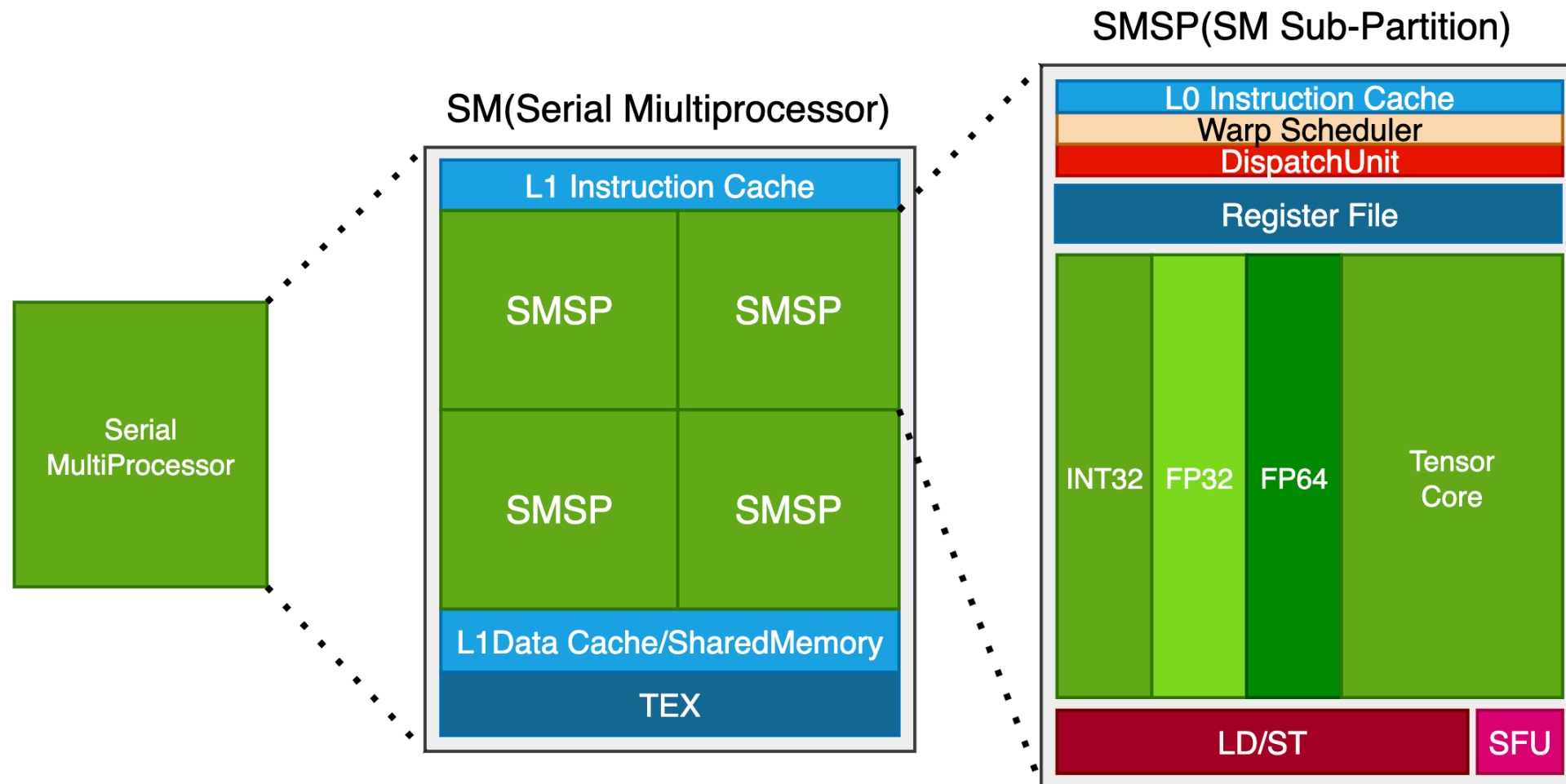
Model	Pytorch/Tensorflow/JAX
Graph	XLA/HLO TVM/Relay Pytorch/fx
Kernel	CUDA OpenCL ROCM HIP SYCL Triton
Device	GPUs/CPUs/...



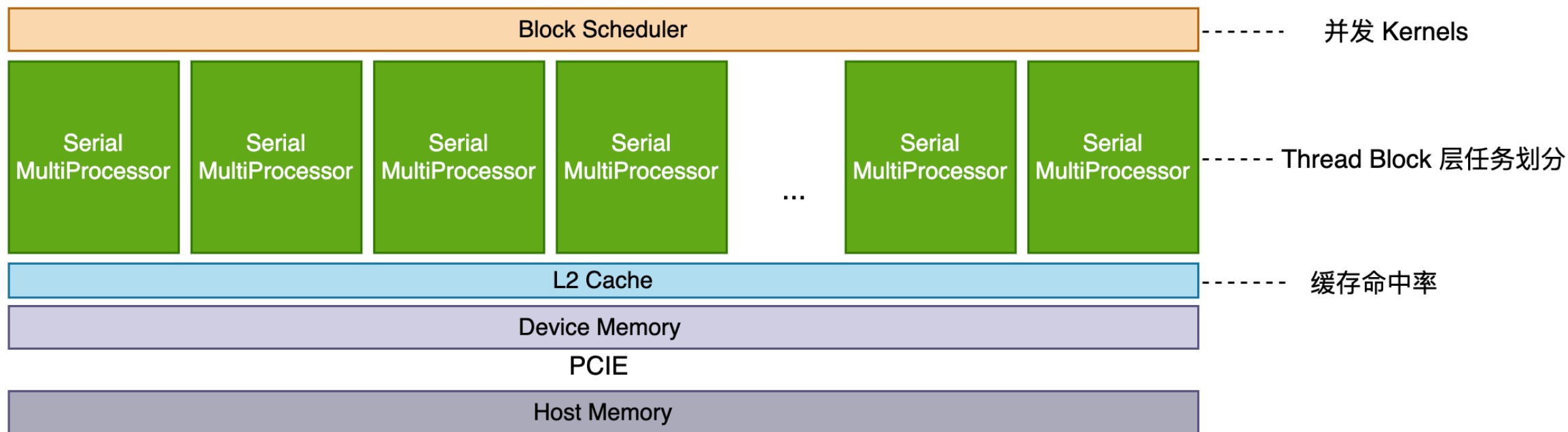
Serial MultiProcessor(SM) 以上的结构



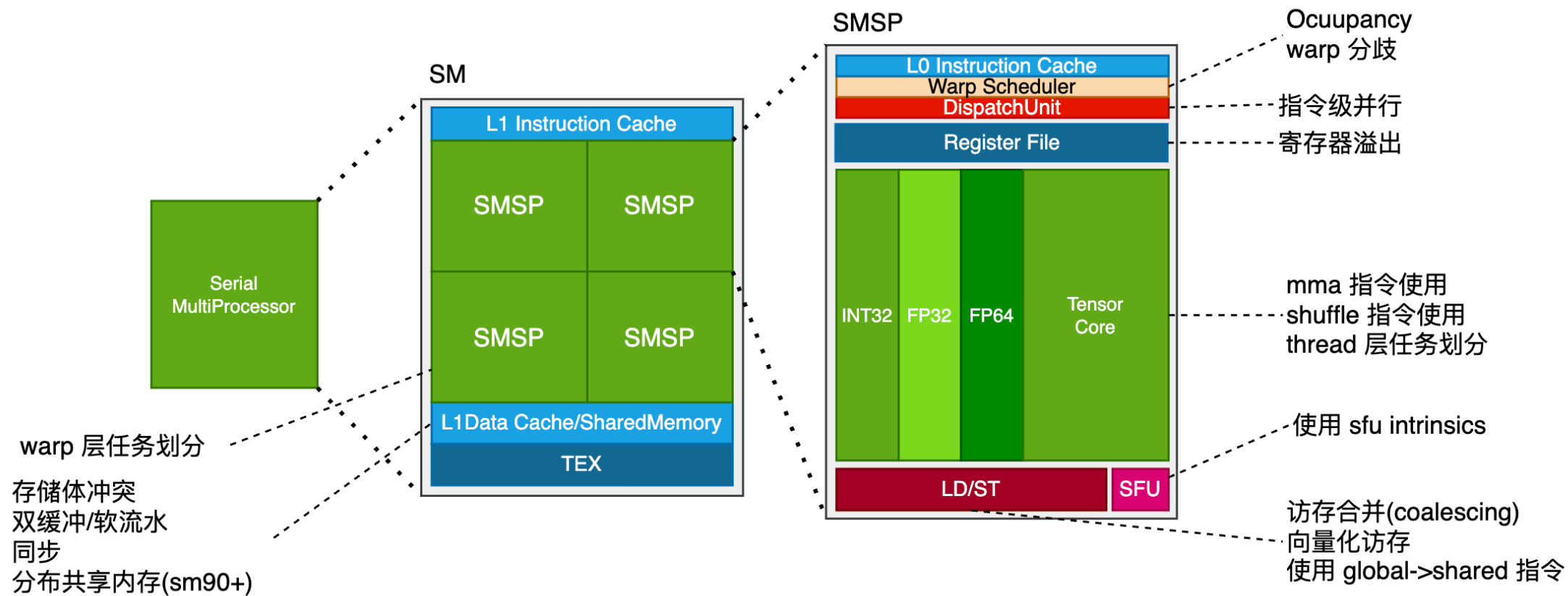
SM 以下的结构

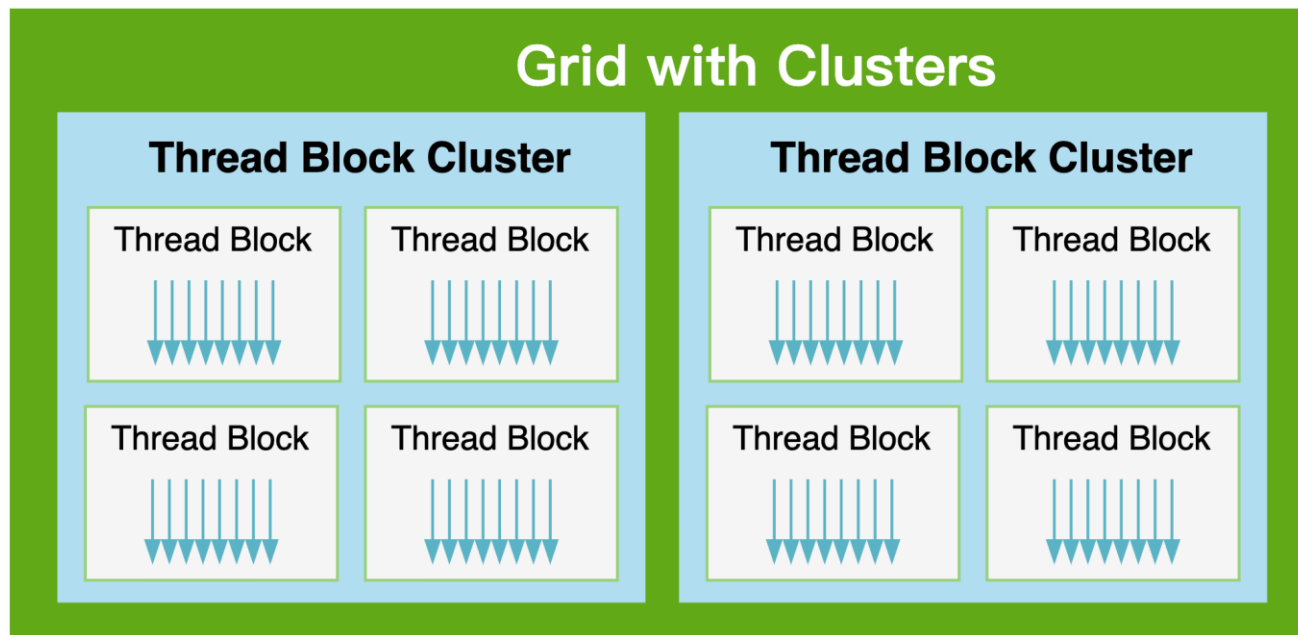


SM 以上需要关注的主要是 grid 的任务划分，并发 Kernel 调度等。



SM 以下需要考虑的问题

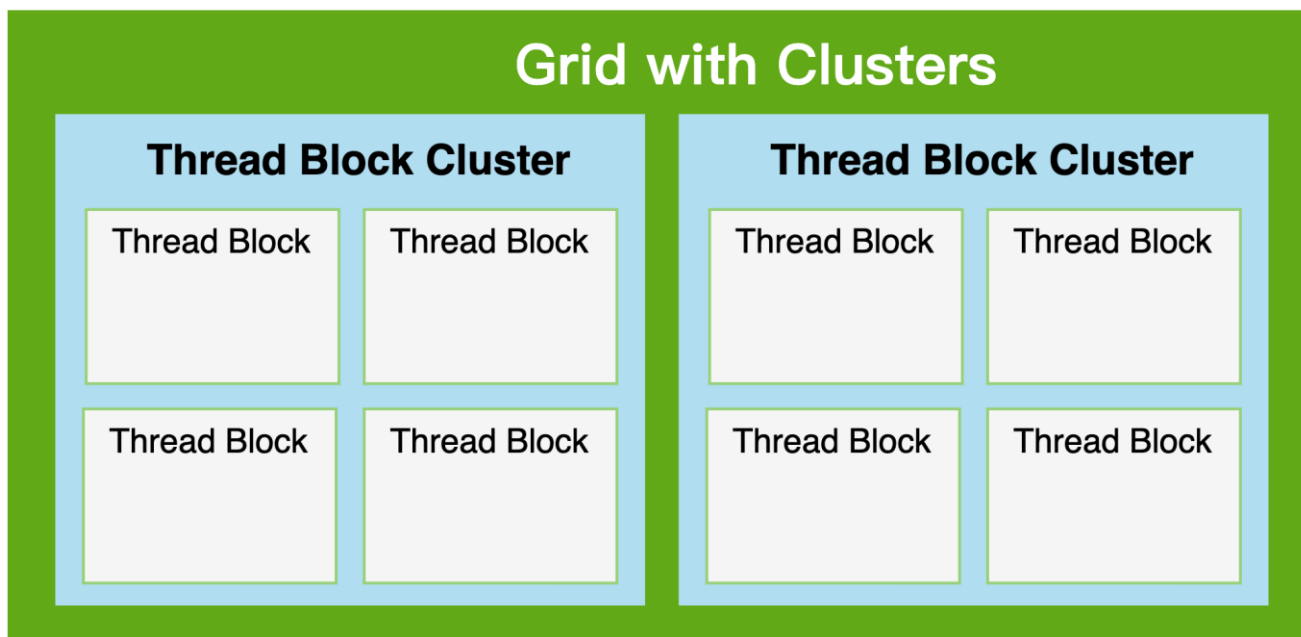




software concept	hardware concept
grid(Kernel)	device
thread block cluster (TBC)	GPU Processing Cluster (GPC)
thread block (CTA)	Serial MultiProcessor(SM)
warp	Serial MultiProcessor Sub-Partiom (SMSP)
thread	cuda core

CUDA 编程是 Thread 级别的编程。SIMT Programming Model

每个 `__global__` 函数仅描述了单个 thread 的计算逻辑。尽管实际的执行单元是 warp, 甚至有些 instruction 是 warp 级别的它们都需要表示为每个 thread 的计算。



software concept	hardware concept
Kernel	device
cluster	GPU Processing Cluster (GPC)
program(CTA)	Serial MultiProcessor(SM)

Triton 编程是 CTA 级别编程。 SPMD Programming Model

Triton Program 表示的是一个 CTA 的计算逻辑。而不必深入到每个 warp 做什么，每个 thread 做什么，什么时候同步，什么时候使用 shared memory. CTA 以下的细节和优化都由编译器处理。开发者主要关心算法以及 CTA 级别的任务划分。

Triton abstract complex GPU concept but leaves control to user on the algorithm and tuning.

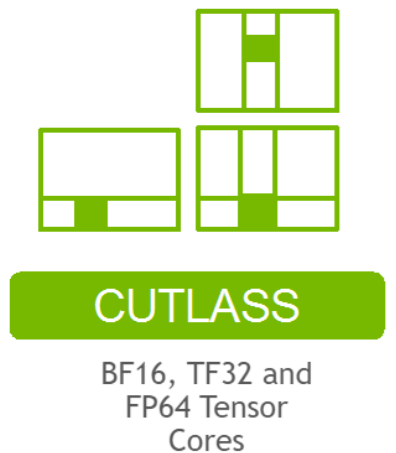
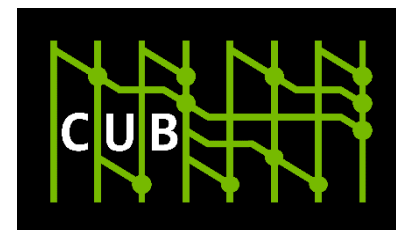
	CUDA	Triton	Torch Op
Algorithm	User	User	Compiler
Shared Memory	User	Compiler	Compiler
Barriers	User	Compiler	Compiler
Distribution to blocks	User	User	Compiler
Grid size	User	User	Compiler
Distribution to Warps/threads	User	Compiler	Compiler
Tensor Core usage	User	Compiler	Compiler
Coalescing	User	Compiler	Compiler
Intermediate data layout	User	Compiler	Compiler
Workgroup size	User	User	Compiler

* 本页内容引用自 Pytorch Conference 2023 Thomas Raoux 的 Talk: Triton Compiler <https://www.youtube.com/watch?v=AtbnRIzpwho>

cuda 不少的库提供的是 Device Level API, 比如 cublas, cufft, cusparse, curand, cusolver, cutensor, cudnn. 但 cuda 生态中的模板库则不太相同。比如 cub (主要提供 cooperative primitives)和 cutlass (主要关于 Linear Algebra Subroutine) 提供了

Thread Level, Warp Level, Block Level, Device Level 的功能, 前三者都是可以用于 Kernel 开发的。

因此 Triton 在与 CUDA 对比的时候, 也不可以忽略这些库, 它们也以模板库的形式, 提供了 Warp Level 和 Block Level 的抽象。但就易用性和多元AI芯片移植性而言, Triton 更胜一筹。



更高抽象层级的编程模型，更低的心智负担。

- Block 级别编程，SPMD 编程模型，更简单的任务划分;
- 定义在 SRAM (寄存器和 shared memory) 上的 tensor;
- 用于 tensor 的 primitive.(reduce, scan, sort, dot, ...);
- triton compiler 的优化使非 gpu 编程专家也能写出性能不俗的 kernel.

Triton 编译器代码开源，对多元 AI 芯片有更好的可移植性。

Triton 语言入门教程

01 Triton 语言简介

02 Triton 算子开发示例

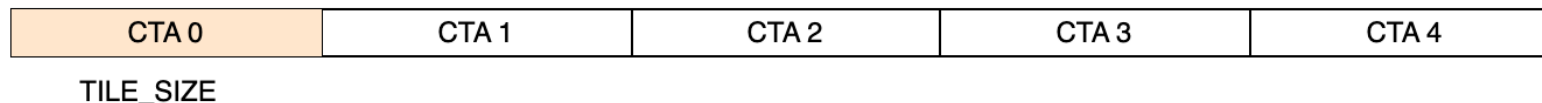
03 Triton 编译器

04 Triton 运行时

05 总结

Vector Add(1 tile/CTA)

1 tile/CTA



```
@triton.jit
```

```
def add_kernel(a_ptr, b_ptr, o_ptr, N, TILE_SIZE: tl.constexpr):
```

```
    pid = tl.program_id(0)
```

```
    offsets = pid * TILE_SIZE + tl.arange(0, TILE_SIZE)
```

```
    mask = offsets < N
```

```
    a = tl.load(a_ptr + offsets, mask=mask)
```

```
    b = tl.load(b_ptr + offsets, mask=mask)
```

```
    o = a + b
```

```
    tl.store(o_ptr + offsets, o, mask=mask)
```

```
def add(a, b):
```

```
    o = torch.empty_like(a)
```

```
    N = a.numel()
```

```
    TILE_SIZE = 128
```

```
    grid = (triton.cdiv(N, TILE_SIZE), 1, 1)
```

```
    add_kernel[grid](a, b, o, N, TILE_SIZE, num_warps=4)
```

```
    return o
```

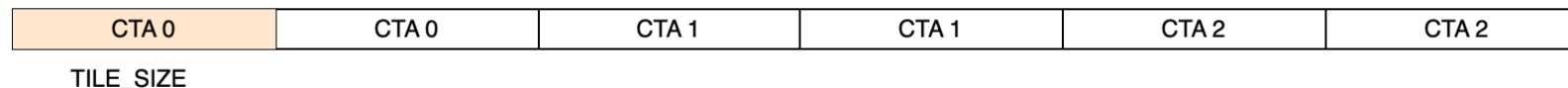
kernel: 描述一个 **CTA** 的运算。
各个 CTA 之间的差别在于 program_id.

没有细化到每个 thread 做什么。通过
range 生成 tensor.

wrapper: 指定 grid, 启动 kernel.

Vector Add(ntiles/CTA)

2 tiles/CTA



@triton.jit

```
def add_kernel(a_ptr, b_ptr, o_ptr, N, num_tiles_per_CTA, TILE_SIZE: tl.constexpr):  
    pid = tl.program_id(0)  
    program_offset = pid * num_tiles_per_CTA * TILE_SIZE
```

```
    offsets = program_offset + tl.arange(0, TILE_SIZE)
```

```
    for i in range(num_tiles_per_CTA):
```

```
        mask = offsets < N
```

```
        a = tl.load(a_ptr + offsets, mask=mask)
```

```
        b = tl.load(b_ptr + offsets, mask=mask)
```

```
        o = a + b
```

```
        tl.store(o_ptr + offsets, o, mask=mask)
```

```
        offsets += TILE_SIZE
```

```
def add(a, b):
```

```
    o = torch.empty_like(a)
```

```
    N = a.numel()
```

```
    num_tiles_per_CTA = 2
```

```
    TILE_SIZE = 128
```

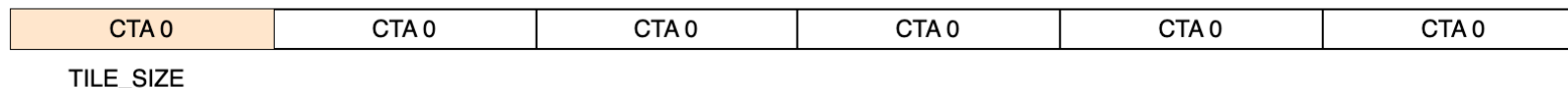
```
    grid = (triton.cdiv(N, num_tiles_per_CTA * TILE_SIZE), 1, 1)
```

```
    add_kernel[grid](a, b, o, N, num_tiles_per_CTA, TILE_SIZE)
```

```
    return o
```

Vector Add (1 CTA for all tiles)

all tiles for 1
CTA



@triton.jit

```
def add_kernel(a_ptr, b_ptr, o_ptr, N, TILE_SIZE: tl.constexpr):  
    pid = tl.program_id(0)
```

```
    offsets = tl.arange(0, TILE_SIZE)
```

```
    for _ in range(0, N, TILE_SIZE):
```

```
        mask = offsets < N
```

```
        a = tl.load(a_ptr + offsets, mask=mask)
```

```
        b = tl.load(b_ptr + offsets, mask=mask)
```

```
        o = a + b
```

```
        tl.store(o_ptr + offsets, o, mask=mask)
```

```
        offsets += TILE_SIZE
```

```
def add(a, b):
```

```
    o = torch.empty_like(a)
```

```
    N = a.numel()
```

```
    TILE_SIZE = 128
```

```
    grid = (1, 1, 1)
```

```
    add_kernel[grid](a, b, o, N, TILE_SIZE)
```

```
    return o
```

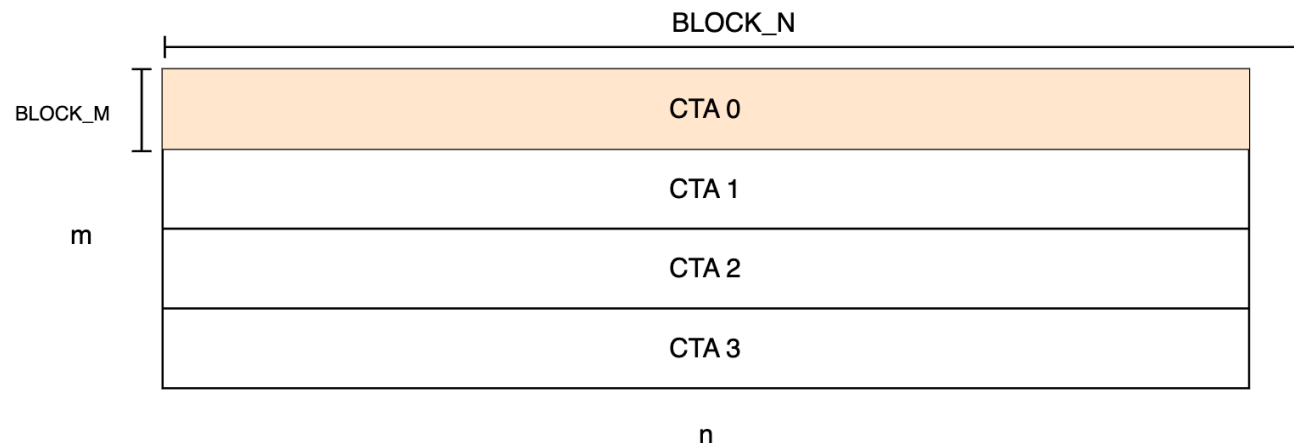
Q: Why bother,
为什么不用一个巨大的 TILE_SIZE 把整个
vector 都加载了?

A: 每一个 tensor 操作都会分到整个 CTA
上去。如果**计算**下来每个 thread 需要处理
多个元素, 这是会 unroll 的(如果用户没有
写 loop, triton 是不会帮你产生出 loop)。
因此寄存器的使用量也会增加。
因此 tensor 的最大尺寸是有限的。(tensor
on SRAM)

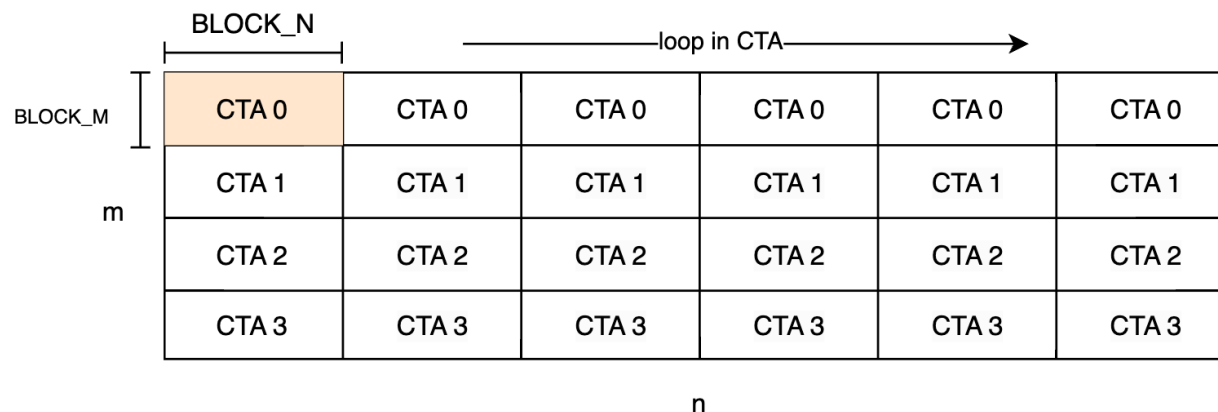
试试计算 $TILE_SIZE / (warp_size * num_warps)$
看看生成的 ptx 就可以发现

形状 (m, n) 的行主序矩阵，沿着最后一个维度求和。

任务划分方案 1:
一个 CTA 处理若干行，作为一个 TILE 处理完，不使用循环



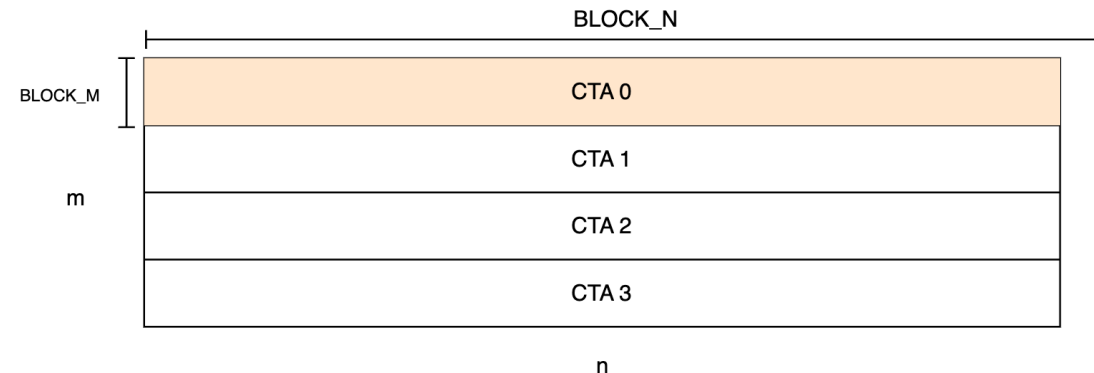
任务划分方案 2:
一个 CTA 处理若干行，每次处理 $(BLOCK_M, BLOCK_N)$ 的数据块
CTA 内包含循环



Reduce Sum(TILE 覆盖整行)

@triton.jit

```
def sum_kernel(output_ptr, input_ptr, M, N,  
              BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr,  
              ):  
    pid = tl.program_id(0)  
  
    m_offset = pid * BLOCK_M + tl.arange(0, BLOCK_M)  
    n_offset = tl.arange(0, BLOCK_N)  
    offset = m_offset[:, None] * N + n_offset[None, :]  
  
    m_mask = m_offset < M  
    n_mask = n_offset < N  
    mask = m_mask[:, None] & n_mask[None, :]  
  
    inp = tl.load(input_ptr + offset, mask=mask, other=0)  
    out = tl.sum(inp, axis=1)  
  
    tl.store(output_ptr + m_offset, out, mask=m_mask)
```



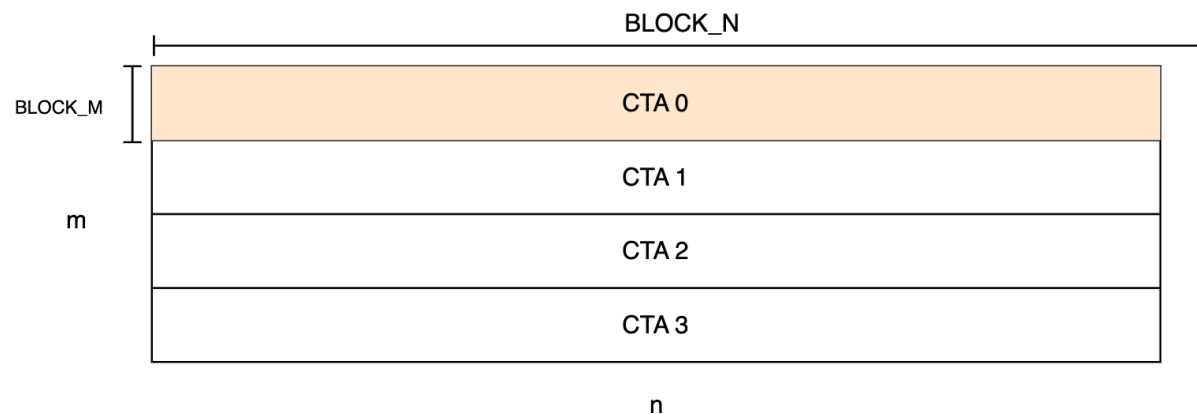
一个 CTA 计算 BLOCK_M 行

使用 2D tensor: 注意 broadcast 语法，和 numpy/torch 是类似的。

tl 有 cooperative primitive

Reduce Sum (TILE 覆盖整行)

一个 CTA 计算 BLOCK_M 行



```
def sum(x):  
    m, n = x.shape  
    out = torch.empty(x.shape[:-1], dtype=x.dtype, device=x.device)
```

```
    BLOCK_M = 4  
    BLOCK_N = triton.next_power_of_2(n)  
    grid = triton.cdiv(m, BLOCK_M), 1, 1
```

```
    sum_kernel[grid](out, x, m, n, BLOCK_M, BLOCK_N)  
    return out
```

Q: 怎么保证一个数据块能加载完一整行?

tl.arange 要求 size 需要是 2 的幂。

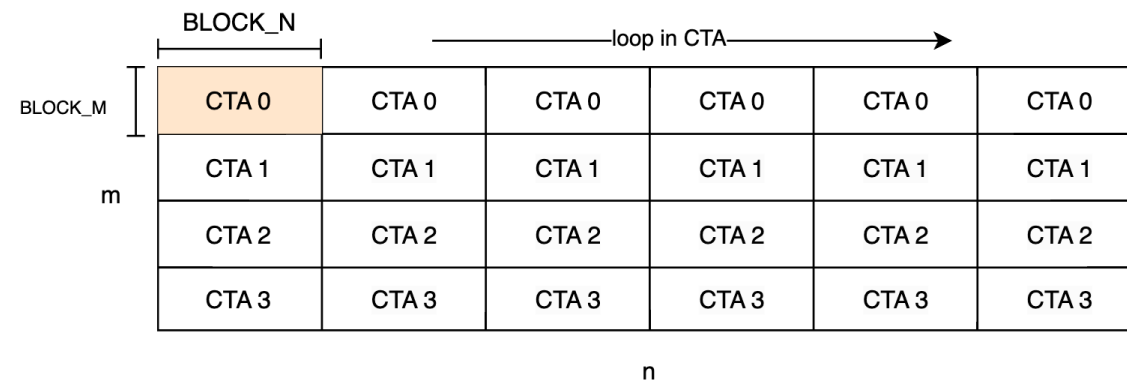
Reduce Sum (CTA 内循环)

```
@triton.jit
def sum_kernel(output_ptr, input_ptr,
               M, N,
               BLOCK_M: tl.constexpr,
               BLOCK_N: tl.constexpr,
               ):
    pid_m = tl.program_id(0)
    m_offset = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    m_mask = m_offset < M

    acc = tl.zeros([BLOCK_M, BLOCK_N], dtype=tl.float32)
    for start_n in range(0, N, BLOCK_N):
        n_offset = start_n + tl.arange(0, BLOCK_N)
        n_mask = n_offset < N

        offset = m_offset[:, None] * N + n_offset[None, :]
        mask = m_mask[:, None] & n_mask[None, :]
        inp = tl.load(input_ptr + offset, mask=mask, other=0)
        acc += inp

    out = tl.sum(acc, axis=1)
    tl.store(output_ptr + m_offset, out, mask=m_mask)
```



数据块大小为 (BLOCK_M, BLOCK_N), 一个 CTA 处理若干块, 直到一整行完结。

块之间 add, 最后再 sum

kernel launch 任务划分

grid = triton.cdiv(m, BLOCK_M), 1, 1

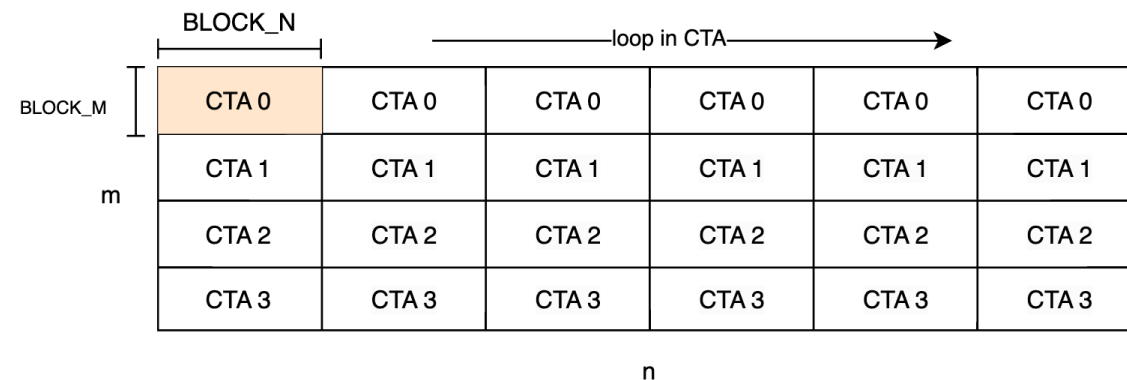
Reduce Sum (CTA 内循环)

```
@triton.jit
def sum_kernel(output_ptr, input_ptr,
              M, N,
              BLOCK_M: tl.constexpr,
              BLOCK_N: tl.constexpr,
              ):
    pid_m = tl.program_id(0)
    m_offset = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    m_mask = m_offset < M

    acc = tl.zeros([BLOCK_M], dtype=tl.float32)
    for start_n in range(0, N, BLOCK_N):
        n_offset = start_n + tl.arange(0, BLOCK_N)
        n_mask = n_offset < N
        offset = m_offset[:, None] * N + n_offset[None, :]
        mask = m_mask[:, None] & n_mask[None, :]

        inp = tl.load(input_ptr + offset, mask=mask, other=0)
        acc += tl.sum(inp, axis=1)

    tl.store(output_ptr + m_offset, acc, mask=m_mask)
```



数据块大小为 (BLOCK_M, BLOCK_N), 一个 CTA 处理若干块, 直到一整行完结。

块内 reduce sum, 然后累加结果

kernel launch 任务划分

grid = triton.cdiv(m, BLOCK_M), 1, 1

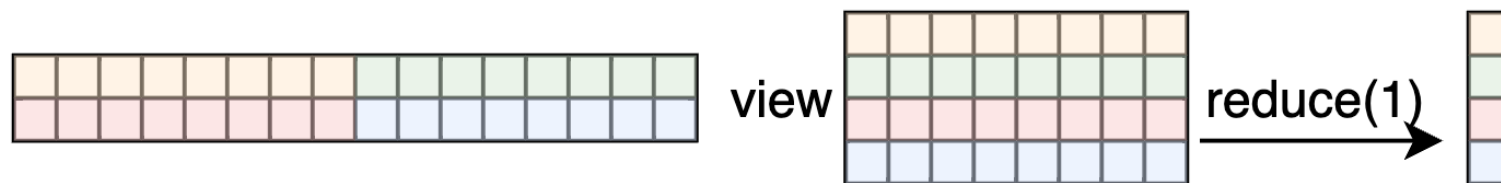
Reduce Sum: two pass reduction

(m, n) 沿 axis 1 求和, n 较大, m 相对较小 (仅利用 m 维度上的并行性无法充分利用 GPU 的 SM 时使用)。使用两次 reduce, 调用两个 Kernel.

step1:

输入视为 $(m * n / K, K)$,

沿 axis 1 求和 $\rightarrow (m * n / K,)$



step2:

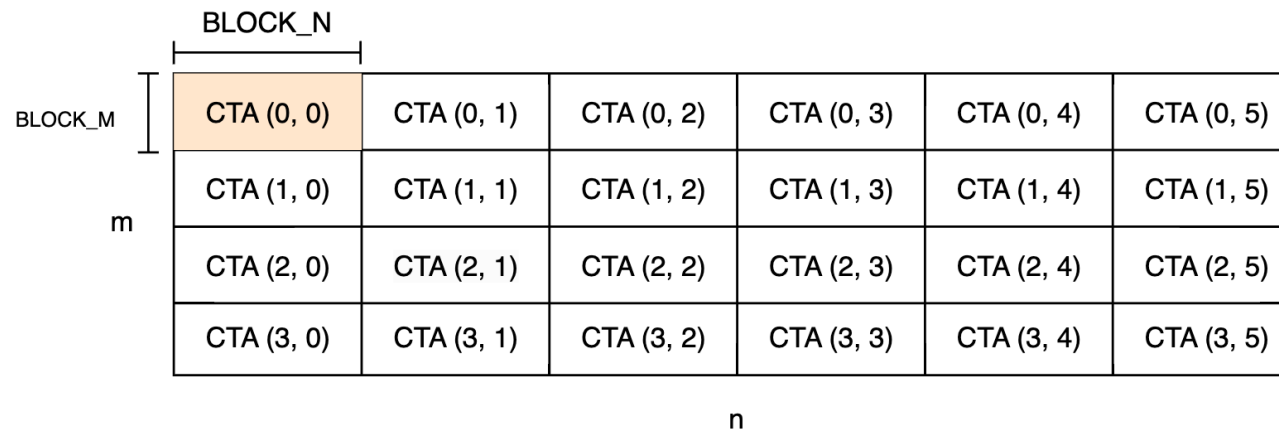
结果视为 $(m, n / K)$,

沿 axis 1 求和, $\rightarrow (m,)$



Reduce Sum: reduction with atomic op

每个 CTA reduce 自身处理的区域，然后 atomic_add 跨 CTA 相加。



```
grid = triton.cdiv(m, BLOCK_M), triton.cdiv(n, BLOCK_N), 1
```

triton 提供了 tensor 上的 primitive 以及 SPMD 的编程模型，但是留给开发者发挥的机会仍然很多。可以尝试不同的方法。

行主序的 (M, K) 矩阵乘行主序的 (K, N) 矩阵，输出到 (M, N) 行主序矩阵。

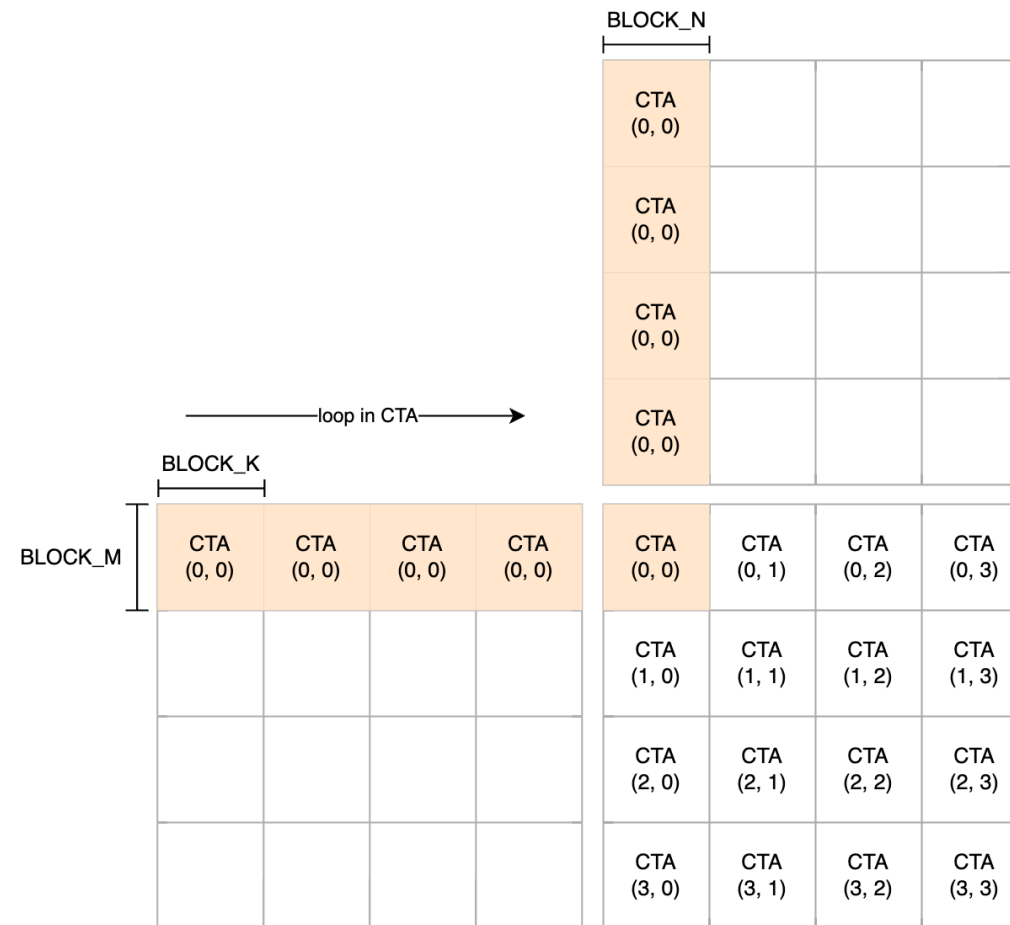
```
@triton.jit
def mm_kernel(
    A, B, C,
    M, N, K,
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
):
    # 2d -grid
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)

    offsets_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    mask_m = offsets_m < M
    offsets_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    mask_n = offsets_n < N

    # 2d tile
    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
    for start_k in range(0, K, BLOCK_K):
        offsets_k = start_k + tl.arange(0, BLOCK_K)
        mask_k = offsets_k < K
        a_ptrs = A + offsets_m[:, None] * K + offsets_k[None, :]
        mask_a = mask_m[:, None] & mask_k[None, :]
        b_ptrs = B + offsets_k[:, None] * N + offsets_n[None, :]
        mask_b = mask_k[:, None] & mask_n[None, :]

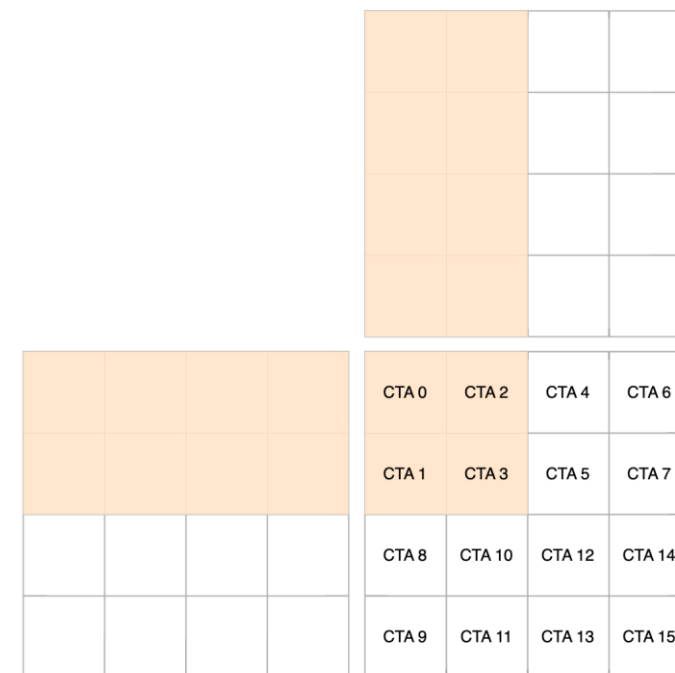
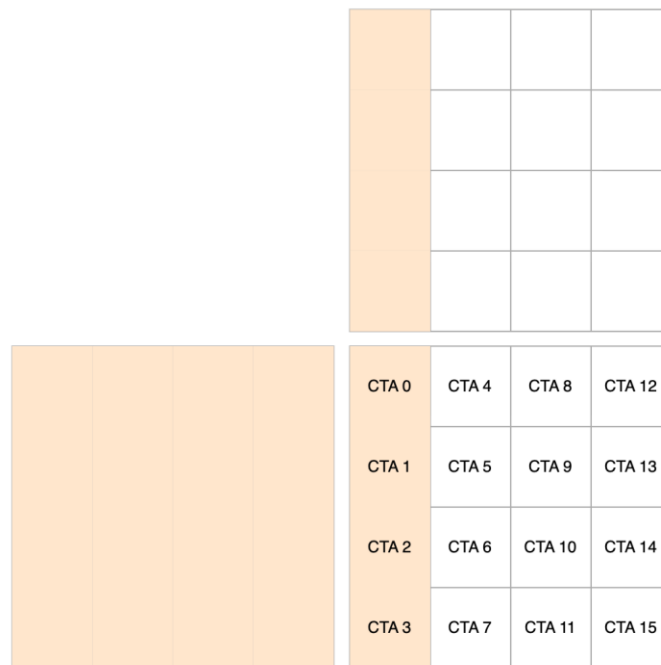
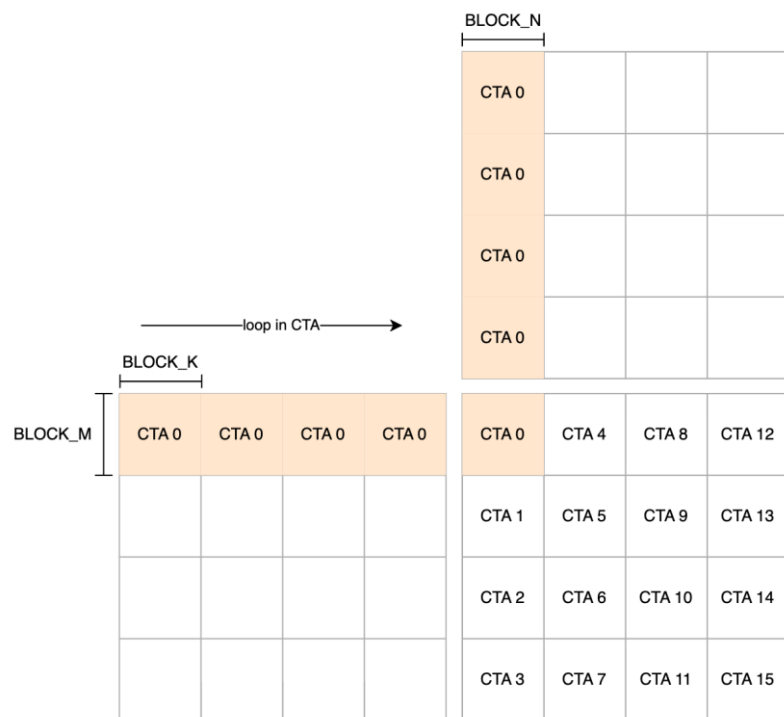
        a = tl.load(a_ptrs, mask=mask_a, other=0)
        b = tl.load(b_ptrs, mask=mask_b, other=0)
        acc += tl.dot(a, b)

    c_ptrs = C + offsets_m[:, None] * N + offsets_n
    mask_c = mask_m[:, None] & mask_n[None, :]
    tl.store(c_ptrs, acc, mask=mask_c)
```



grid = triton.cdiv(M, BLOCK_M), triton.cdiv(N, BLOCK_N), 1

Matmul: Optimize for L2 locality



CTA 的编号和启动顺序是有关的。
xyz, x 变动最快。所以前面的写法实际相当于列主序的方式排列 CTA.

读 16 + 4 块, 写 4 块

读 8 + 8 块, 写 4 块
提升 L2 cache 命中率

重排 CTA

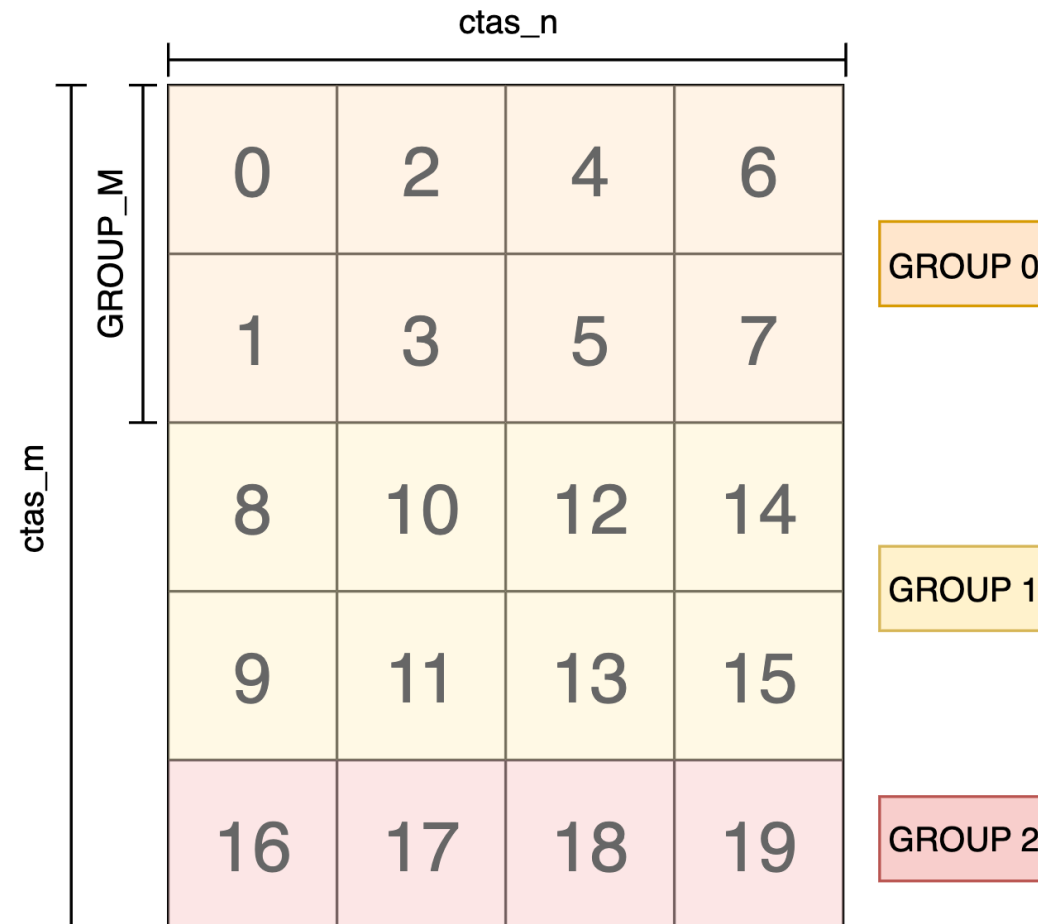
```
ctas_m, ctas_n = tl.cdiv(M, BLOCK_M), tl.cdiv(N, BLOCK_N)
ctas_per_group = GROUP_M * ctas_n
```

```
pid = tl.program_id(0)
group_id = pid // ctas_per_group
inner_group_id = pid % ctas_per_group
```

```
start_pid_m = group_id * GROUP_M
group_m_size = min(ctas_m - start_pid_m, GROUP_M)
pid_m = start_pid_m + inner_group_id % group_m_size
pid_n = inner_group_id // group_m_size
```

其他 CTA 层任务划分的优化手段:

- 调节 BLOCK_M, BLOCK_N, BLOCK_K 的大小
- split-k, K 维度划分, 用不同的 CTA 来计算, 当 M, N 小而 K 很大的时候适用。用 atomic 的方式累加结果。
- 用类似 two-pass reduction 的方式启动多个 Kernel 分阶段求和。



Triton 中有很多优化用于优化矩阵乘运算。

- 使用 tensor core 的 mma 指令。
- 使用 global -> shared 的 ldgsts 指令，不经过寄存器直接复制到 shared memory。
- software pipelining:
 - global -> shared (triton 中称为 Pipeliner, 使用 **num_stages** 参数控制)
 - shared -> register (triton 中称为 Prefetch)
- swizzle: shared memory 重排与 bank conflict 消除

更多的信息请参考 tensor core 相关的文档与 cutlass 的文档。

triton JIT Function 的参数中有一部分是编译器常量参数，用 **tl.constexpr** 标记，这部分参数类似模板参数，直接编译进 kernel 里，不再出现在 kernel 参数列表。

- 比如 mm_kernel 里的 BLOCK_M, BLOCK_N, BLOCK_K, GROUP_M.

还有部分不出现在 JIT Function 定义中，比如

- num_warps: 决定 CTA 中的 warp 数;
- num_stages: 决定 for 循环里 tl.dot 运算的 global -> shared 软流水 stage 数，影响共享内存的使用量。

但可以在调用 jit function 的时候传。

这些组成了一组配置 triton.Config. 用于 Tunning.

```
@triton.autotune(
```

cofigs: 备选配置

```
    configs=[
        triton.Config({"BLOCK_M": 128, "BLOCK_N": 128, "BLOCK_K": 64, "GROUP_M": 16}, num_stages=3, num_warps=8),
        triton.Config({"BLOCK_M": 128, "BLOCK_N": 64, "BLOCK_K": 32, "GROUP_M": 8}, num_stages=4, num_warps=4),
        triton.Config({"BLOCK_M": 128, "BLOCK_N": 128, "BLOCK_K": 32, "GROUP_M": 4}, num_stages=4, num_warps=4),
        triton.Config({"BLOCK_M": 128, "BLOCK_N": 64, "BLOCK_K": 32, "GROUP_M": 2}, num_stages=4, num_warps=4)
    ],
    key=["M", "N", "K"]
)
```

key: 指明 jit function 的哪些运行时参数影响配置选取

```
@triton.jit
```

```
def mm_kernel(
```

```
    A, B, C,
    M, N, K,
    BLOCK_M: tl.constexpr,
    BLOCK_N: tl.constexpr,
    BLOCK_K: tl.constexpr,
    GROUP_M: tl.constexpr,
):
```

```
def mm(a, b):
```

如果 grid 需要由 jit function 的静态参数决定, 为了使用 autotune, 需要把 grid 写成函数 (args -> grid)的形式。

```
    ....
    def grid(args):
        return triton.cdiv(M, args["BLOCK_M"]) * triton.cdiv(N, args["BLOCK_N"]), 1, 1
    mm_kernel[grid](
        a, b, c,
        M, N, K,
    )
    return c
```

调用 jit function 不需要再传 config 里制定的参数, autotuner 会填上。

triton(with group_m): 1897.4720239639282ms

torch.mm: 1969.1519737243652ms

BLOCK_M: 128, BLOCK_N: 128, BLOCK_K: 64, GROUP_M: 16, num_warps: 8, num_ctas: 1, num_stages: 3, enable_warp_specialization: False, enable_persistent: False => timing: 2.151423931121826

BLOCK_M: 128, BLOCK_N: 64, BLOCK_K: 32, GROUP_M: 8, num_warps: 4, num_ctas: 1, num_stages: 4, enable_warp_specialization: False, enable_persistent: False => timing: 1.9194880326588948

BLOCK_M: 128, BLOCK_N: 128, BLOCK_K: 32, GROUP_M: 4, num_warps: 4, num_ctas: 1, num_stages: 4, enable_warp_specialization: False, enable_persistent: False => timing: 1.887436827023824

BLOCK_M: 128, BLOCK_N: 64, BLOCK_K: 32, GROUP_M: 2, num_warps: 4, num_ctas: 1, num_stages: 4, enable_warp_specialization: False, enable_persistent: False => timing: 1.9714730580647786

BLOCK_M: 64, BLOCK_N: 128, BLOCK_K: 32, GROUP_M: 8, num_warps: 4, num_ctas: 1, num_stages: 4, enable_warp_specialization: False, enable_persistent: False => timing: 1.9225887854894002

BLOCK_M: 128, BLOCK_N: 32, BLOCK_K: 32, GROUP_M: 8, num_warps: 4, num_ctas: 1, num_stages: 4, enable_warp_specialization: False, enable_persistent: False => timing: 2.4984916845957437

BLOCK_M: 64, BLOCK_N: 32, BLOCK_K: 32, GROUP_M: 8, num_warps: 2, num_ctas: 1, num_stages: 5, enable_warp_specialization: False, enable_persistent: False => timing: 2.7368106047312417

BLOCK_M: 32, BLOCK_N: 64, BLOCK_K: 32, GROUP_M: 16, num_warps: 2, num_ctas: 1, num_stages: 5, enable_warp_specialization: False, enable_persistent: False => timing: 2.726024548212687

Trick: 根据 m, n, k 是否能整除对应维度的 `TILE_SIZE` 来决定代码中是否使用 `masking`. 是否使用 `mask` 也可以作为静态信息影响 Kernel 的行为。虽然 m, n, k 是运行时信息。但是可以生成带 `mask` 和不带 `mask` 的不同的 kernel, 再选择其中一个。

可以根据 Autotune 结果给出的 `TILE_SIZE` 来选择使用是否带 `mask` 的版本。

FlagGems 里就使用了这样的技巧。把 Autotune 套在 heuristics 外面。

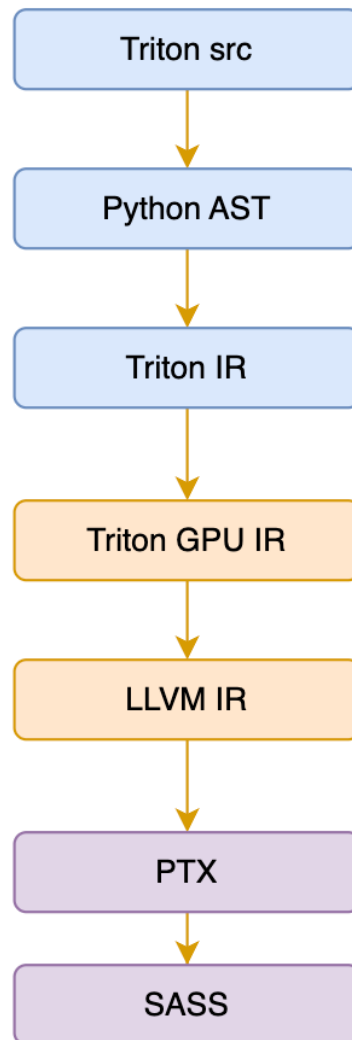
简而言之是使用一些参数来计算另一些参数。

```
@triton.heuristics(  
    {  
        "DIVISIBLE_M": lambda args: args["M"] % args["TILE_M"] == 0,  
        "DIVISIBLE_N": lambda args: args["N"] % args["TILE_N"] == 0,  
        "DIVISIBLE_K": lambda args: args["K"] % args["TILE_K"] == 0,  
    }  
)  
@triton.jit  
def bmm_kernel(  
    A, B, O, M, N, K,  
    TILE_M: tl.constexpr,  
    TILE_N: tl.constexpr,  
    TILE_K: tl.constexpr,  
    GROUP_M: tl.constexpr,  
    DIVISIBLE_M: tl.constexpr,  
    DIVISIBLE_N: tl.constexpr,  
    DIVISIBLE_K: tl.constexpr,  
):
```

Triton 语言入门教程

- 01 Triton 语言简介
- 02 Triton 算子开发示例
- 03 Triton 编译器**
- 04 Triton 运行时
- 05 总结

lowering 过程中的 IR 转换



Triton IR 基本和 Triton 语言一一对应。

Convert to TritonGPU IR: 给每个 tensor 分配 Layout.(开始涉及 CTA 内任务划分)

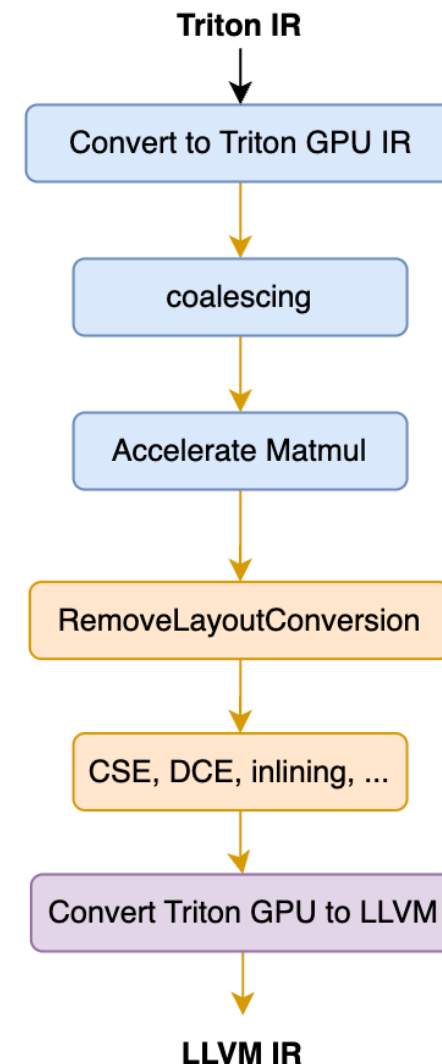
coalescing: 针对 load 和 store 修改 layout, 合并全局访存

Accelerate Matmul: 修改 layout 以使用 tensor core, 软流水, bank conflict 去除。

RemoveLayoutConversion: 沿着数据流传播 layout, 消除不必要的 layout 转换

Convert Triton GPU to LLVM: 生成 SIMT 的 LLVMIR。添加 shared memory 和 barrier.

针对算子的一些成熟的优化模式实现到了编译器里, 所有很多本来需要 kernel 开发者完成的事情由编译器代劳了。



在 nvidia 后端上, Triton 编译器生成了 LLVM IR 之后就由 LLVM 的 nvgpu 后端编译为 nvptx 再由 ptx 汇编器生成 cubinary. 而 cubinary 运行仍然是使用 nvidia 的 Driver API 来调用, 这和使用 CUDA C 编程基本是类似的。

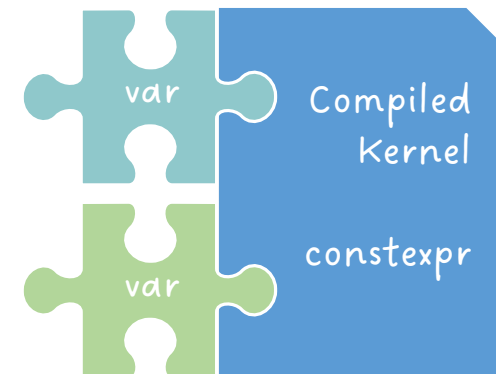
而从 Triton 的 jit function 到 设备的 driver API 之间的衔接, 则主要是 Triton 的运行时相关的 Topic.

Triton 语言入门教程

- 01 Triton 语言简介
- 02 Triton 算子开发示例
- 03 Triton 编译器
- 04 Triton 运行时**
- 05 总结

Triton Runtime

- `tl.constexpr`
 - 在kernel中特化的固定参数，编译时已知
 - 通常需要在函数声明中以annotation形式声明
 - 可以是：常数 / 常数列表 / 常数元组 / 表达式
 - 可以参与：基本数学运算 / 比较 / 逻辑运算 / 位运算
- 无需声明也会成为被特化的变量
 - 张量数据类型
 - `num_warps` / `num_ctas` / `num_stages`



```
@triton.jit
def add_kernel(x_ptr, # *Pointer* to first input vector.
               y_ptr, # *Pointer* to second input vector.
               output_ptr, # *Pointer* to output vector.
               n_elements, # Size of the vector.
               BLOCK_SIZE: tl.constexpr, # Number of elements each program should process.
               # NOTE: `constexpr` so it can be used as a shape value.
               ):
    # ...
```



Config

kernel的可选配置参数，包括constexpr和num_warps / num_ctas / num_stages



Autotuner (optional)

使用自动调优方法选择kernel的配置参数Config



Heuristics (optional)

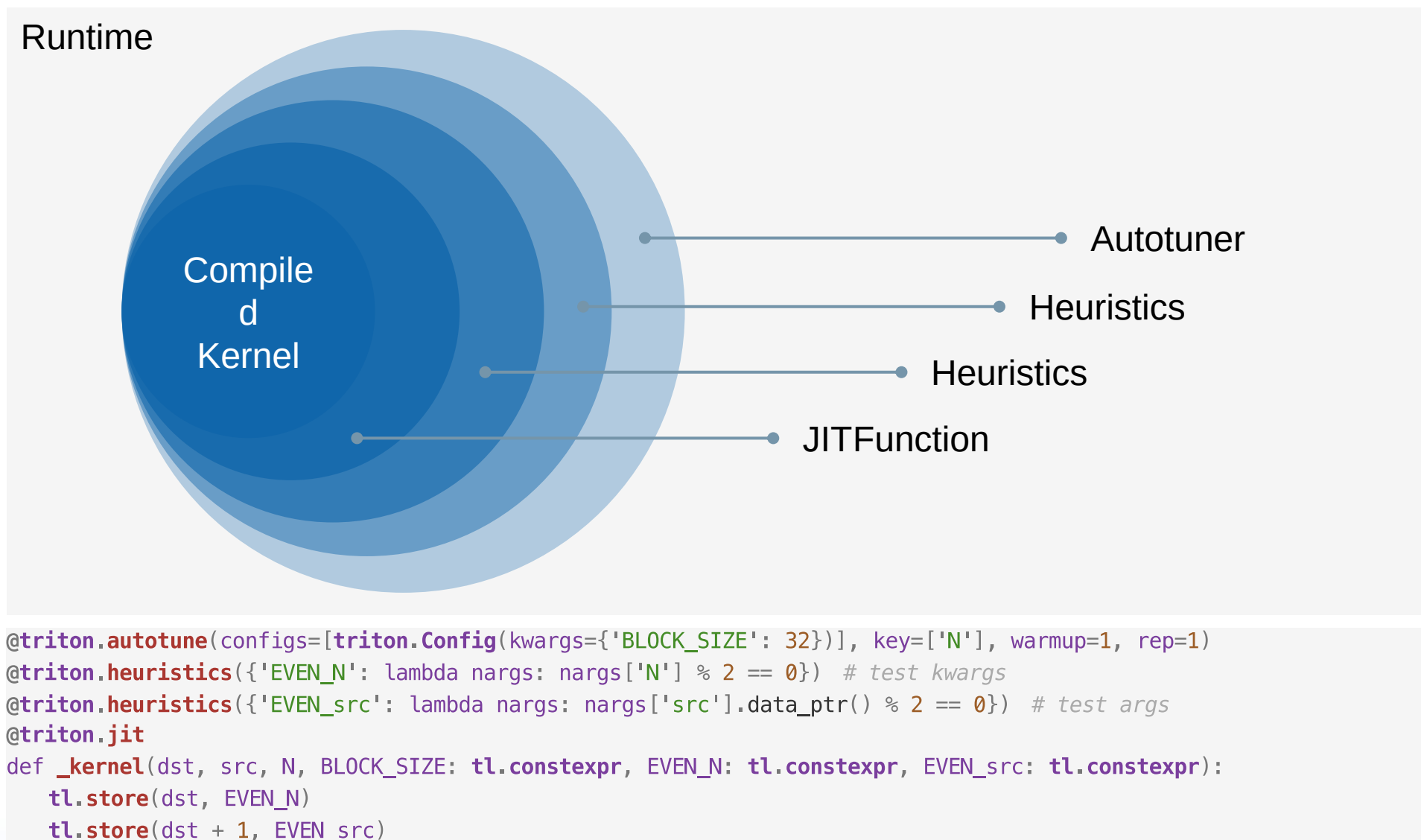
使用启发式方法计算kernel配置参数constexpr



JITFunction (required)

执行kernel的即时编译

三种方式均可指定编译配置参数：Autotuner / Heuristics / 运行时传参



- Autotuner

- 通过编译、运行和测定，来选择耗时最小的配置与kernel
- 可以自定义剪枝策略在调优前缩小配置空间
- 参数
 - 可选的常量配置空间：list of Config
 - 用于选择配置的变量键值：list of runtime arguments
 - 测定kernel所需的循环配置：warmup, rep
 - 剪枝可用的策略模型：perf_model, top_k, early_config_prune
- 默认参数
 - num_warps=4, num_stages=2, num_ctas=1
 - num_ctas只面向SM 90以上的架构

```
@triton.autotune(  
    configs=[  
        triton.Config({  
            'BLOCK_SIZE_M': 128,  
            'BLOCK_SIZE_N': 128,  
            'BLOCK_SIZE_K': 32,  
            'NUM_SM': 84,  
        }),  
        triton.Config({  
            'BLOCK_SIZE_M': 128,  
            'BLOCK_SIZE_N': 128,  
            'BLOCK_SIZE_K': 32,  
            'NUM_SM': 128,  
        }),  
        triton.Config({  
            'BLOCK_SIZE_M': 64,  
            'BLOCK_SIZE_N': 64,  
            'BLOCK_SIZE_K': 32,  
            'NUM_SM': 84,  
        }),  
        triton.Config({  
            'BLOCK_SIZE_M': 64,  
            'BLOCK_SIZE_N': 64,  
            'BLOCK_SIZE_K': 32,  
            'NUM_SM': 128,  
        }),  
    ],  
    key=['group_size'],  
)
```

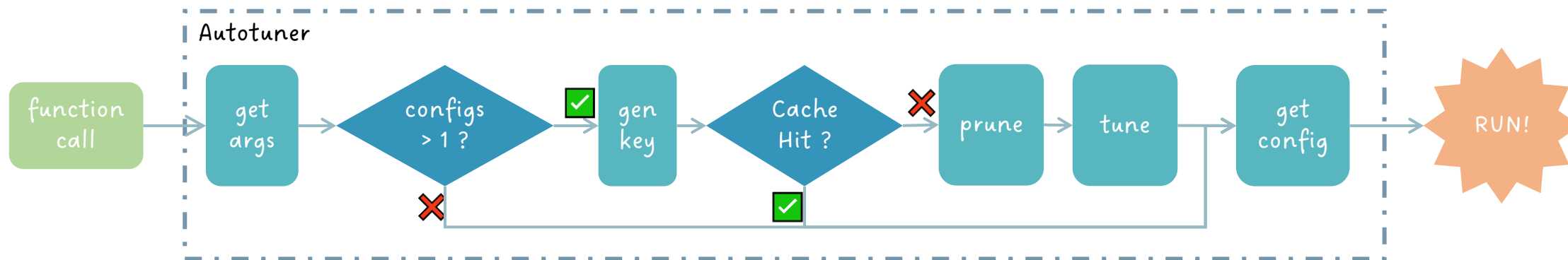
- Autotuner

- 配置缓存config cache

- Key: 键值及其数据类型
 - Value: 调优获得的最佳配置参数

- 跳过条件

- 相同键值可读取缓存, 无需重新调参
 - 参数空间中Config配置唯一, 无需调参



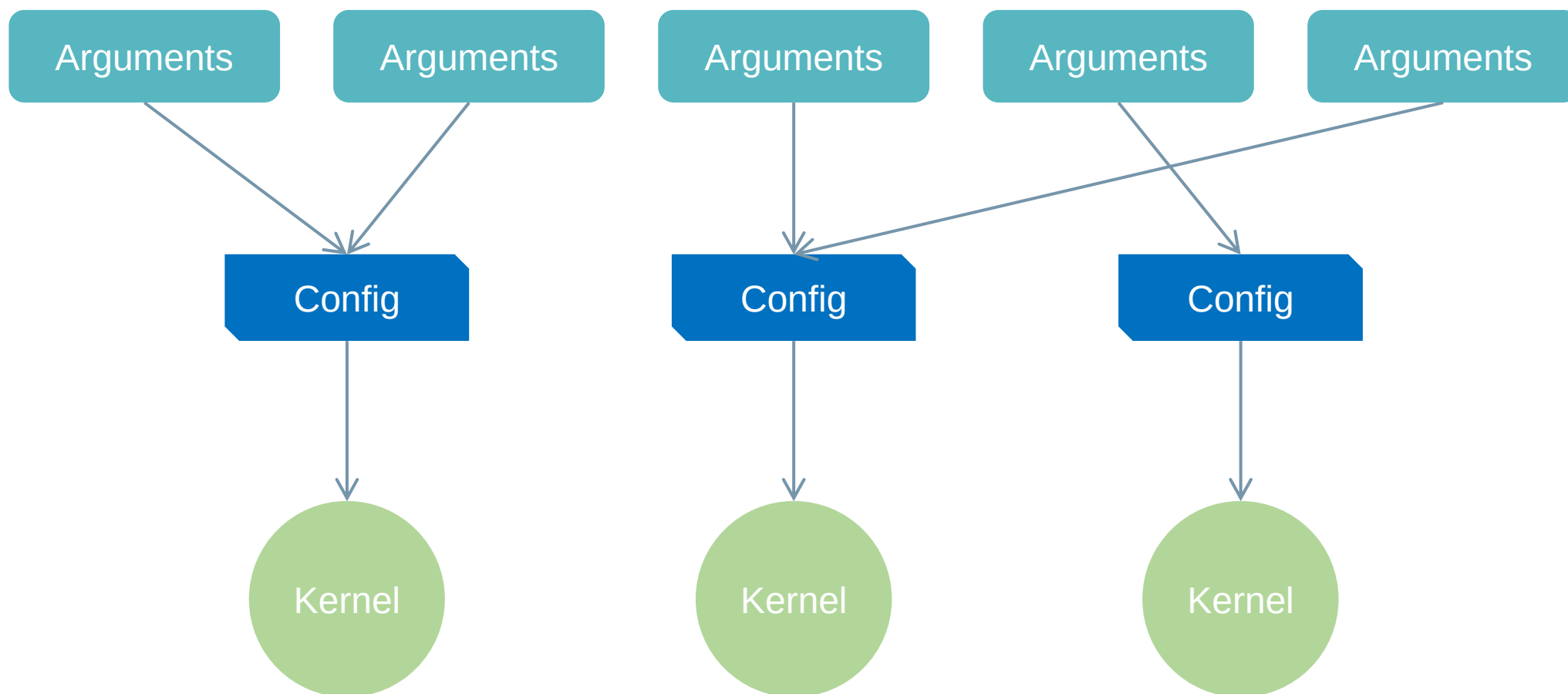
- Heuristics

- 给定所需配置参数，以及启发式的计算表达式
- 可使用运行时传入参数进行计算
- 当heuristics装饰在autotune内层，也可以使用调优所得参数进行计算
- values为字典形式，key为参数名，value为可调用的表达式
- 在调参开销过大的场景下，可以使用启发式方法迅速获得参数

```
@triton.heuristics(  
    values={  
        "BLOCK_N": lambda args: triton.next_power_of_2(args["N"]),  
        "num_warps": lambda args: (  
            4 if args["N"] <= 1024 else (8 if args["N"] <= 2048 else 16)  
        ),  
    },  
)
```

- JITFunction

- 维护kernel cache: 缓存已编译的kernel
- 键值构造: 调用inspect绑定参数
 - 参数名
 - 常量参数值、特化参数值
 - 非常量参数值及其annotation数据类型
- v2键值 (在v3得到优化)
 - Cuda版本号
 - 特化参数: 张量地址对齐, 标量数值对齐
- 缓存键值可直接取出kernel运行
- 新键值需编译kernel, 缓存后运行
- 其他
 - 参数包装、设备检查、stream检查



Triton 语言入门教程

- 01 Triton 语言简介
- 02 Triton 算子开发示例
- 03 Triton 编译器
- 04 Triton 运行时
- 05 总结**

Triton 提供了 CTA 级别的 SPMD 编程模型。

- 定义在 SRAM 上的 tensor 和用于 tensor 的 primitive
- 编译器完成 SPMD 到 SIMT 的转换

Triton 可以降低 gpu 编程的心智负担

- 开发可以集中注意力在算法和 CTA 层的任务划分上，善用 Autotuner 。
- Triton 帮助 AI 研究者快速尝试新的算法

Triton 开源

- 利于支持多元 AI 芯片
- 利于 AI 学术研究成果更快地在多种 AI 芯片的平台落地



谢谢聆听!

欢迎访问开源社区:

<https://github.com/FlagOpen/FlagGems>

<https://github.com/FlagOpen/FlagAttention>

Triton中国生态Meetup反馈收集表



Triton中国社区微信群



扫码添加微信，私信“加群”